

Data-pipelines basados en computación en la nube para el análisis, procesamiento y modelado de operaciones de mantenimiento

20 octubre 2025



Universidad
Internacional
de Valencia

Titulación:

MBID

Curso académico

2024 – 2025

Alumno/a:

Palacios Merino, Ernesto

D.N.I: 1104059215

Director/a de TFM:

Yudith Cardinale

Convocatoria:

Tercera

De:

 Planeta Formación y Universidades

Índice

Resumen	8
1. Introducción	9
1.1. Problemática.	9
1.2. Objetivos	10
1.2.1. Objetivo General.....	10
1.2.2. Objetivos Específicos	10
2. Marco Teórico.....	11
2.1. El ciclo de vida de los datos	11
2.2. Paradigmas fundacionales	14
2.2.1. Diferencias entre ETL y ELT	14
2.2.2. El Procesamiento de datos	16
2.3. Arquitecturas tradicionales para el manejo de datos.....	17
2.3.1. Arquitectura Lambda	17
2.3.2. Arquitectura Kappa:	18
2.4. La evolución de los sistemas de gestión y análisis de datos.....	19
2.4.1. Arquitectura Medallón	20
2.5. Plataforma de transmisión distribuida	23
2.5.1. Apache Kafka	24
3. Sistema de Gestión de Órdenes de Trabajo	26
3.1. El origen de los datos	27
3.1.1. Procesamiento de texto	30
3.2. Arquitectura del sistema	31
3.2.1. La ingesta de datos	32
3.2.2. Organización de documentos	33
3.2.3. Consolidación de actividades	33
3.2.4. Limpieza de datos.....	34
3.3. Solución propuesta.....	36
3.3.1. Configuración de Apache Kafka.....	37
3.4. Extracción de datos (productor).....	38
3.4.1. Configuración de MongoDB	41
3.5. Carga de datos (consumidor)	42
3.5.1. Configuración de Delta Lake.....	43

3.6.	Transformación de los datos	44
3.6.1.	Procesamiento Natural del Lenguaje (NLP)	45
3.6.2.	Limpieza manual de actividades	48
4.	Evaluación del sistema	50
4.1.	Objetivos de la Evaluación	50
4.2.	Configuración del entorno de pruebas	50
4.2.1.	Dataset de Evaluación	50
4.2.2.	Infraestructura de Hardware	52
4.3.	Métricas de Rendimiento del Pipeline	53
4.3.1.	Evaluación del Clasificador Naive Bayes	56
5.	Conclusión	60
5.1.	Cumplimiento de objetivos específicos	60
5.2.	Contribuciones Metodológicas	62
5.3.	Limitaciones y Desafíos Encontrados	63
5.3.1.	Integración con Sistemas Propietarios	63
5.3.2.	Desbalance de Clases en Dataset de Entrenamiento	63
5.3.3.	Ausencia de Validación en Campo	64
6.	Trabajos Futuros	65
6.1.1.	Mejoras al Modelo de Clasificación	65
6.1.2.	Implementación de Gobernanza de Datos	66
6.1.3.	Sistemas Tabulares Alternativos	67
6.1.4.	Implementación de metodología CI/CD	68
6.2.	Reflexión final	70
7.	Referencias	72
	Apéndice I: Implementación del mecanismo <i>Heartbeat</i>	74
	Apéndice II: Desarrollo del Clasificador Naive Bayes	77
	Apéndice III: Reporte de Orden de Trabajo	84
	Anexo I: El presente y futuro de PySpark	85
	Anexo II: Reflexión acerca del pronóstico de fallas eléctricas	89
	Visión y Alcance del Modelo Predictivo	89
	Delimitación del Caso de Estudio: Alimentador Primario Zamora I	90
	Perspectiva Climatológica: El Factor Ambiental	91
	Variables del Modelo Predictivo	92



Enfoque Metodológico: Análisis de Series Temporales Multivariadas	93
Más Allá del Pronóstico Temporal: Localización Espacial de Fallas	94
Perspectiva de Desarrollo: Un Programa de Investigación de Largo Plazo	95

Índice de ilustraciones

Ilustración 1. "The amazing Data pipeline". Fuente (ByteByteGo, 2024).....	13
Ilustración 2. La Arquitectura Lambda. Fuente (Feick, Kleer, & Kohn, 2018)	18
Ilustración 3. La Arquitectura Kappa. Fuente (Feick, Kleer, & Kohn, 2018).....	18
Ilustración 4. Descripción general de la Arquitectura Medallón. Fuente (Databricks Inc., 2025)	21
Ilustración 5. Arquitectura Medallón. Fuente (Strengtholt, 2025).....	22
Ilustración 6. Orden de Trabajo típica. Elaboración propia.....	28
Ilustración 7. Arquitectura del Sistema de Gestión de Ordenes de Trabajo para la EERSSA. Elaboración propia.....	35
Ilustración 8. Diagrama del sistema de distribución eléctrico. Elaboración propia	45
Ilustración 9. Ejemplo de interfaz de usuario en Marimo Notebooks. Elaboración propia.	49
Ilustración 10. Captura del monitor de procesamiento en paralelo DASK. Elaboración propia.	55
Ilustración 11. Matriz de confusión del modelo clasificador de actividades. Elaboración propia.	59
Ilustración 12. Concepto de ventanas de saltos (tumbling windows). Fuente (Apache Software Foundation, 2025).....	74
Ilustración 13. Diagrama de flujo de la implementación del algoritmo heartbeat. Elaboración propia.....	76
Ilustración 14. Captura del archivo con los datos de entrenamiento. Elaboración propia	79
Ilustración 15. código python para la transformación del texto de la actividad. Elaboración propia.....	81
Ilustración 16. Transformación del texto con corrección de palabras. Elaboración propia.	81



Ilustración 17. Muestra de dataset con todas las transformaciones del texto aplicadas. Elaboración propia.....	82
Ilustración 18. Captura del proceso de entrenamiento del clasificador Naive Bayes. Elaboración propia.....	82
Ilustración 19. Captura de la evaluación del modelo clasificador. Elaboración propia.	83
Ilustración 20. Ejemplo de "Reporte de Orden de Trabajo". Elaboración propia	84

Índice de tablas

Tabla 1. Resumen de diferencias entre ETL y ELT. Fuente (Amazon Web Services, 2025)	15
Tabla 2. Stack tecnológico base para el desarrollo del Sistema de órdenes de trabajo. Elaboración propia	26
Tabla 3. Niveles de criticidad en el reporte de orden de trabajo. Elaboración propia ..	39
Tabla 4. Funcionalidades y sus librerías en el microservicio productor. Elaboración propia.	41
Tabla 5. Funcionalidades y sus librerías en el microservicio consumidor. Elaboración propia	43
Tabla 6. Funcionalidades y sus librerías en el microservicio concatenador. Elaboración propia	47
Tabla 7. Metadatos de documentos para evaluación del sistema. Elaboración propia.	51
Tabla 8. Características del entorno de evaluación del sistema. Elaboración propia. ..	52
Tabla 9. Metadatos de la primera OT procesada. Elaboración propia	53
Tabla 10. Metadatos de la última OT procesada. Elaboración propia	54
Tabla 11. Métricas de rendimiento del sistema desarrollado. Elaboración Propia.	54
Tabla 12. Evaluación de métricas de procesamiento en paralelo. Elaboración Propia.	56
Tabla 13. Conversión de cuenta contable a tipo de actividad. Elaboración propia	57
Tabla 13. Desempeño por clases del modelo clasificador Naive Bayes. Elaboración propia.	57
Tabla 15. Cuentas contables para clasificación de actividades. Elaboración propia. ..	78
Tabla 16. Cantidad de eventos para cada Cuenta Contable. Elaboración propia.	80
Tabla 17. Cantidad de eventos luego de balancear el Dataset. Elaboración propia	80

Resumen

La gestión de datos operativos en empresas del sector eléctrico enfrenta desafíos significativos debido al procesamiento manual de información crítica. En la Empresa Eléctrica Regional del Sur S.A. (EERSSA), las operaciones de mantenimiento generan diariamente documentos denominados "Órdenes de Trabajo" que contienen información valiosa sobre actividades correctivas, preventivas y predictivas en la red de distribución eléctrica. Actualmente, el procesamiento manual de estos documentos consume tiempo considerable del personal técnico y limita la capacidad de análisis estratégico y toma de decisiones basadas en datos.

Este trabajo presenta una revisión del estado del arte en arquitecturas de datos modernas, profundiza en la Arquitectura Medallón como solución contemporánea para la gestión de datos. Se analizan tecnologías de streaming distribuido como Apache Kafka, sistemas de almacenamiento NoSQL (MongoDB) y soluciones *Lakehouse* basadas en Delta Lake, estableciendo las bases teóricas para la implementación de un data-pipeline escalable y robusto.

En este trabajo se ha desarrollado e implementado un sistema integral de data-pipeline basado en una arquitectura de microservicios, empleando Apache Kafka como tecnología clave para la ingesta distribuida de datos, MongoDB para el almacenamiento de documentos en su formato original, y Delta Lake para la consolidación y transformación de actividades en formato tabular optimizado para análisis (OLAP). El sistema implementa técnicas de procesamiento de lenguaje natural (NLP) mediante algoritmos de *Naïve Bayes* para la clasificación automática de actividades, y proporciona interfaces interactivas desarrolladas con *Notebooks* interactivos que permiten a usuarios no técnicos la validación y limpieza de datos. La arquitectura implementada sigue el paradigma ELT, combinando procesamiento *batch* y streaming mediante ventanas de saltos, garantizando la trazabilidad completa de la información y cumpliendo con los principios ACID en todas las capas del sistema.

Se ha desarrollado exitosamente un sistema funcional que automatiza la extracción, procesamiento y organización de órdenes de trabajo en formato PDF, transformando los documentos a datos estructurados listos para análisis. La implementación de la Arquitectura Medallón en tres capas (Bronce, Plata, Oro) permite mantener la integridad de los datos originales mientras facilita su transformación progresiva hacia formatos analíticos de alto valor. El sistema procesa documentos históricos desde septiembre de 2018, implementando mecanismos de validación automática que generan reportes de inconsistencias, permitiendo al personal técnico identificar y corregir errores rápidamente. Como trabajo futuro, se contempla la expansión del sistema para integrar múltiples departamentos de la organización, el desarrollo de modelos predictivos de *Machine Learning* para anticipar fallas en la red eléctrica, y la eventual migración hacia soluciones *cloud-native* que aprovechen el procesamiento distribuido a gran escala mediante motores de procesamiento como Apache Spark.

Palabras clave: Data-pipeline, Data-Warehouse, Power-Distribution, Event-Driven.

1. Introducción

El análisis de datos y el uso de herramientas estadísticas transforman la forma en que las empresas operan, permite que la toma de decisiones se base en evidencia y en las experiencias propias. En la actualidad se puede llegar aún más allá, es posible desarrollar modelos predictivos basados en técnicas de *Machine Learning* que permitan actuar proactivamente y optimizar los recursos.

Este Trabajo Final de Máster se centra en el análisis de las actividades de operación y mantenimiento en las redes eléctricas en la provincia de Zamora Chinchipe, Ecuador. Estas tareas son cruciales y las llevan a cabo el personal de los diversos grupos de trabajo de la Empresa Eléctrica Regional del Sur S.A. (EERSSA).

Las actividades ejecutadas pueden ser, planificadas o emergentes, preventivas o correctivas, tienen su incidencia desde una a múltiples viviendas. Toda esta información es registrada diariamente en documentos denominados "Órdenes de Trabajo", los cuales contienen una gran cantidad de datos. Dicho esto, actualmente no existe una herramienta informática que sintetice la información eficientemente. Si bien estos documentos son útiles para la elaboración de informes mensuales, el procesamiento de los datos se está realizando de una manera manual. Este es un proceso que consume mucho tiempo, es tedioso y, a pesar del esfuerzo, no permite aprovechar al máximo toda la información disponible.

Se propone la creación de herramientas informáticas modernas, que permitan la recolección de datos de una manera distribuida, mediante el uso de técnicas de Big Data y computación en la nube.

1.1. Problemática.

Actualmente, la toma de decisiones operativas en la EERSSA se la realiza en base a la información más reciente de la que se tiene conocimiento y se depende fuertemente del conocimiento empírico directo y de la memoria del personal involucrado. No existe un análisis estadístico profundo de los datos que permita evaluar el impacto de las actividades preventivas.

El personal obrero dispone de un sistema en línea para el registro de sus actividades, el resultado es un archivo en formato PDF denominado "Orden de Trabajo". En este documento se plasma información valiosa para el análisis operativo diario del sistema eléctrico, sin embargo, esta valiosa información se encuentra subutilizada, su revisión es manual y tediosa, los informes mensuales, que se derivan de los datos son elaborados manualmente, sin ningún tipo de automatización que agilice su elaboración, la búsqueda de información histórica se realiza directamente en los archivos PDF y es ineficiente.

Es necesario modernizar este proceso mediante la aplicación de tecnologías de la información enfocadas en el análisis de los datos. Esto permite una ágil gestión de la información, generación automatizada de informes, indexación de las actividades que permitan una revisión histórica y una trazabilidad de los problemas y sus soluciones.

La solución planteada es la implementación de un sistema que permita la gestión de la información contenida en estos documentos, su validación y limpieza, con un enfoque en las tecnologías de manejo de datos su procesamiento analítico. Es importante enriquecer esta información con diversas fuentes, por lo que se plantea la implementación de toda una plataforma informática, que permita el crecimiento e inclusión de nuevas fuentes de información en un esquema de *Big Data* que garantice un sistema escalable de acuerdo con las necesidades de la organización.

1.2. Objetivos

El objetivo es automatizar el análisis y la explotación de los datos, los cuales alimentarán modelos predictivos robustos que faciliten la toma de decisiones estratégicas y operativas.

1.2.1. Objetivo General

Desarrollar e implementar una solución integral de *data-pipeline* para la recolección, procesamiento y análisis de los datos generados por las operaciones de mantenimiento en la Jefatura Zonal Zamora de la Empresa Eléctrica Regional del Sur S.A. (EERSSA), con el fin de facilitar la toma de decisiones estratégicas.

1.2.2. Objetivos Específicos

Para desarrollar y evaluar el uso de la solución propuesta se han considerado los siguientes objetivos específicos:

1. Diseñar e implementar un *data-pipeline* escalable y basado en una arquitectura de eventos, para la ingesta y procesamiento eficiente de información, asegurando su disponibilidad para análisis.
2. Diseñar e implementar un modelo predictivo basado técnicas de *Machine Learning* utilizando datos históricos para estimar el cumplimiento de parámetros de calidad del servicio eléctrico, permitiendo una gestión proactiva y la mitigación de riesgos operativos.
3. Implementar un repositorio centralizado de datos estructurados, para almacenar los datos procesados y estructurados que permita la generación de informes y el seguimiento de indicadores clave de rendimiento (KPI) para el seguimiento continuo de las operaciones de mantenimiento.

2. Marco Teórico

Hoy en día el volumen y velocidad de información generada a nivel mundial han alcanzado una magnitud sin precedentes, y tenemos contacto con ello diariamente. Esto lo podemos evidenciar a través de las plataformas digitales y redes sociales, donde se interactúa con un flujo constante de información, fotografías, videos y demás. Sin embargo, el verdadero desafío no reside en el acceso a estos datos, sino en la capacidad para procesarlos, transformarlos y convertirlos en conocimiento útil. Esto es particularmente importante en el entorno empresarial, y es en este contexto que la ingeniería de datos emerge como una disciplina fundamental. Esta especialización se encarga del diseño, construcción y el mantenimiento de la infraestructura necesaria para gestionar y procesar grandes volúmenes de datos, garantizando su disponibilidad y calidad para análisis posteriores. En esencia, la ingeniería de datos sienta las bases que permiten a las organizaciones aprovechar el valor de sus activos de información.

2.1. El ciclo de vida de los datos

Reis & Housley (2022) indican que, dentro de la ingeniería de datos, un componente esencial para la gestión del flujo de información es el ciclo de vida de la ingeniería de datos (*data engineering lifecycle*) este ciclo está conectado mediante la tubería de datos (*data-pipeline*). Esta se define como una secuencia de procesos interconectados que transportan, transforman y consolidan datos desde su origen hasta su destino final. Cada etapa en la tubería cumple una función específica; la salida de un proceso se convierte en la entrada para el siguiente, creando un flujo de trabajo secuencial y automatizado. El objetivo principal de una tubería de datos es garantizar que la información llegue a su punto de uso de manera confiable, limpia y en el formato adecuado.

Reis & Housley, (2022) dividen este ciclo de datos en cinco etapas:

- Generación
- Almacenamiento
- Ingestión
- Transformación
- Entrega de datos

La fase de generación de datos es el punto de partida en el ciclo de vida de la ingeniería de datos, donde la información se origina en diversas fuentes, como dispositivos IoT, bases de datos transaccionales o colas de mensajes. Es importante el comprender cómo operan estos sistemas, la frecuencia y el volumen de los datos que producen, y la variedad de sus formatos. Un desafío clave en esta etapa es la gestión de esquemas de datos, los cuales pueden ser fijos o flexibles y, a menudo, evolucionan con el tiempo. El rol de la ingeniería es fundamental para adaptar las

tuberías de datos a estos cambios y asegurar que la información cruda se transforme en un formato útil para el análisis.

El almacenamiento es una etapa crucial y compleja en el ciclo de vida de los datos, ya que la elección de la solución de almacenamiento impacta todas las fases subsiguientes. En arquitecturas modernas, es común utilizar múltiples tecnologías de almacenamiento que no solo guardan datos, sino que también permiten su procesamiento. Un aspecto fundamental para la selección del sistema de almacenamiento adecuado es comprender la frecuencia de acceso a los datos, lo que se conoce como su "temperatura": datos "calientes" de acceso frecuente, datos "tibios" de acceso ocasional y datos "fríos" para archivos históricos. La decisión final sobre la tecnología de almacenamiento depende de factores como los volúmenes de datos, la frecuencia de ingesta y los casos de uso, sin que exista una única solución universal para todos los proyectos.

La ingesta de datos es el proceso de recolectar y mover la información desde los sistemas de origen. Esta fase representa uno de los mayores cuellos de botella en el ciclo de vida de la ingeniería de datos, ya que tanto los sistemas de origen como los servicios de ingesta pueden ser inestables. Un sistema de ingesta eficiente debe ser robusto y confiable para asegurar que el flujo de datos no se detenga. Para construir una arquitectura sólida, es fundamental conocer los casos de uso, el destino final de los datos, la frecuencia de acceso y el volumen esperado.

Una vez que los datos han sido ingeridos y almacenados, la transformación es el proceso de convertirlos desde su formato original a una estructura útil para el análisis y consumo. Esta es la fase donde los datos brutos comienzan a generar valor real. Las transformaciones iniciales incluyen tareas básicas como el mapeo de tipos de datos, la estandarización de formatos y la limpieza de registros. En etapas más avanzadas, se pueden aplicar procesos complejos como la normalización del esquema, la agregación de datos para la generación de reportes o la creación de características para modelos de *Machine Learning*.

La entrega de datos es la etapa fina, donde los datos transformados se ponen a disposición de los usuarios finales para generar valor. El valor de los datos se materializa solo cuando son consumidos, ya sea a través de análisis de negocios, la ejecución de modelos de *Machine Learning* o la creación de informes. Sin un propósito claro para su consumo, los proyectos de datos corren el riesgo de convertirse en "proyectos de vanidad", donde se recolecta y almacena una gran cantidad de información sin un uso práctico. Esta fase destaca la importancia de diseñar todo el ciclo de vida de los datos con un objetivo de negocio bien definido en mente.

Aplicando estos conceptos y desde un punto de vista práctico y moderno la arquitectura del *pipeline* a implementar se resumiría en la Ilustración 1:

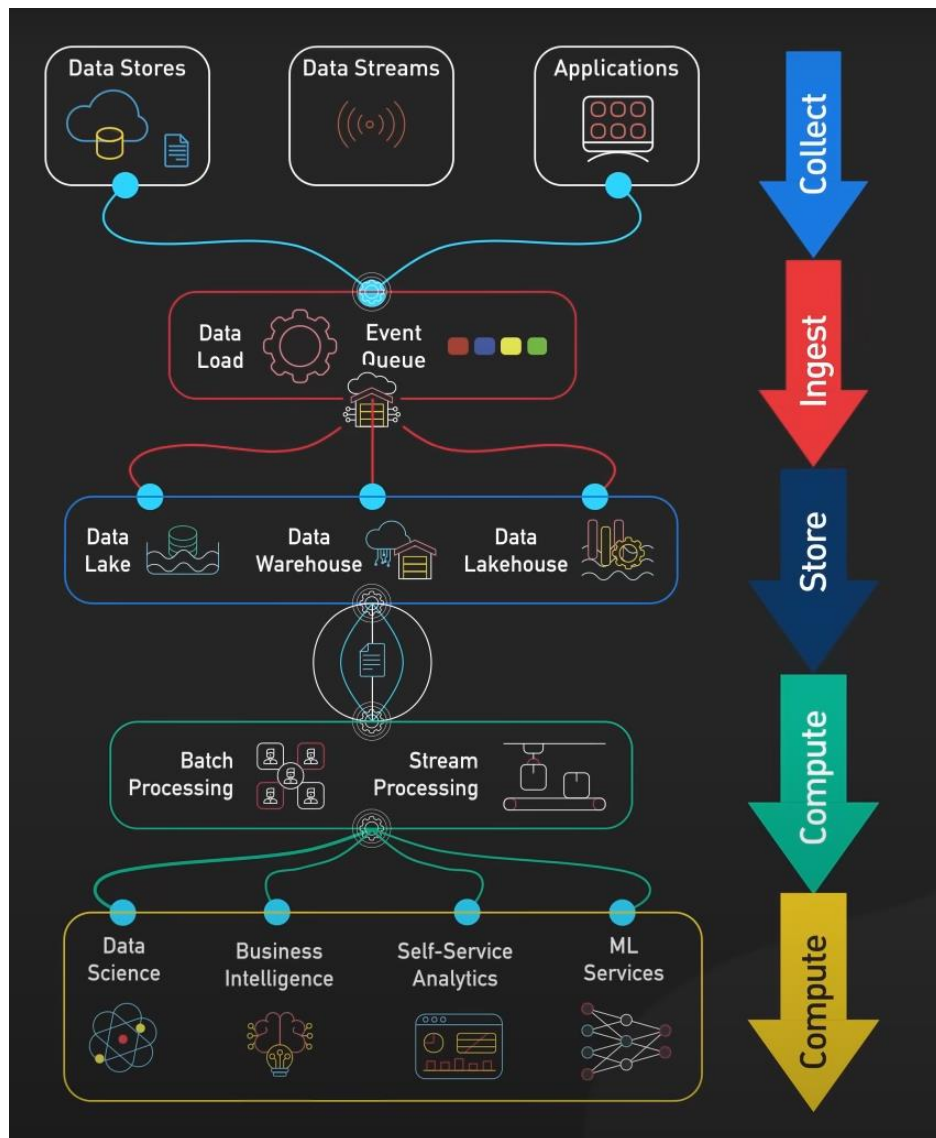


Ilustración 1. "The amazing Data pipeline". Fuente (ByteByteGo, 2024).

La Ilustración 1 resume la arquitectura global a implementarse durante este trabajo de titulación. Existen diversas fuentes de datos desde las cuales se genera información. Estas serán recogidas y agregadas a un sistema de eventos de datos, una solución robusta y escalable de acuerdo con las necesidades de la organización. Luego se almacenarán en al menos dos tipos de bases de datos, una de tipo NoSQL que brinda mayor flexibilidad para luego ser procesada y transformada a un sistema de almacén de datos (*Data Warehouse*), en un formato estandarizado que permita realizar consultas analíticas y que sirva de sustento para la generación de reportes, y entrenamiento de algoritmos de *Machine Learning*.

Los *Data Warehouse* surgen como una práctica fundamental para centralizar e integrar la información de una organización, con el objetivo de establecer una "versión única de la verdad" que sirviera de base para la toma de decisiones empresariales.

2.2. Paradigmas fundacionales

Existen dos conceptos fundamentales en la secuencia de integración de datos, estos son, en el enfoque tradicional: Extracción, Transformación y Carga, ETL por sus siglas en inglés y un enfoque más moderno: Extracción, Carga y Transformación, la decisión entre uno y otro refleja la filosofía y el enfoque de la infraestructura a implementar.

2.2.1. Diferencias entre ETL y ELT

Extracción, Transformación y Carga, este paradigma involucra tres pasos, la extracción de los datos, puede ser de una o varias fuentes, luego la transformación como un proceso separado, y finalmente la carga en un el sistema de destino en un formato estructurado. Esta arquitectura se basa en las restricciones tecnológicas de la organización, en los que el cómputo y el almacenamiento son restringidos y costosos, prefiriendo realizar la transformación en sitio y almacenando el resultado en un servicio especializado, usualmente en una nube de datos. El enfoque ELT por otra parte carga los datos tal cual como se extraen y los transforma en una etapa posterior, dependiendo de los casos de uso. En el proceso ETL se requiere una definición formal desde el principio, definiendo así los tipos de datos y esquemas.

Mediante una mayor adopción de la computación en la nube y la creación de plataformas de dato modernas se ha producido un cambio hacia un enfoque ELT. En este modelo, los datos son extraídos desde las fuentes e inmediatamente cargados a sistemas de datos como pueden ser *Data Lake* o *Data Warehouse*. La transformación de estos datos se produce directamente en el repositorio, aprovechando la escalabilidad y el procesamiento en la nube. Este enfoque es posible gracias a las innovaciones en la arquitectura de las plataformas de datos, en las que el procesamiento de datos puede ser provisionado bajo demanda y enriquecido a medida que nuevas fuentes de datos son agregadas (Amazon Web Services, 2025).

La Tabla 1 (abajo) compara las características del modelo ETL frente al modelo ELT considerando varios puntos clave que permiten una adecuada toma de decisiones acerca de qué tipo de la secuencia de integración de datos más adecuada para las necesidades de cada organización.

Tabla 1. Resumen de diferencias entre ETL y ELT. Fuente (Amazon Web Services, 2025)

Característica	ETL	ELT
Lugar de Transformación	Ocurre en un servidor de procesamiento secundario o en un área de preparación (<i>staging</i>), fuera del <i>Data Warehouse</i> .	Ocurre directamente dentro del <i>Data Warehouse</i> en la nube o <i>Lakehouse</i> de destino.
Idoneidad de la estructura de datos	Más adecuado para datos estructurados que pueden mapearse a un esquema relacional predefinido.	Superior con datos estructurados, semiestructurados y no estructurados.
Velocidad de procesamiento	Generalmente más lento, ya que la transformación es un paso distinto y propenso a cuellos de botella antes de la carga.	Carga más rápida de datos crudos; las transformaciones se pueden procesar en paralelo.
Escalabilidad	Difícil de escalar, el motor de transformación suele ser único y se ralentiza a medida que aumenta el volumen de datos.	Altamente escalable, las plataformas de datos en la nube pueden escalar elásticamente los recursos de cómputo.
Disponibilidad de datos	Rígida. La lógica de transformación se define de antemano y es difícil de cambiar o iterar.	Altamente flexible e iterativa. Las transformaciones se pueden modificar o agregar sin necesidad de recargar los datos.
Inversión económica	Puede incurrir en costos más altos debido a la necesidad de una infraestructura separada para la etapa de transformación.	A menudo más rentable, aprovechando los precios de pago por uso de la nube y una pila de datos más simple y consolidada.
Seguridad	Puede requerir desarrollo de software a medida para cumplir con requerimientos de seguridad	Cuenta con características incluidas en la plataforma para el manejo de datos.

2.2.2. El Procesamiento de datos

El factor determinante en estos dos paradigmas es el tiempo. Históricamente el procesamiento de datos ha sido dominado por el paradigma “por lotes” (*Batch Processing*). Como ejemplo de este enfoque tenemos las primeras tecnologías de *Big Data* como Hadoop MapReduce, el cual resulta muy adecuado para casos de uso que no requerían información inmediata, como la generación de informes de ventas diarios o la realización de conciliaciones financieras mensuales. Sin embargo, su principal desventaja es que hay un retraso significativo, desde horas o incluso días, entre el momento en que se generan los datos y el momento en que la información resultante está disponible, lo que lo hace inadecuado para aplicaciones sensibles al tiempo. Para estos casos se cuenta con el paradigma de *Streaming* también llamado en tiempo real.

Reis & Housley (2022) brindan un resumen conciso:

"Las transformaciones por lotes se ejecutan sobre fragmentos discretos de datos, en contraste con las transformaciones de *streaming*, donde los datos se procesan de forma continua a medida que llegan." (2022, pág. 310)

La constante demanda por datos inmediatos ha llevado al desarrollo del procesamiento *Streaming*, en el cual se procesa la información continuamente a medida que esta se genera, capturando eventos individuales y registrándolos. Los sistemas desarrollados han sido diseñados para una baja latencia de procesamiento, una alta tolerancia a fallas y su escalabilidad.

Es importante entender a que nos referimos cuando hablamos de flujo “*stream*” de datos:

“Un flujo de datos es una abstracción que representa un conjunto de datos ilimitado. Ilimitado significa infinito y en constante crecimiento. El conjunto de datos es ilimitado porque, con el tiempo, continúan llegando nuevos registros. [...] El procesamiento por streaming se refiere al procesamiento continuo de uno o más flujos de datos. El procesamiento de *streaming* es un paradigma de programación, al igual que el procesamiento por lotes y el de solicitud-respuesta” (Narkhede, Shapira, & Palino, 2017, pág. 248)

El procesamiento mediante *streaming* es una evolución del procesamiento por lotes, y dentro de esa cadena evolutiva encontramos también un paso intermedio, el denominado *micro-batch*, el cual es una forma de tomar un paradigma por lotes y aplicarlo en una situación en la que tenemos un flujo de datos. El procesamiento por *micro-batch* puede ser ejecutado en periodos cortos, desde un par de minutos hasta cada segundo. Un verdadero sistema *streaming* está diseñado para procesar un evento a la vez. Sin embargo, esto con lleva una sobrecarga significativa. Además, es importante tener en cuenta que incluso estos sistemas de *streaming* puros, muchos procesos seguirán ocurriendo en lotes (Reis & Housley, 2022)

La utilización y enfoque del sistema de procesamiento dependerá de las necesidades de la organización, estas pueden ser dinámicas y cambiar con el tiempo, por lo que la elección de una arquitectura que permita la combinación de estos paradigmas puede resultar deseable. Es muy importante elegir una arquitectura que permita la evolución del sistema, ajustándose a necesidades institucionales y a las nuevas tecnologías. A continuación, conoceremos acerca de las arquitecturas de datos modernas.

2.3. Arquitecturas tradicionales para el manejo de datos

Para el desarrollo de un sistema de gestión de datos debemos considerar el uso de tecnologías y técnicas de manejo de datos modernas y probadas, las cuales permitirán hacer uso de herramientas de código abierto y que sean fácilmente escalables.

Actualmente existen herramientas fáciles de utilizar, hechas a medida para el tratamiento de datos, soluciones listas que permiten la creación modular de arquitecturas de datos, de estas herramientas han emergido varios patrones de diseño, enfocados en la confiabilidad, escalabilidad y mantenibilidad, revisaremos los más relevantes para nuestro caso de uso.

2.3.1. Arquitectura Lambda

A principios del año 2010 la popularidad del streaming de datos explotó con el surgimiento de Apache Kafka, un software especializado, escalable para el manejo de eventos, combinado con tecnologías como Apache Storm y Samza permitieron a las empresas ejecutar análisis y modelado de grandes cantidades de datos en tiempo real. Los ingenieros de datos necesitaban resolver y reconciliar tanto el procesamiento por lotes tradicional, con el nuevo sistema de streaming bajo una misma arquitectura. Es así como surge la arquitectura lambda en respuesta a este desafío.

En la arquitectura Lambda se tiene dos sistemas operando independientemente el uno del otro, por un lado, el sistema *Streaming* cuyo propósito es entregar los datos con la menor latencia posible, en una capa de “velocidad”, y por el otro una capa de lotes, en la que los datos son procesados y transformados en un sistema de *Data Warehouse*, creando vistas de datos ya procesadas y enriquecidas.

Esta arquitectura tiene varias desventajas. Es necesario mantener al menos dos sistemas y fuentes de código. Como en el caso del *micro-batch* se trata de un paso intermedio y necesario. Al igual que el *micro-batch* tiene sus usos y aplicaciones, como puede ser en los motores de búsqueda. La arquitectura Lambda no es la más recomendable si lo que se busca es combinar datos de *streaming* y de *batch*. Como respuesta a las limitaciones de Lambda se ha desarrollado una nueva arquitectura (Reis & Housley, 2022)

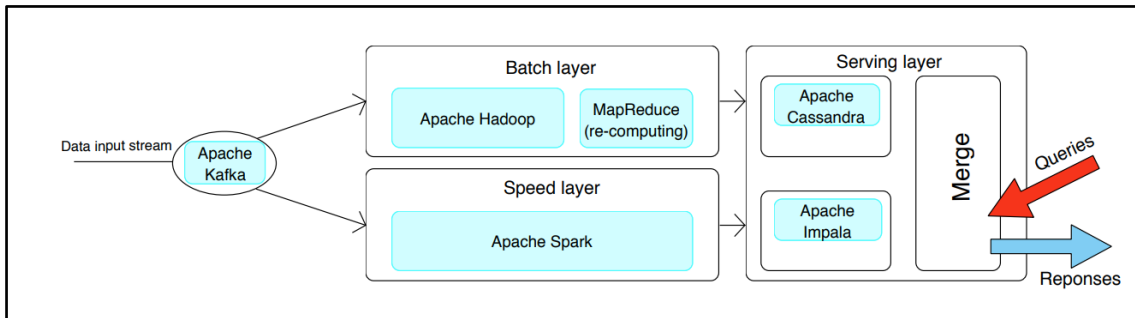


Ilustración 2. La Arquitectura Lambda. Fuente (Feick, Kleer, & Kohn, 2018)

La Ilustración 2 muestra un esquema simplificado de la puesta en marcha de la Arquitectura Lambda, se puede observar como una misma fuente de datos se divide y alimenta dos capas, una capa *Batch* que implementa tecnología de Hadoop Map Reduce, y una segunda capa que implementa tecnología Apache Spark. Finalmente, ambas capas se unen en la capa de vistas, desde la cual se realizan las consultas.

2.3.2. Arquitectura Kappa:

La arquitectura Kappa, propuesta por Jay Kreps y llamada así ya que Kappa es la siguiente letra en el alfabeto griego luego de Lambda, indicando una evolución sobre su predecesor a favor de la simplicidad, esto se logra incrementando el paralelismo en los sistemas de *Streaming* y valiéndose de la funcionalidad de sistemas de eventos por ejemplo, Apache Kafka, los cuales retienen el historial de eventos, los cuales pueden ser reprocesados a medida que ingresan nuevos datos, aplicando procesos tanto de tiempo real, como de procesamiento por lotes sobre los mismos datos y reutilizando el código que es útil para ambos tipos de procesamiento, simplificando así su mantenimiento (Kreps, 2014)

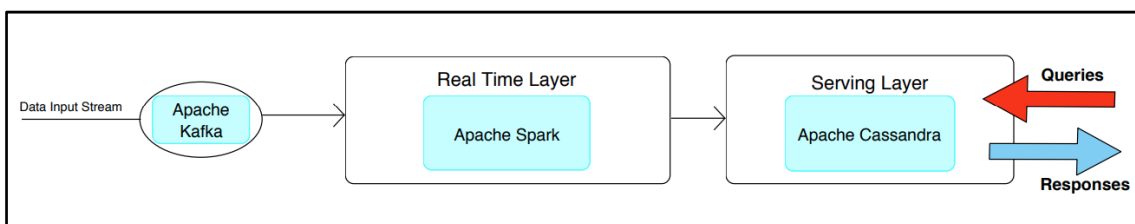


Ilustración 3. La Arquitectura Kappa. Fuente (Feick, Kleer, & Kohn, 2018)

La Ilustración 3 muestra un esquema simplificado de la Arquitectura Kappa, se puede observar un único flujo de datos, sin divisiones, se cuenta con una capa de tiempo real que procesa los datos, y una capa de visualización desde del cual se realizan consultas.

Es notable que los inicios de la arquitectura Kappa datan del año 2014, la cual no ha sido ampliamente adoptada, sus limitaciones radican en su complejidad al momento de su implementación y elevados costos de cómputo sobre todo en elevados volúmenes de datos con enormes cantidades de datos históricos. Ambas arquitecturas

buscan mejorar las limitaciones del tradicional ecosistema Hadoop al intentar pegar con cinta herramientas complicadas que no fueron inicialmente concebidas para trabajar de manera conjunta. Actualmente la filosofía de “*el procesamiento por lotes como un caso especial del streaming*” es ahora más común. Diversos *frameworks* como Apache Flink y Spark han adoptado un enfoque similar. (Reis & Housley, 2022).

2.4. La evolución de los sistemas de gestión y análisis de datos

Una vez establecidos los conceptos fundamentales y las arquitecturas tradicionales para el manejo de datos, es importante conocer la evolución que ha tenido los sistemas de análisis de datos, el comprender esta evolución ayudará a escoger la mejor solución para el sistema que se va a desarrollar.

En esta sección viene guiada del trabajo de síntesis desarrollado por Strengtholt (2025) quien nos lleva a través de la historia y evolución de los sistemas de gestión y análisis de datos:

Iniciamos con los *Data Warehouses*, recordamos que su objetivo es establecer una "versión única de la verdad" como base para la toma de decisiones. Este proceso involucra la recolección de datos de diversas fuentes, su transformación a un formato uniforme y su carga en un repositorio central. En el corazón de esta arquitectura se encuentran los sistemas de origen, conocidos como sistemas de procesamiento de transacciones en línea (OLTP), diseñados para manejar cargas de trabajo operativas, como la lectura, actualización y eliminación de registros. Estos sistemas OLTP se caracterizan por sus consultas predecibles y de bajo volumen, y su modelo de datos está optimizado para estas operaciones, lo que se refleja en una estructura altamente normalizada para evitar la redundancia de la información. Estos sistemas no están diseñados para proveer una vista analítica completa y consolidada del estado del negocio, esto debido a que extraer esta información involucra consultas complejas y demandantes.

Se vuelve necesario separar la carga analítica de los sistemas operacionales, preservando la integridad de los datos históricos, pero también optimizando el sistema para un mejor procesamiento analítico. En este contexto el *Data warehouse* funciona como un núcleo central donde se unifican los datos provenientes de múltiples sistemas de origen. Su propósito principal es transformar esta información en un formato coherente, optimizado para el procesamiento analítico en línea (OLAP). A diferencia de los sistemas OLTP, que priorizan la integridad de las transacciones, los sistemas OLAP están diseñados para un alto rendimiento en análisis, con un enfoque en operaciones de lectura intensiva. Esto permite la optimización del diseño de la base de datos, lo que a menudo resulta en la duplicación de datos y la desnormalización de tablas. Las tablas suelen ser grandes y "aplanadas", lo que agiliza la ejecución de consultas analíticas complejas y de gran escala.

Los lagos de datos (*Data Lakes*) surgieron a mediados de la década de 2000 como una alternativa a las limitaciones de los *Data warehouses* tradicionales. Su principal innovación fue una arquitectura distribuida que permitía almacenar y gestionar enormes volúmenes de datos en su estado original —estructurado, semiestructurado o no estructurado—. Esta flexibilidad, combinada con el uso de software de código abierto como Hadoop, que opera en hardware comercial asequible, marcó un cambio fundamental en el manejo del *Big Data*, alejándose de las costosas soluciones propietarias. Los *Data Lakes* también integraron tecnologías de *Machine Learning*, expandiendo su utilidad más allá de los informes analíticos, un rasgo distintivo de los *Data warehouses*.

La arquitectura *Lakehouse* surge como la evolución natural del *Data warehouse* y el *Data Lake*, combinando sus mejores características. Esta nueva arquitectura se fundamenta en formatos de tablas de datos de código abierto, como *Delta Lake*, clave para su desarrollo. El crecimiento de Apache Spark fue un catalizador para esta transición, convirtiéndose en un proyecto de primer nivel de Apache. En 2013, los creadores de *Spark* fundaron *Databricks*, una empresa que buscaba simplificar el procesamiento de *Big Data* para todo tipo de usuarios. Su estrategia de enfocarse exclusivamente en la nube, a diferencia de los enfoques *on-premise* de sus competidores, resultó ser una decisión acertada, coincidiendo con el auge de las soluciones desacopladas de almacenamiento y cómputo.

Apache Spark ha evolucionado, operando de manera independiente en la nube y permitiendo la creación de múltiples clústeres elásticos y dinámicos que interactúan con una capa de almacenamiento de objetos unificada.

El surgimiento de formatos de tablas de código abierto ha sido un punto de inflexión. Proyectos como Apache Hudi (Uber, 2017), Apache Iceberg (Netflix, 2018) y Delta Lake (Databricks, 2019) se desarrollaron para abordar las limitaciones de los *Data Lakes* tradicionales, como la falta de garantías transaccionales. Estos formatos proporcionan características avanzadas como transacciones ACID, manejo de metadatos escalable y la unificación del procesamiento de datos por streaming y por lotes. *Delta Lake*, en particular, utiliza el formato Parquet y un registro de transacciones (*Delta Log*) para asegurar la integridad de los datos. Recientemente, Databricks ha adquirido Tabular para impulsar la interoperabilidad entre estos formatos, buscando establecer un estándar abierto común. Esta iniciativa, junto con proyectos como Apache XTable, promete una gestión de datos más cohesiva y fluida en el futuro. (Strenght, 2025)

2.4.1. Arquitectura Medallón

Tanto en el caso de Lambda como en el de Kappa, las tecnologías de base son las que limitan su aplicación. La evolución de estas herramientas ha ido relegando los sistemas tradicionales, basados en Hadoop y Map Reduce, incluso Apache Spark, una tecnología fundamental en el ecosistema de ingeniería de datos empieza a ser

utilizada exclusivamente para sistemas con grandes volúmenes de datos (véase [Anexo I](#)), debido a su costo operativo innecesario en muchos de los casos, en su lugar, se han desarrollado herramientas ligeras y versátiles. Nuevas tecnologías para el manejo de base de datos, como *DuckDB*, *Delta Lake*, *Iceberg*, herramientas especializadas para el manejo de *Datatables*, como lo son *Polars* y aún más recientemente *Daft*, una librería distribuida y multimodal, creada en el año 2023 y pensada para simplificar el procesamiento de variadas fuentes de información con miras a un futuro en la revolución de la denominada “Inteligencia Artificial”.

Estas nuevas herramientas amplían el horizonte y permiten un enfoque moderno, sacando aún mayor provecho a las tecnologías en la nube, computación distribuida y multiplataforma, permitiendo una organización de la información por capas, es así como surge la denominada **Arquitectura Medallón**, con el objetivo de una mejora incremental y progresiva de los datos, con un enfoque más directo a la Ciencia de Datos como un escenario central y no simplemente un agregado final.

La arquitectura Medallón **es un patrón de diseño** usado para organizar la información de una manera lógica, usando tres capas de calidad. La capa Bronce contiene datos crudos y sin filtrar, ingeridos desde los sistemas de origen. La capa Plata alberga datos que han sido limpiados, estandarizados y enriquecidos. La capa Oro consiste en conjuntos de datos altamente curados y agregados, optimizados para casos de uso específicos de inteligencia de negocio y análisis. Este enfoque por capas proporciona un linaje de datos claro, apoya el refinamiento incremental de los datos y aísla las diferentes etapas del procesamiento (Strengtholt, 2025)

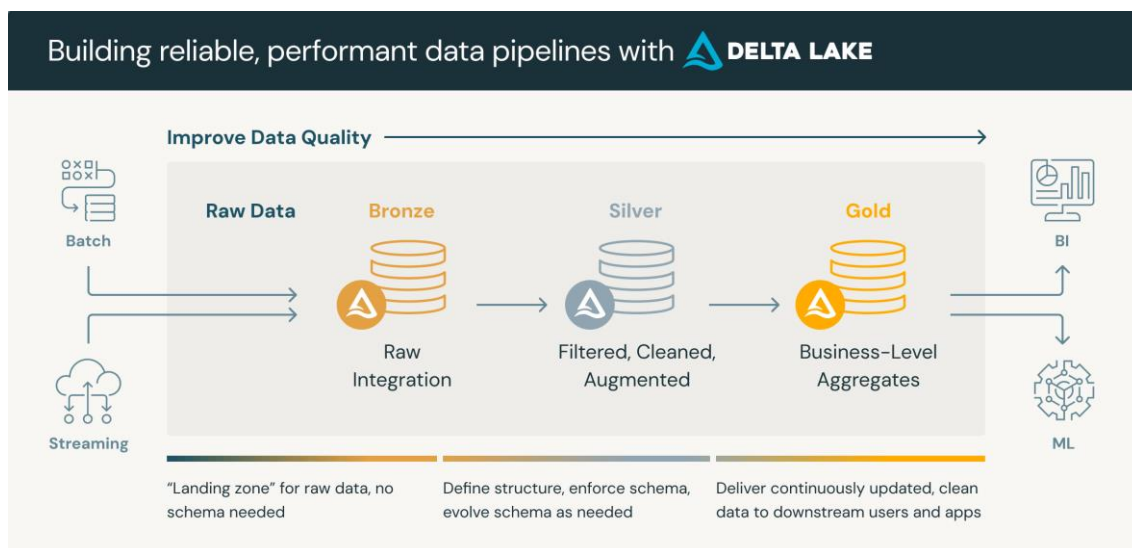


Ilustración 4. Descripción general de la Arquitectura Medallón. Fuente (Databricks Inc., 2025)

La Ilustración 4 muestra la Arquitectura Medallón, ideada por Databricks, es un patrón de diseño de datos que organiza de manera lógica y estructurada la información dentro de un *Lakehouse*. Este modelo representa una mejora progresiva en la calidad. A pesar de la claridad de sus nombres, el patrón carece de una guía práctica

unificada, lo que genera inconsistencias en su implementación. Esto subraya que, si bien la Arquitectura Medallón es un punto de partida útil, el verdadero reto reside en el modelado de datos y en la correcta aplicación de sus principios.

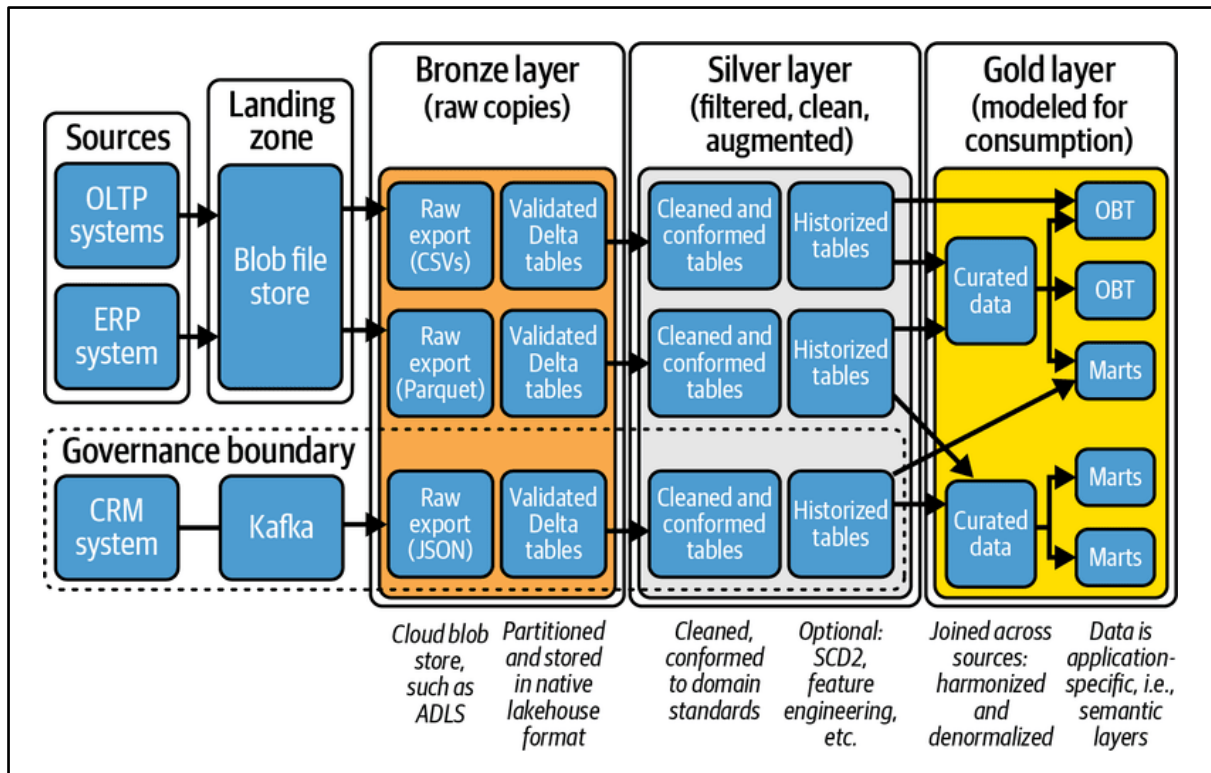


Ilustración 5. Arquitectura Medallón. Fuente (Strenghtolt, 2025)

La Ilustración 5 indica la implementación práctica de la Arquitectura Medallón, provee de una plantilla base para su implementación e integración con una organización establecida, como es el caso de estudio. Permite organizar, modularizar y establecer fronteras para el tratamiento de datos. Esta arquitectura abraza el concepto de transformación constante de los datos, la integración de diversas fuentes en cada etapa del proceso. Si bien es tarea del ingeniero de datos el diseño y supervisión general de la arquitectura, cada especialista aporta al enriquecimiento de los datos.

La Arquitectura Medallón demuestra su utilidad práctica en escenarios de datos generados por sensores, como un sensor de humedad que envía mediciones a intervalos regulares. En el caso de que este sensor experimente un desperfecto y genere valores anómalos, la capa *Bronze* conserva la información cruda e inalterada, actuando como la "verdad" histórica. Este registro original es invaluable para el técnico de campo, ya que le permite analizar la línea de tiempo de la falla, los valores atípicos arrojados y el tiempo de corrección, información crucial para el análisis de fallas y la planificación de mantenimientos. En contraste, el científico de datos requiere un dataset limpio para el modelado. En este escenario, la capa *Plata* permite aplicar procesos de limpieza y transformación que filtran los datos erróneos, generando una vista optimizada. De esta manera, el diseño de la Arquitectura Medallón permite que

las capas actúen como un patrón flexible que satisface las necesidades específicas de cada profesional, manteniendo la integridad de los datos crudos mientras ofrece vistas filtradas y aumentadas para el análisis.

Es importante entender que la arquitectura de Medallón no debe ser vista como un concepto rígido, más bien, representa un espectro de posibilidades que puede ser adaptado a situaciones únicas, incluyendo la adopción de múltiples Arquitecturas de Medallón hechas a medida. Esta arquitectura representa el estado del arte en cuanto al diseño enfocado a la recolección, curado y entrega de valor a las organizaciones.

El siguiente apartado abordará el análisis y metodología para aplicar esta arquitectura, detallando la mejor estrategia para su integración en nuestro caso de estudio.

2.5. Plataforma de transmisión distribuida

Una vez que se ha identificado la necesidad de procesar los documentos de texto, surge la pregunta crítica sobre la ubicación y el flujo de estos archivos. La información es generada por los trabajadores en campo, pero su validación y aprobación final recae en los revisores. Estos grupos de trabajo, que se estructuran jerárquicamente bajo coordinadores, no operan en un único punto geográfico, sino que están distribuidos a lo largo del territorio donde la empresa eléctrica ofrece sus servicios.

Esta dispersión geográfica y la naturaleza secuencial del proceso de validación presentan dos desafíos fundamentales:

Latencia y Descentralización: Dependiendo de métodos tradicionales (como el correo electrónico o la transferencia manual de archivos) genera una alta latencia en el flujo de datos y dificulta la centralización de la información.

Escalabilidad y Fiabilidad: Un sistema centralizado tradicional podría ser susceptible de intermitencias en la comunicación, además, un objetivo a largo plazo es la integración de varios departamentos dentro de la organización, si bien esto queda fuera del alcance del trabajo actual, es hacia donde apunta el desarrollo del sistema.

Es aquí donde se vuelve indispensable implementar un mecanismo robusto que permita la recolección, centralización y procesamiento asíncrono de la información. La solución a este desafío se encuentra en una Plataforma de transmisión distribuida, como Apache Kafka. Este tipo de plataforma está específicamente diseñado para manejar grandes volúmenes de eventos y mensajes de manera tolerante a fallos y escalable. Al funcionar como una capa desacoplada entre los productores de datos (los revisores) y los consumidores (los sistemas de análisis), Kafka asegura que todos los reportes sean capturados de manera inmediata, independientemente de su origen geográfico o la carga del sistema. En esencia, actúa como la columna vertebral que transforma una red dispersa de aprobaciones en un flujo de datos centralizado y continuo, esencial para el subsiguiente procesamiento analítico.

2.5.1. Apache Kafka

Existen diversas soluciones de mensajería que podrían implementarse, pero la selección de Apache Kafka se justifica por su capacidad para actuar como una Plataforma de Streaming Distribuido, alineándose con los requisitos de escalabilidad, desacoplamiento y flexibilidad inherentes a una arquitectura de *Data Lakehouse*.

Colas de Mensajes vs Publicación/Suscripción (Pub/Sub)

Una alternativa común a Apache Kafka son los sistemas basados en colas de mensajes punto a punto (*Message Queues*), como RabbitMQ. Si bien estos sistemas son excelentes para simplificar el flujo de tareas y asegurar que cada mensaje sea procesado exactamente una vez por un único consumidor, su adopción supondría una limitación significativa para nuestro sistema.

La arquitectura de *Data Lakehouse* demanda el paradigma de Publicación/Suscripción (Pub/Sub), una característica nativa de Kafka. Este modelo permite que múltiples consumidores accedan y procesen el mismo flujo de datos de manera independiente y asíncrona. Esta capacidad es crucial para la Arquitectura Medallón, donde es frecuente generar diversas vistas analíticas (capas Plata y Oro) a partir de una única fuente de datos crudos (capa Bronce). Al permitir que diferentes servicios, como un módulo de limpieza de datos, un motor de *Machine Learning* y un sistema de reportería en tiempo real, consuman el mismo *stream* en paralelo, se garantiza la flexibilidad y la interoperabilidad, facilitando la futura evolución, experimentación y adaptación del sistema a nuevas necesidades de negocio.

Sistemas Emergentes (NATS JetStream)

Apache Kafka, lanzado como código abierto en el año 2011, ha demostrado una confiabilidad y estabilidad probadas en entornos de misión crítica a escala global. Sin embargo, en el ecosistema de streaming han surgido tecnologías que expanden las capacidades tradicionales de Pub/Sub, como NATS JetStream.

NATS JetStream, basado en el núcleo de mensajería NATS, añade una capa de persistencia con características avanzadas de streaming. Esto incluye garantías de entrega de mensajes (al menos una y exactamente una), funcionalidades de almacenamiento de datos históricos y capacidades de *Key-Value*. Su diseño difumina la línea entre una plataforma de streaming y una base de datos distribuida, presentándose como una tecnología prometedora (NATS Authors, 2025).

No obstante, la complejidad que implica la implementación de una tecnología de streaming de última generación requiere una planificación profunda y la adaptación de los procesos de gestión de datos a las necesidades específicas de cada departamento. Dada la necesidad de establecer una base sólida y funcional que se ajuste al alcance de este trabajo de titulación, Apache Kafka se posiciona como la elección óptima debido a su madurez, amplia documentación y comunidad activa, lo

que minimiza la curva de aprendizaje e implementación inicial, dejando la migración o la integración de *JetStream* como una futura expansión estratégica del sistema.

Habilitador de Microservicios

La implementación de Apache Kafka facilita la adopción del paradigma de microservicios, un concepto central en el desarrollo *cloud-native* y distribuido. Al utilizar Kafka como bróker central, el sistema no se desarrolla como un monolito, sino como una colección de servicios especializados y desacoplados.

Esta aproximación constructiva, en la que el primer hito es la ingesta distribuida de datos, proporciona una flexibilidad inherente al diseño. A futuro permite el desarrollo modular de cada componente del sistema, lo que se traduce en:

- Desacoplamiento: Cada microservicio puede evolucionar de forma independiente.
- Escalabilidad Horizontal: Los servicios pueden escalarse individualmente según la demanda sin afectar a otros componentes.
- Interoperabilidad: Se establecen interfaces claras (los “*topics*” de Kafka) para la comunicación, lo que simplifica futuras expansiones, mejoras o la integración con sistemas externos.

3. Sistema de Gestión de Órdenes de Trabajo

En este capítulo se describe el desarrollo del sistema que permitirá implementar el *pipeline* de ingesta, carga y procesamiento de los datos. Se describirá cada una de las tecnologías implementadas, las características de cada microservicio y como se conectan para formar el sistema de gestión de órdenes de trabajo.

El desarrollo del sistema se ha llevado a cabo en un entorno Linux, el cual proporciona herramientas robustas y documentación exhaustiva que facilitan un desarrollo ágil y eficiente. El lenguaje de programación principal utilizado es Python, que se ha consolidado como el estándar de facto en la ingeniería de datos moderna debido a su versatilidad, amplio ecosistema de librerías especializadas y fuerte respaldo de la comunidad.

Considerando que el proyecto será desplegado en múltiples equipos personales con configuraciones potencialmente heterogéneas, la gestión consistente de librerías y dependencias resulta crítica para garantizar la reproducibilidad del entorno de ejecución. Para abordar este desafío, se ha implementado “UV”, un gestor de proyectos moderno que permite recrear el ambiente de desarrollo de manera precisa y determinística, asegurando que el sistema funcione correctamente independientemente del equipo donde se ejecute.

Finalmente, se ha utilizado GitHub como plataforma de control de versiones y distribución del código fuente, facilitando la colaboración, el seguimiento de cambios y la gestión del ciclo de vida del software. El repositorio se encuentra alojado y puede ser accedido desde el siguiente enlace: [Repositorio TFM Ernesto Palacios](#)

Tabla 2. Stack tecnológico base para el desarrollo del Sistema de órdenes de trabajo. Elaboración propia

Tecnología	Aplicación
Linux	Permite un ambiente de desarrollo rápido, con herramientas sólidas y de fácil instalación y configuración.
Python	Lenguaje de ingeniería de datos por excelencia. Fácil manejo y un sinnúmero de herramientas enfocadas al análisis de datos
UV	Organización, control y despliegue de entornos virtuales en Python. Ayuda con la documentación y configuración de cada microservicio.
GitHub	Herramienta indispensable para el control de versiones. Herramientas en línea para la implementación de integración y despliegue continuo CI/CD.

La Tabla 2 resume el *stack* tecnológico base para el desarrollo del sistema. Se ha cuidado que la programación sea agnóstica al sistema operativo utilizado, para ello se

ha empleado librerías nativas de Python para el manejo de ficheros de archivos y directorios.

Java Virtual Machine (JVM)

En el desarrollo de este trabajo se ha evitado, en la medida de lo posible, el uso directo de tecnologías que dependan del ecosistema Java Virtual Machine (JVM). Esta decisión arquitectónica responde a la necesidad de minimizar la complejidad operativa relacionada con la gestión de múltiples versiones de la JVM y sus conflictos de compatibilidad.

Esta consideración cobra especial relevancia al analizar la diferencia sustancial entre el ambiente de desarrollo y el entorno de producción. El despliegue del sistema está diseñado para ser implementado en la infraestructura propia de la organización, configurando una nube privada que cumpla con los estrictos parámetros de seguridad institucionales. En este contexto, mantener dependencias de JVM incrementaría significativamente los requisitos de configuración, monitoreo y actualización del sistema.

Como se detallará en el análisis individual de cada componente tecnológico, la exclusión de dependencias JVM permite reducir la complejidad del sistema, facilitando tanto su mantenimiento continuo como su despliegue en diferentes entornos. Esta simplificación arquitectónica resulta particularmente ventajosa considerando las limitaciones de recursos y conocimiento técnico disponible en la organización.

No obstante, uno de los componentes fundamentales del sistema, Apache Kafka, está inherentemente vinculado a la JVM, constituyendo una excepción necesaria a este principio arquitectónico. Dada su importancia crítica para la arquitectura de mensajería y procesamiento de eventos del sistema, su análisis detallado y la justificación de su inclusión se presentan en la sección **2.5.1 Apache Kafka**.

3.1. El origen de los datos

Los datos fueron extraídos desde los documentos conocidos como: orden de trabajo, los cuales son elaborados diariamente por el personal operativo en campo. Son archivos electrónicos en formato de PDF que sirven para registrar y llevar el control de la planificación y ejecución de actividades y novedades suscitadas en la red de distribución eléctrica. Existe un sistema desarrollado internamente por la EERSSA para su elaboración y almacenamiento.

Este sistema tiene ya ocho años de existencia y se accede a él mediante la página oficial de la empresa desde cualquier navegador web. Si bien el diseño de su interfaz no fue ideado para dispositivos móviles si es posible realizar el registro de las actividades desde uno, si bien la ergonomía del usuario deja mucho que desear.

		EMPRESA ELÉCTRICA REGIONAL DEL SUR S.A. <i>¡La energía somos todos!</i>		ORDEN DE TRABAJO Gualaquiza Z1 (Cuadrilla, Nro. 3)		No: 137.601	
CÓDIGO: P-GE04-12-F01		GERENCIA: OPERACIÓN Y MANTENIMIENTO		Sitio: Gualaquiza, Guabi Alto		Fecha: lunes, 16 de septiembre del 2024	
DESCRIPCIÓN DE LA ACTIVIDAD / PROYECTO / TRABAJO				MOVILIZACIÓN			
SOLICITANTE DEL PERMISO DE TRABAJO		CARGO FIRMA		Número/Código de vehículo: 4-62			
Responsable: LITUMA CORDOVA CESAR RUBEN		JECU		Placa: LEI-1053			
GUAMAN CONTENTO MARLON STALIN		LNI1		Descripción: CAMIONETA CHEVROLET LUV DMAX 3.0L DIESEL BLANCO			
Colaboradores: GUZMAN BARRIOS MARCO FERNANDO		LINS		Rentado: NO			
				Propietario: EERSSA			
				Chofer: GUZMAN BARRIOS MARCO FERNANDO			
TIPO (S) DE TRABAJO:							
A - SIN VOLTAJE		TRABAJO EN ALTURA - BAJO NIVEL					
CON ENERGIA/VOLTAJE - BAJO VOLTAJE		TRABAJO EN ALTURA - SOBRE NIVEL					
CON ENERGIA/VOLTAJE - MEDIO VOLTAJE							
DESCRIPCIÓN DEL TRABAJO A REALIZAR:							
Guabi Alto revisión de LMV y B/V. Recorrido y revisión derivación hacia El Pontón. En apego a la WR 316036183. Atender daños reportados por el Centro de Control.							
FECHA DE INICIO: lunes, 16 de septiembre del 2024		TIEMPO ESTIMADO DE DURACIÓN (HORAS): 8 H					
ANÁLISIS DE SEGURIDAD EN EL TRABAJO - AST							
RIESGOS EXISTENTES:		CONSECUENCIAS PROBABLES:		ELEMENTOS DE PREVENCIÓN A UTILIZAR			
BIOLOGICOS		Virus		Infección viral		Utilizar elementos de protección	
BIOLOGICOS		Mordeduras animales		Enfermedades zoonóticas		Uso de ropa de protección	
ELECTRICOS		Descarga eléctrica		Electrocución o electrocución		Uso de EPP	
ERGONOMICOS		Ambiente de trabajo		Lesiones musculares/ esqueléticas		Descansos programados	
MECANICOS		Caída de objetos		Golpes en cabeza y cuerpo		Uso de EPP	
MEDIDAS DE SEGURIDAD:							
MEDIDAS		ESTADO		EQUIPOS DE PROTECCIÓN		ESTADO	
Equipo Aislado Eléctricamente		BUENO		A. Protección para la Cabeza (Casco con barbiquejo)		REGULAR	
Herramientas Manuales Adecuadas		REGULAR		B. Protección Ocular (Gafas de acuerdo al trabajo)		REGULAR	
Identificada Área de Seguridad		REGULAR		D. Protección Respiratoria (Mascarilla, respiradores)		BUENO	
Medio de transporte		REGULAR		E. Ropa de Trabajo (Uniforme entregado por EERSSA)		BUENO	
				G. Protección manos (Guantes de acuerdo nivel voltaje)		BUENO	
				H. Protección piernas (Guantes de trabajo - cuero nitrilo)		REGULAR	
				I. Protección extremidades inferiores (Cinturón de seguridad)		BUENO	
				L. Equipo de posicionamiento (Cinturón seguridad)		REGULAR	
				M. Andamios y/o escaleras (Herr. trab. en altura)		BUENO	
				N. Herramienta aislada		REGULAR	
PRECAUCIONES ADICIONALES O CARENCIAS:							
PRECAUCIONES: Cumplir con el código de ética de la EERSSA. Coordinar maniobras con el Centro de Control. Cumplir con las 5 reglas de oro.							
CARENCIAS: FALTA DE MATERIALES PARA MANTENIMIENTO / FUSIBLES							
SOLICITUD DE DESCONEXIÓN ELÉCTRICA							
Solicitud de desconexión o aislamiento eléctrico del siguiente equipo:							
Equipo No.:		Ubicación:		Nombre a quien solicitó:		Fecha:	
Descripción:							
CONFIRMACIÓN DE DESCONEXIÓN ELÉCTRICA							
Confirma que el equipo ESTÁ DESCONECTADO eléctricamente desde:							
Hora y fecha (h:m:s día/mes/año):		Firma:					
Instalación de puesta a tierra:		Medidas complementarias:					
TRABAJO CANCELADO (SI APLICA)							
Hora y fecha (h:m:s día/mes/año):		Responsable:					
Causas de cancelación del trabajo:		Firma:					
AUTORIZACIONES:							
NOMBRE: LITUMA CORDOVA CESAR RUBEN		PALACIOS MERINO ERNESTO VLADIMIR		QUEZADA ORDOEZ ANDREW ISRAEL			
FECHA: lunes, 16 de septiembre del 2024		JEFE ZONAL		INGENIERO 1			
FIRMA:							
SOLICITADO POR:		AUTORIZADO POR:		APROBADO POR:			

Gualaquiza Z1 (Cuadrilla, Nro. 3)		Fecha: lunes, 16 de septiembre del 2024		No: 137.601			
VEHICULO:	Número: 4-62	Placa: LEI-1053	Rentado: NO	Chofer: GUZMAN BARRIOS MARCO FERNANDO			
KILOMETRAJE:	Inicial: 274.921	Final: 275.039	Recorridos: 118				
ACTIVIDADES							
No	ACTIVIDAD	DESCRIPCIÓN DE LA ACTIVIDAD REALIZADA	LST/ALIMEN/SUBEST	TIPO	INICIO	FIN	
1	PROG	Se elabora la Orden de trabajo y se carga las herramientas y materiales en el vehículo.		RUTINARIA	2024-09-16 07:50:00	2024-09-16 08:25:00	
2	TRANSP	Traslado de la agencia de Gualaquiza al sector de Guabi Alto.		TRANSPORTE	2024-09-16 08:25:00	2024-09-16 09:05:00	
3	PROG	En Guabi Alto se realiza revisión de la LMV por est. desde la Est # 126968, se encuentra vegetación sobre dicha línea entre las est # 126976 y 126977.	ALI	Bomboiza	CORRECTIVO	2024-09-16 09:05:00	2024-09-16 13:00:00
4	ALIMEN	LUNCH EN EL SECTOR DE GUABI ALTO		ALIMENTACI	2024-09-16 13:00:00	2024-09-16 14:00:00	
5	PROG	Se realiza tala de árboles y desbroce de vegetación, en la est # 126968 se cambia una tira fusible de 20 A T y se restituye el servicio.	ALI	Bomboiza	CORRECTIVO	2024-09-16 14:00:00	2024-09-16 16:00:00
6		CARGA: 53 KVA FUSIBLE: 1x 20 A-T FECHA DESC: 15/09/2024 05:00 aprox. FECHA CONX: 15:45 CAUSA: vegetación					
7	TRANSP	DAÑO REPORTADO POR CC. (Mensaje de WhatsApp) Traslado desde el sector de El Guabi Alto al sector de San Pedro. Informar sector sin energía.			TRANSPORTE	2024-09-16 16:00:00	2024-09-16 16:55:00
8	NO PROG	En el sector de San Pedro en la Est # 28530 se encuentra abierto el breaker del tramo # 24463 de 10 KVA, se revisa la L/B/V y nose encuentra ninguna novedad.	ALI	Bomboiza	CORRECTIVO	2024-09-16 16:55:00	2024-09-16 17:25:00
9		Se rehabilita el tramo a las 17:20 y se restituye el servicio.					
10	TRANSP	DAÑO REPORTADO POR CC. (Mensaje de WhatsApp) Traslado desde sector de San Pedro a la Recta de Sevilla. Informar sector sin energía.			TRANSPORTE	2024-09-16 17:25:00	2024-09-16 17:50:00
11	NO PROG	En la recta de Sevilla en la est # 40375 se realiza el cambio de una tira fusible de 0.4 A SR en la derivación que va para un transformador privado de 15 KVA	ALI	Bomboiza	CORRECTIVO	2024-09-16 17:50:00	2024-09-16 18:05:00
12	TRANSP	DAÑO REPORTADO POR CC. (Mensaje de WhatsApp) Traslado desde el sector de La Recta de Sevilla al sector de El Guayuzal. Informar sector sin energía.			TRANSPORTE	2024-09-16 18:05:00	2024-09-16 18:30:00
13	NO PROG	En el sector de El Guayuzal en la est # 128503 se realiza el cambio de una tira fusible de 0.4 A SR derivación para un tramo privado de 10 KVA.	ALI	Bomboiza	CORRECTIVO	2024-09-16 18:30:00	2024-09-16 18:55:00
14	TRANSP	Traslado del sector de El Guayuzal a la agencia de Gualaquiza.			TRANSPORTE	2024-09-16 18:55:00	2024-09-16 19:15:00
15							
16		SE LABORA: CL, FG y MG, de 07:50 a 13:00 y de 14:00 a 19:15					
17							
OBSERVACIONES:							
TRABAJO TERMINADO							
Terminado:		TERMINADO					
Fecha y hora:		16/09/2024 19:15:00					
Responsable:		LITUMA CORDOVA CESAR RUBEN		Observaciones: Se reportan accidentes?			
		PALACIOS MERINO ERNESTO VLADIMIR		NO			

Ilustración 6. Orden de Trabajo típica. Elaboración propia.

La Ilustración 6 muestra una orden de trabajo detallando las actividades de la Cuadrilla Gualaquiza del 16 de septiembre del 2024, se ha resaltado las diferentes secciones que contienen información útil para su extracción. Cada Orden de Trabajo (OT) consta de al menos dos páginas, en la primera página se encuentra información general del grupo de trabajo, la planificación de los trabajos a realizarse y una sección para las autorizaciones (firmas), en esta primera hoja se diferencian las siguientes secciones:

1. Personal involucrado (zona amarilla): contiene información del personal que conforma el grupo de trabajo (cuadrilla)
2. Movilización (zona rosa): contiene información del vehículo utilizado para el transporte del personal.
3. Fecha (verde intenso): el día y fecha en el cual se desarrollan las actividades.
4. Precauciones adicionales (verde lima): indicaciones adicionales al personal para la correcta ejecución de sus actividades e información acerca de las carencias de la cuadrilla.

En la segunda página se resalta: el kilometraje recorrido por el vehículo (verde intenso), y la tabla de Actividades, donde se detallan los eventos desarrollados en columnas definidas. Cada actividad realizada por la cuadrilla es una fila dentro de la tabla, se destacan las siguientes secciones:

1. Registro del tipo de actividad:
 - a. Programada: planificada al inicio de la jornada laboral
 - b. No programada: actividad emergente que requiere atención inmediata.
 - c. Transporte: tiempo utilizado en la movilización del personal
 - d. Alimentación: tiempo utilizado en ingerir alimentos y descanso.
2. Descripción de la actividad:
 - a. El usuario es libre de detallar la actividad realizada.
 - b. No existe un formato definido para el registro de actividades, pese a ello se fomenta el uso de buenas prácticas, registrar información detallada indicando numeraciones de postes, medidores de energía, sectores. Su correcta aplicación depende del usuario y del revisor.
 - c. Se fomenta el uso de palabras clave que permitan identificar rápidamente el tipo de actividad que se está desarrollando.
 - d. Una misma actividad se puede extender en varias filas, se lo realiza de esta manera para detallar la información y rodear la limitación de la cantidad de caracteres en una sola fila. Esto debe ser considerado cuando se procesa el texto de cada actividad.
3. Alimentador: Se refiere al ramal eléctrico en el cual se desarrollan las actividades.
 - a. El sistema eléctrico está dividido en ramales para cada área geográfica a la cual se brinda el servicio.
 - b. Es crucial identificar adecuadamente el alimentador en el cual se ha laborado para la elaboración de los informes mensuales.

- c. El usuario debe escoger el nombre del alimentador desde un listado predefinido, esto elimina la posibilidad de una mala escritura, sin embargo, puede equivocarse en escoger el nombre correcto.
- 4. Tipo de intervención: es una clasificación adicional que permite identificar el tipo de actividad desarrollada.
 - a. Correctiva: se utiliza este identificador cuando la intervención realizada es para corregir un problema ya presentado y, que requiera de la suspensión temporal del suministro eléctrico.
 - b. Preventiva: se utiliza en los trabajos encaminados a prevenir un fallo en el sistema eléctrico y que no requieren de la suspensión del servicio.
 - c. Predictiva: se utiliza para la revisión e identificación de problemas potenciales en el tendido eléctrico, la diferencia con una actividad preventiva radica en que solo se registra y planifica la intervención.
 - d. Rutinaria: se utiliza para las actividades diarias que están relacionadas indirectamente con las labores preventivas y correctivas. El mantenimiento de vehículos y herramientas, reuniones de trabajo, revisiones e inspecciones de medidores, etc.
 - e. Expansión: sirve para identificar trabajos encaminados a la ampliación del servicio eléctrico, típicamente mediante la instalación de nuevos medidores a la red eléctrica, o la instalación de nuevas luminarias.
- 5. Registro del tiempo empleado en la actividad: Se detalla la fecha y hora tanto del inicio del evento como de su finalización. Debe existir una concatenación de eventos uno tras otro, es común que se den errores tanto en la fecha como en el registro de las horas, actualmente es tarea del revisor validar que estos datos sean los correctos.

Todos los días laborados son: planificados, ejecutados e informados utilizando este documento.

3.1.1. Procesamiento de texto

Si bien es cierto que la organización cuenta con un sistema para la creación y el almacenamiento de esta información, una base de datos relacional. Se ha elegido que la ingesta de datos no se la realice directamente de esta base de datos, sino más bien del documento final en formato PDF por las siguientes razones:

No toda la información contenida en la base de datos es una OT válida

Siendo que los trabajadores son quienes elaboran estos documentos, es muy común que el proceso de registro de actividades sea necesario realizar correcciones, cambios de planificación, o incluso el abandono y cambio por una nueva orden de trabajo. Es la responsabilidad de la persona que revisa y aprueba el trabajo realizado asegurar que el contenido del documento sea correcto. Este es un paso muy importante y que se lo realiza a conciencia ya que el documento de trabajo sirve para la elaboración de diversos informes mensuales, por ejemplo, para el pago de sobretiempos, un proceso

para el cual se necesita información clara y concisa. Una vez que el revisor está conforme con el documento generado proseguirá con su legalización (firma) y es también en ese instante que capturamos la información para su procesamiento.

El acceso directo a la base de datos es restringido

Como toda organización, la empresa eléctrica cuenta con un departamento informático, en el cuál es el encargado del acceso a la información, al tratarse de un proyecto investigativo se ha autorizado el uso de la información, más no el acceso directo a la base de datos. Esto conllevaría un largo y tedioso proceso, no solamente burocrático sino también de auditoría del sistema, sistema que se encuentra en desarrollo.

Por estas razones lo más conveniente es el procesamiento del texto contenido en los archivos finales de una orden de trabajo, los cuales habrán pasado ya por un primer filtro, simplificando en gran medida la labor de la gestión de estos documentos.

Esto nos puede suponer una ventaja, ya que se puede automatizar ciertos aspectos de la revisión del documento, verificación de fechas, datos faltantes y/o erróneos, y esta información puede ser retroalimentada al revisor en tiempo real, siendo la autoridad que decide si es procedente la modificación del documento.

3.2. Arquitectura del sistema

El proceso de diseño de cualquier arquitectura de datos inicia con la identificación de los sistemas de origen y planificar la mejor estrategia para su implementación. El primer paso consiste en delimitar rigurosamente los sistemas de origen y establecer la estrategia de ingesta de datos más eficiente. Por lo que, es crucial decidir si se utilizarán zonas de aterrizaje intermedias que faciliten la organización y transformación de los datos antes de su integración final en la Arquitectura Medallón. Paralelamente, se debe seleccionar el paradigma de ingesta adecuado (por lotes o en tiempo real), ya que esta decisión impactará directamente en las herramientas y técnicas que se utilizarán para la integración, orquestación y gestión de las tablas de datos.

Ingesta y Capa Bronce: La Verdad Inalterada

Con base en la justificación de la necesidad de una plataforma de *streaming* distribuido y los principios de la Arquitectura Medallón, es posible identificar los componentes principales que constituyen la arquitectura del sistema propuesto. Se ha identificado un primer punto de partida, la ingesta distribuida de documentos, las órdenes de trabajo. Estos archivos son el activo de información primario y deben preservarse con la máxima fidelidad a la fuente original. Los aspectos principales son:

- El objetivo es garantizar la inmutabilidad y la trazabilidad de los datos.
- La Capa Bronce actúa como el repositorio de datos crudos. Los documentos se almacenarán sin alteraciones significativas, sirviendo como la fuente de verdad inalterada para fines de auditoría y reproducibilidad de datos.

Estandarización y Capa Plata

Una vez asegurada la persistencia del dato original, el siguiente paso es la transformación para la utilidad analítica. En esta capa **se realiza la limpieza de los datos previo a ser enriquecidos**. Los aspectos principales son:

- El objetivo es organizar, limpiar y enriquecer la información.
- La Capa Plata permite realizar esta estandarización de formatos, se aplica un esquema de datos, y se aplican rutinas de limpieza para mitigar valores anómalos o incompletos.

Contextualización y Capa Oro.

El paso final se centra en la contextualización y la destilación de la información para el consumo directo del negocio. Los aspectos principales son:

- El objetivo es obtener una versión final, agregada y altamente enriquecida, que permita la generación de valor para la empresa.
- Se aplican transformaciones de valor agregado, como la unión de las órdenes de trabajo con datos maestros (geográficos, de personal, de equipos) para entender el contexto operativo completo. Esta Capa Oro contiene datasets altamente optimizados, listos para ser consumidos por herramientas de *Business-Intelligence*, modelos de *Machine Learning* y reportes estratégicos.

Este diseño en capas no solo separa las preocupaciones de cada etapa del pipeline (ingesta, limpieza, consumo), sino que también garantiza la fiabilidad, escalabilidad y adaptabilidad necesarias para un sistema de información moderno.

3.2.1. La ingesta de datos

La ingesta de datos representa la primera aplicación de un microservicio, como ya se ha explicado. La ingesta de datos debe realizarse de manera distribuida en cada computador de cada aprobador, este microservicio se encarga del procesamiento de los documentos y de enviarlos hacia el bróker más cercano en formato JSON. Se ha elegido este formato de archivo ya que es un tipo de archivo flexible que permite adaptarse a la información contenida en los documentos, la cual puede igualmente llegar a ser muy variada en su contenido.

Es importante proveer al usuario de una respuesta rápida en su interacción con el sistema, por ello este microservicio implementa un paradigma de *Streaming* y se centra en su tarea fundamental que es la de procesar los archivos de formato PDF a formato JSON, enviarlos hacia Kafka y de proveer retroalimentación al revisor acerca del documento procesado. Es así como este primer microservicio tendrá el rol de **productor** dentro del paradigma planteado por Kafka.

3.2.2. Organización de documentos

Una vez que los documentos se encuentran en el registro de Kafka es necesario administrarlos, es decir, se debe verificar si se trata de un nuevo documento, o si se ha actualizado uno existente, además, las transacciones deben cumplir con los principios de Atomicidad, Consistencia, Aislamiento (*Isolation*), Durabilidad (ACID), para cumplir con estos requerimientos es necesario hacer uso de una base de datos, lo más natural es que al tratarse de documentos en formato JSON se implemente en una base de datos NoSQL de tipo documental, por lo tanto, se ha elegido la implementación de la base de datos *MongoDB*, si bien esta tecnología no cumple con ser considerada estrictamente *Open Source* como el resto del *stack* tecnológico a utilizar si cuenta con una edición comunitaria, y de una versión en la nube, resulta de muy fácil integración por lo que se ha elegido que los documentos sean almacenados en su versión *cloud MongoDB Atlas*, esto permite la gestión remota de los archivos y el prototipado de diversas soluciones que involucren el uso de las ordenes de trabajo.

Al igual que el microservicio *productor* se ha implementado un microservicio *consumidor* el cual se encarga de organizar y gestionar la base documental en el servidor *MongoDB*, actualizando los documentos cuando sea el caso e insertando nuevos cuando se requiera. Este microservicio también hace uso del paradigma *Streaming* para el consumo de los datos enviados por los productores.

Adicionalmente a su labor como organizador, este microservicio es el encargado de publicar aquellas nuevas órdenes de trabajo que se han agregado al servidor para que otros microservicios puedan hacer uso de ellas. Por lo tanto, este microservicio también publica en Kafka los identificadores únicos de las órdenes de trabajo.

3.2.3. Consolidación de actividades

Si bien hasta este punto de la arquitectura se encuentran las órdenes de trabajo en una base de datos, es necesario realizar la tarea de consolidación de las actividades, esto sirve para analizar y limpiar la información, asegurándonos que la misma sea correcta.

En la arquitectura de Medallón, este paso no se debe realizar sobre la misma base de datos Bronce, ya que se desea mantener esta parte del sistema como futura referencia y como una base fidedigna del documento elaborado por los trabajadores. Al igual que con la organización de los documentos, para este paso se requiere de una base de datos que garantice contar con los principios ACID, ya que se tienen lecturas, escrituras y actualizaciones constantemente, tanto por nuevos documentos agregados como por correcciones y ampliaciones a la base de datos.

Para cumplir con este requerimiento se disponen de varias posibles soluciones, incluyendo una base de datos relacional, pero al proyectar esta solución a futuro lo mejor es implementar una solución *Lakehouse*. La información permanece disponible y organizada, permite una consulta rápida de tipo OLAP y permite además que la

información pueda crecer manteniendo un formato eficiente que permita optimizar las consultas históricas. Para ello se ha elegido la implementación del sistema *Delta Lake*, el cual cumple con todas las capacidades necesarias. Es un proyecto de código abierto que cuenta con soporte y documentación que facilita su implementación, además cuenta con modernas librerías para su manejo permitiendo un rápido despliegue e implementación.

3.2.4. Limpieza de datos

La limpieza de los datos puede ser llevada a cabo por el personal aprobador, o cualquier personal técnico autorizado para la revisión de las actividades. Este personal cuenta con la experiencia técnica en la rama de la ingeniería eléctrica pero no cuenta necesariamente con las habilidades de programación necesarias para llevar a cabo las tareas de limpieza desde un lenguaje como puede ser Python. Es necesario por lo tanto brindar a estos técnicos una interfaz que les permita interactuar con los datos en un ambiente controlado, evitando causar desprevénidamente la eliminación de información de forma involuntaria.

Nuevamente se cuenta con varias soluciones que se pueden implementar, la más completa puede pasar por crear una interfaz web a medida, que parametrize y restrinja el tipo de acciones que puede llevar a cabo el usuario. Esta solución inevitablemente requeriría de grandes recursos y es algo que está más allá del alcance de este trabajo de titulación, el cual está enfocado en un desarrollo de prototipado rápido, que permita la iteración y despliegue de interfaces, es así que se ha optado por el desarrollo de “*notebooks*” interactivos implementados mediante la tecnología denominada *Marimo Notebook*.

Se trata de un nuevo enfoque que expande las capacidades de los tradicionales *Jupyter Notebooks*, mantiene la estructura de bloques de código y expande las capacidades volviéndolo un notebook interactivo. Cuenta con librerías enfocadas al análisis de datos que permiten la visualización y edición de datos tabulares. Permite el desarrollo del notebook en un entorno Python y para su despliegue se puede ocultar el código fuente exponiendo al usuario solamente los controles interactivos, actuando efectivamente como una interfaz para interactuar y editar *Delta Lake*.

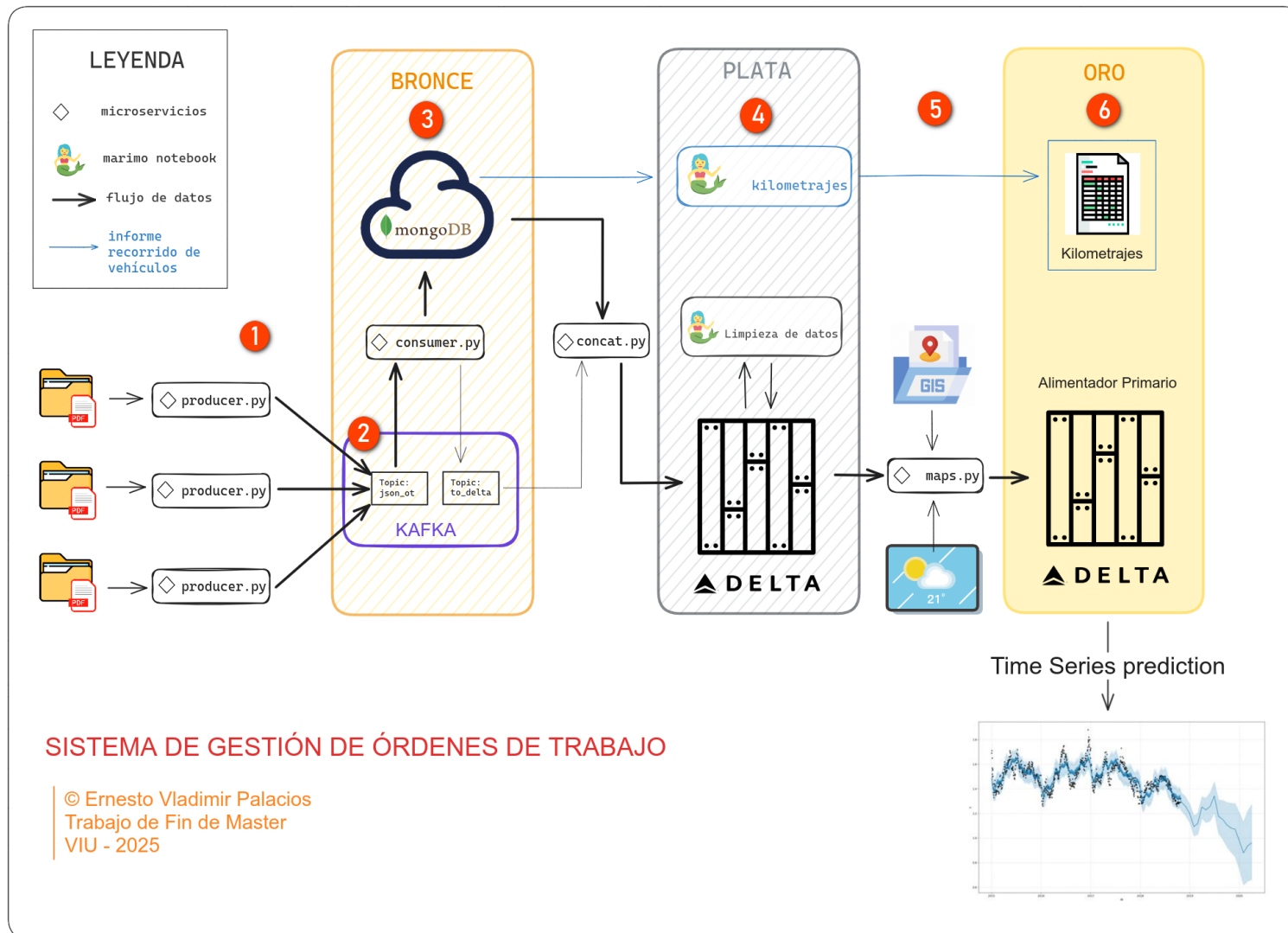


Ilustración 7. Arquitectura del Sistema de Gestión de Ordenes de Trabajo para la EERSSA. Elaboración propia

3.3. Solución propuesta

En la Ilustración 7 se presenta la arquitectura completa del Sistema de Gestión de Órdenes de Trabajo que se implementó en el presente trabajo de titulación. Se distinguen las capas de la arquitectura Medallón, así como el flujo de la información a través del pipeline de datos a media que va siendo transformado, revisado, y ampliado.

① CAPA DE INGESTA DISTRIBUIDA

Los revisores, distribuidos geográficamente en diferentes cantones de Zamora Chinchipe, aprueban órdenes de trabajo (OT) generadas por las cuadrillas. El microservicio “*producer.py*”, instalado en cada estación de revisión, procesa los archivos PDF localmente, extrae la información, y publica los documentos en formato JSON hacia el tópico de Kafka “*json_ot*”.

② SISTEMA DE MENSAJERÍA (Apache Kafka)

Kafka actúa como sistema nervioso central del pipeline, implementa el patrón Pub/Sub que desacopla completamente productores de consumidores. Se configuran dos tópicos principales:

- “*json_ot*”: Recibe documentos JSON desde productores
- “*to_delta*”: Transmite identificadores de OT para procesamiento en Delta Lake

③ CAPA BRONCE - Almacenamiento Documental (MongoDB Atlas)

El microservicio “*consumer.py*” consume del tópico “*json_ot*”, valida la estructura de cada documento y lo almacena en *MongoDB Atlas*. Se mantienen dos colecciones:

- “*ot_v30*”: Colección principal de documentos validados
- “*ot_reload*”: Colección de *staging* para versiones actualizadas pendientes de validación

Esta capa garantiza la inmutabilidad y trazabilidad completa de los datos originales.

④ CAPA PLATA - Transformación y Consolidación (Delta Lake)

El microservicio “*concat.py*”, consume identificadores del topic “*to_delta*” mediante ventanas de saltos (*tumbling window*) cada 5 segundos. Transforma los documentos JSON en formato tabular, aplica:

- Corrección automática de fechas erróneas.
- Clasificación del texto de las actividades mediante *Naive Bayes*.
- Normalización de esquemas.

Los datos transformados se escriben en Delta Lake, aprovechando transacciones ACID y capacidades de versionamiento (*time-travel*) para auditoría. Desde aquí se continúa con el enriquecimiento de la información.

⑤ **CAPA DE ENRIQUECIMIENTO**

Se integran fuentes de datos adicionales:

- Información geográfica: Mapeo de postes a coordenadas GPS con las que ya cuenta la organización.
- Datos climatológicos: Condiciones meteorológicas por fecha/ubicación.
- Metadatos de personal: Composición y especialización de cuadrillas.

Este enriquecimiento contextualiza las actividades para análisis avanzado.

⑥ **INTERFAZ DE LIMPIEZA Y VALIDACIÓN (Marimo Notebooks)**

Personal técnico accede a notebooks interactivos que exponen controles para visualización y edición de datos tabulares en Delta Lake, sin necesidad de conocimientos de programación. Permite corrección manual de clasificaciones erróneas y validación de actividades complejas.

⑦ **CAPA ORO - Consumo Analítico**

Datasets altamente curados alimentan:

- *Dashboards* de *Business Intelligence*
- Modelos de *Machine Learning* para predicción de fallas
- Generación automatizada de informes mensuales
- APIs para integración con sistemas externos: Registros biométricos, Despachos de combustible.

3.3.1. Configuración de Apache Kafka

La implementación y configuración de Apache Kafka es un tema profundo que requiere un análisis exhaustivo en cuanto a las capacidades y prestaciones que se espera obtener. Una correcta y confiable instalación y configuración de Apache Kafka es crucial para el funcionamiento del sistema desarrollado. Esta configuración es dependiente de las capacidades actuales y futuras de la organización y está más allá del alcance de este TFM.

Una de las grandes ventajas que brinda Apache Kafka es que el desarrollo de los microservicios y su comunicación es independiente de esta configuración, por esta razón se ha optado por una implementación rápida que permita el desarrollo de los microservicios asociados, esto lo logramos a través de la tecnología de contenedores, específicamente *Docker*.

Docker

La solución para un despliegue rápido que permite virtualizar y desplegar una instancia de Apache Kafka configurada y lista para ser utilizada se logra mediante la tecnología de contenedores virtualizados de Docker. Con el agregado adicional del despliegue de la JVM, crucial para el uso de Kafka, se encuentra dentro del mismo contenedor evitando así el mantenimiento desde el entorno de desarrollo.

Se ha implementado Apache Kafka en su versión 7.4.4 desde la imagen oficial del desarrollador. Gracias a la tecnología de Docker ejecutando unos pocos comandos se cuenta con un servidor de Apache Kafka en modo local para el desarrollo de nuestro sistema.

3.4. Extracción de datos (productor)

Este microservicio es el más extenso y cargado de características, es posible que el término “microservicio” no sea del todo aplicable, dicho esto se ha optado por no particionar sus funcionalidades en varios sub-programas ya que está destinado a ser instalado en los computadores de cada revisor, y para futuras auditorías será más sencillo entregar un solo programa para su análisis y aprobación. A continuación, se realizará un análisis detallado sus características y capacidades como productor de la información.

Lectura de archivos PDF

Existen varias librerías para el lenguaje Python que permiten la lectura de archivos en formato PDF, en las órdenes de trabajo la información se encuentra organizada en varias tablas de datos, siempre ubicadas en el mismo lugar del documento, lo que facilita su ubicación y lectura. Dos de estos lectores de PDF se destacan: *tabula*, un proyecto basado en el lenguaje Java con una interfaz para Python y, *MuPDF*, basado en C++ igualmente con una API para Python llamada *PyMuPDF*. Luego de analizar y probar las dos librerías se ha decidido hacer uso de la librería con tecnología en C++, la razón, no es necesario instalar, configurar y mantener una Máquina virtual de Java (JVM), sin este requerimiento es posible la paralelización en la ingesta de documentos.

Computación paralela

Uno de los objetivos es el procesamiento de datos históricos, esto requiere procesar años de archivos PDF almacenados, desde septiembre del 2018 específicamente, esto crea un cuello de botella, ya que el lenguaje Python es primordialmente *single thread*, es decir, normalmente utiliza uno solo de los núcleos del cómputo. Esto es una limitación importante, para sortearla se ha implementado el *framework* de cómputo distribuido DASK, el cual cuenta con una librería en Python que permite paralelizar el cómputo, tanto de manera local (varios núcleos), como de manera distribuida, mediante la conexión con servicios especializados de procesamiento paralelo masivo en la nube, sin que esto se traduzca en grandes cambios en el código fuente.

Conexión con Apache Kafka

Existen librerías para Python desarrolladas por los mantenedores de Apache Kafka, existe una versión basada en JVM y otra en C/C++ las cuales cuentan con las funcionalidades básicas y necesarias para ser utilizadas en este proyecto. Dicho eso, se ha optado por una tercera opción, la comunicación con Apache Kafka se la realiza mediante la librería *Quix*, la cual es un cliente implementado directamente en Python de manera nativa, la razón para su uso es su fácil escalabilidad, si bien en el estado actual del proyecto utilizamos sus funciones básicas, la principal ventaja es que permite el procesamiento de los datos en *streaming*, operaciones de ventana (*windowing*), y principalmente, “*stateful operations*” operaciones de *streaming* que dependen del último dato obtenido, el cual puede afectar el estado actual de los datos (Quix Analytics, 2025). Esta librería será crucial para el desarrollo del proyecto más allá del alcance del presente TFM.

Retroalimentación del estado de la Orden de Trabajo

Es importante proveer de la información relevante e informar al revisor en el caso de encontrar datos que resulten conflictivos, esto con el objetivo de que puedan ser corregidos desde el origen y con prontitud. Para implementar esta funcionalidad se genera un reporte en caso de que se encuentre alguna novedad con la orden de trabajo. Este reporte será igualmente en formato PDF, brinda información general de la orden de trabajo y también indica cada novedad encontrada, esta información es comunicada utilizando una paleta de colores que permita identificar rápidamente diferentes tipos de novedades. Esto se logra utilizando la librería *Typst*, una alternativa a Látex para la generación de documentos PDF.

Tabla 3. Niveles de criticidad en el reporte de orden de trabajo. Elaboración propia

Nivel	Color	Detalle
Fatal	ROJO	No es posible enviar esta orden de trabajo al bróker/servidor. Se detalla el motivo
Error	NARANJA	Es una orden de trabajo válida, se envía al servidor De ser el caso revisar y corregir las novedades detalladas. De ser el caso se puede reenviar una versión corregida de la orden de trabajo.

Nivel	Color	Detalle
Revisión	AZUL	Es una orden de trabajo válida, se envía al servidor Se ha detectado novedades no graves. De ser el caso se puede reenviar una versión corregida de la orden de trabajo.
Informativo	VERDE	Información de la orden de trabajo, nombre del archivo que se procesa, fecha y hora del procesamiento.

La Tabla 3 nos resume los diferentes niveles de criticidad, cada nivel tiene un color distintivo dependiendo del tipo de evento encontrado, el objetivo es identificar rápidamente las novedades que requieran de atención prioritaria.

En el [Apéndice III](#) se puede visualizar un ejemplo de este tipo de documento.

Este documento solo se genera en el caso de que se haya detectado alguna novedad que requiera la atención del revisor. Esto simplifica el proceso de revisión y carga de información ya que centra la atención del revisor en aquellos documentos que requieren ser revisados minuciosamente.

Una vez se ha extraído la información del documento se verifica que mantenga una consistencia interna. Se verifica:

- **Que el archivo PDF sea una orden de trabajo:** Se revisan marcadores, texto en posiciones específicas, si no se los encuentra no es una orden de trabajo.
- **Que existan actividades registradas:** puede darse el caso de que aún no se han ingresado las ordenes de trabajo y la segunda hoja se encuentre vacía.
- **La fecha de la orden de trabajo:** hay varios campos donde se ingresa la fecha, se verifica que sea la misma en todos estos campos.
- **Reporte de carencias:** En el caso de que se reporten carencias de herramientas y/o materiales esto debe ser considerado por el revisor.

Durante el desarrollo de este microservicio (*producer.py*) se probaron varias tecnologías, procurando, en lo posible, minimizar la cantidad de dependencias adicionales a Python con la finalidad de que su implementación sea ligera y de bajo mantenimiento.

Tabla 4. Funcionalidades y sus librerías en el microservicio productor. Elaboración propia.

Librería	Aplicación
PyMuPDF	Extracción de texto desde el documento PDF
DASK	Procesamiento local en paralelo de documentos.
Quix	Cliente de Apache Kafka para la transmisión de datos.
Typst	Generación de reportes en formato PDF.

La Tabla 4 nos resume las funcionalidades con las que cuenta el microservicio llamado: *producer.py*, se indica el nombre de la librería y como esta ha sido utilizada en el microservicio.

3.4.1. Configuración de MongoDB

En el caso de MongoDB se ha hecho uso de su servicio en la nube llamado MongoDB Atlas, la cual resulta muy beneficioso para el desarrollo del sistema al permitir acceder a los datos extraídos de una manera más sencilla y desde diferentes entornos de desarrollo, en particular el uso de *Google Colab* como ambiente de desarrollo y prototipado. Como se mencionó anteriormente, es posible la instalación de la versión *MongoDB Community* para su uso en local, o incluso, de ser necesario se puede llevar a cabo una migración hacia otros proyectos similares, como por ejemplo *FerretDB* el cual es altamente compatible con el protocolo implementado por *MongoDB*. (Google Gemini, 2025)

En este punto del desarrollo se priorizó una rápida puesta en marcha enfocando los esfuerzos en resolver los problemas puntuales del sistema y aprovechando las funcionalidades que nos brinda la versión gratuita de *MongoDB Atlas*.

Se configura MongoDB con dos colecciones (bases de datos), una principal denominada "*ot_v30*" donde se guardan todas las órdenes de trabajo procesadas para su posterior consumo, y una segunda colección auxiliar llamada "*ot_reload*", destinada a un proceso de espera y validación "*staging*" para aquellas órdenes de trabajo que ya existen en la base de datos principal pero que aún deben ser analizadas y procesadas antes de decidir si es necesario reemplazar a un documento de la colección principal.

3.5. Carga de datos (consumidor)

El programa *consumer.py* establece comunicación con el servidor MongoDB, luego recibe y procesa los documentos en formato JSON provenientes desde Apache Kafka, envía estos documentos hacia MongoDB Atlas y publica los identificadores únicos de los documentos procesados. Para obtener los documentos desde Kafka hace uso de la librería Quix. Su operación se desarrolla en tres etapas principales: establecimiento de conexión, validación e inserción de datos, y publicación de resultados.

Previo a insertar un nuevo documento en MongoDB, el programa verifica la existencia de este registro en la colección principal. Dependiendo del resultado de esta validación el documento se dirige a una de dos bases de datos: si se trata de un nuevo documento, se inserta en la base de datos "*ot_v30*"; si por el contrario, el documento corresponde a una versión actualizada de uno existente, se almacena en la base de datos "*ot_reload*".

Una vez completada la carga de documentos en la colección correspondiente se procede a publicar en Apache Kafka las órdenes de trabajo disponibles para procesamiento. Para ello, el consumidor publica los identificadores únicos de las órdenes de trabajo en el tópico denominado: "*to_delta*". Dentro de cada mensaje publicado se especifica si corresponde a un registro nuevo o una orden de trabajo existente para ser analizada.

La inserción de documentos en el servidor MongoDB se la realiza mediante el paradigma *streaming* procesando los documentos conforme estos están disponibles y publicando simultáneamente los identificadores únicos de las órdenes de trabajo.

El microservicio encargado de consumir estos identificadores implementa el paradigma de "ventana de saltos" (*tumbling window*) con la finalidad que el procesamiento se lleve a cabo en estrategia de *micro-batch*, lo cual permite optimizar el proceso de escrituras continuas en la base de datos tabular.

Al implementar el paradigma "ventanas de saltos" es crítico asegurarnos de que el último dato enviado al bróker sea efectivamente procesado. Se amplía este concepto en el [Apéndice I](#). Esta necesidad de procesamiento se logra mediante la aplicación de un mecanismo de "latido de corazón" (*heartbeat*), cuya finalidad es garantizar que los datos sean procesados completando cada ciclo de la ventana de salto.

El programa *consumer.py* implementa dos procesos paralelos independientes: el primero monitorea y transmite los mensajes identificadores de órdenes de trabajo, mientras que el segundo es responsable de enviar los mensajes de *heartbeat* al servidor. Esta arquitectura dual asegura que la totalidad de los mensajes en cola sean procesados correctamente. El concepto, funcionalidad y aplicación práctica del mecanismo de *heartbeat* se desarrollan extensamente en el [Apéndice I](#).

Tabla 5. Funcionalidades y sus librerías en el microservicio consumidor. Elaboración propia

Librería	Aplicación
Quix	Cliente de Apache Kafka para el consumo y publicación de datos.
Pymongo	Librería oficial para comunicación con MongoDB
Threading	Librería nativa de Python para procesos en paralelo

La Tabla 5 resume las tres librerías fundamentales que integran el microservicio consumidor, cada una con una funcionalidad específica en la arquitectura del sistema. Quix actúa como intermediario para la comunicación con Apache Kafka, permitiendo tanto el consumo como la publicación de datos en los tópicos requeridos. Pymongo facilita la interacción directa con MongoDB, posibilitando las operaciones de validación e inserción de documentos en las bases de datos "ot_v30" y "ot_reload". Finalmente, *Threading* proporciona las capacidades necesarias para implementar el procesamiento paralelo de dos procesos independientes: uno dedicado a la transmisión de identificadores de órdenes de trabajo y otro responsable del envío de mensajes de *heartbeat*. Esta combinación de librerías permite que `consumer.py` ejecute su operación de manera eficiente, integrando la ingesta de datos desde Kafka, su procesamiento y validación, y su almacenamiento en MongoDB, todo ello con garantías de procesamiento completo mediante mecanismos paralelos.

3.5.1. Configuración de Delta Lake

Delta Lake provee una capa de almacenamiento *open source* para alojar bases de datos tabulares. Databricks, la empresa creadora de Delta Lake ofrece una plataforma comercial en la nube que integra esta tecnología con motores de procesamiento distribuido como Apache Spark, Apache Flink y Trino. Esta plataforma está diseñada para organizaciones que gestionan volúmenes masivos de datos (Databricks Inc., 2025). Para el alcance actual del sistema desarrollado no se justifica la inversión en infraestructura de cómputo distribuido, se amplía este análisis en el [Anexo I](#).

Delta Lake provee la opción de su uso en modo local, ejecutándose directamente en el sistema de archivos del servidor, sin sacrificar las características fundamentales que lo distinguen de formatos tradicionales. Esto agiliza el desarrollo y experimentación de sus capacidades y características. La administración de Delta Lake en este proyecto se realiza mediante la librería "*delta-rs*", una implementación nativa en Rust con API para Python. Esta tecnología ofrece ventajas significativas sobre la implementación oficial en Scala/Java, principalmente su independencia de la JVM.

Esta combinación de factores la convierte en ideal para su implementación en el sistema desarrollado: permite el desarrollo local con tecnología de grado corporativo,

cuando el volumen de datos lo justifique se cuenta con herramientas que facilitan la migración hacia la nube.

Se configuró un repositorio Delta Lake local para almacenar datos de la capa Plata de la Arquitectura Medallón. La tabla principal "*delta_v30*" contiene información normalizada de órdenes de trabajo, optimizada para consultas analíticas y entrenamiento de modelos ML. Las operaciones de escritura se realizan mediante transacciones atómicas que garantizan consistencia.

3.6. Transformación de los datos

El tercer microservicio se encarga del procesamiento de datos alojados en los servidores MongoDB Atlas, realizando su transformación, limpieza y almacenamiento en la base de datos local Delta Lake. Esta distinción de roles obedece a las características operacionales de ambas plataformas: MongoDB Atlas, como base de datos documental en la nube, está optimizada para soportar un elevado volumen de operaciones de lectura y escritura simultáneas. (MongoDB, Inc., 2025) Por el contrario, Delta Lake, la base de datos local, está diseñada principalmente para operaciones de lectura de alto rendimiento, presentando limitaciones en cuanto a escrituras concurrentes (The Linux Foundation, 2025). Por esta razón, se implementa una estrategia de acumulación de operaciones mediante el paradigma de ventana de saltos, lo que permite implementar el consumo de identificadores de órdenes de trabajo en *micro-batching*, lo cual garantiza intervalos regulares entre cada operación de escritura hacia Delta Lake. Este enfoque permite compactar y **concatenar** los datos, facilitando una escritura eficiente. De esta estrategia emerge el nombre del microservicio: "concat.py".

Mecanismo de Micro-Batching

La ventana de procesamiento se ha configurado con una duración de cinco segundos, periodo durante el cual se reciben todos los identificadores de las nuevas órdenes de trabajo disponibles en MongoDB. Transcurrido este intervalo, se procesan secuencialmente los registros identificados.

Para los documentos clasificados en la colección "*reload_ot*", es decir, órdenes de trabajo repetidas, el microservicio efectúa una comparación entre las dos versiones existentes en MongoDB. En caso de detectarse diferencias, se procede a reemplazar y actualizar la versión desde *staging* hacia la colección principal. Si no se identifican diferencias, la versión en "*reload_ot*" se descarta, evitando así escrituras innecesarias tanto en MongoDB como en Delta Lake.

Transformación y Enriquecimiento de Datos

Una vez generado el listado de órdenes de trabajo para ser escritas en Delta Lake, se procede con su transformación de formato JSON a formato tabular. Durante este proceso de transformación, se incorporan columnas adicionales a las especificadas

originalmente en las actividades, las cuales se emplean posteriormente en el enriquecimiento de la información asociada.

Adicionalmente, durante la transformación se ejecuta la corrección de errores comunes en los registros de actividades. Debido a que el trabajador realiza la introducción de fechas para cada actividad (cada fila) resulta frecuente la ocurrencia de errores de digitación. El microservicio identifica y corrige estos errores antes de transmitir los datos hacia Delta Lake, garantizando la integridad de la información almacenada.

3.6.1. Procesamiento Natural del Lenguaje (NLP)

Un paso fundamental en la transformación y limpieza de las actividades realizadas por los grupos de trabajo es la identificación del nivel de carga eléctrica asociado a cada intervención. El nivel de carga eléctrica se define como el impacto que genera la actividad sobre la red de distribución. Dado que la organización gestiona eventos que afectan desde instalaciones residenciales individuales hasta la infraestructura de nivel provincial, resulta esencial clasificar las actividades de acuerdo con su alcance. Los niveles de carga se identifican conforme a la siguiente jerarquía estructural:

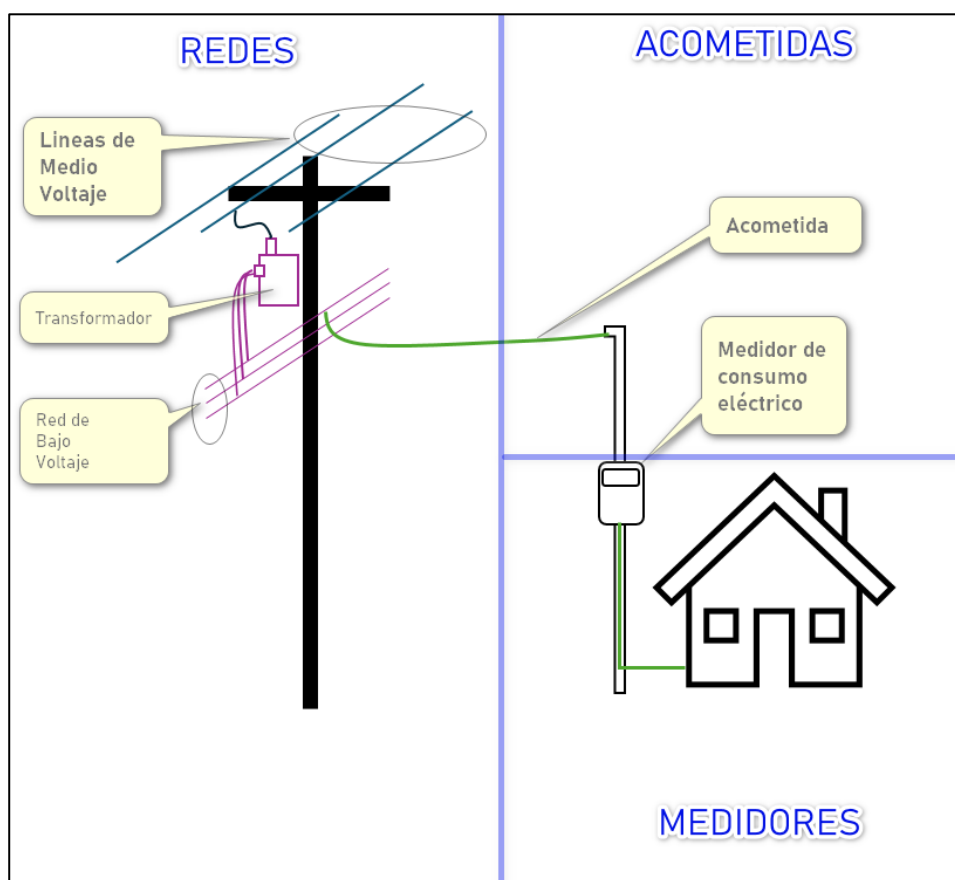


Ilustración 8. Diagrama del sistema de distribución eléctrico. Elaboración propia

La Ilustración 8 presenta el diagrama del sistema de distribución eléctrico, en el cual se especifican los niveles de carga de acuerdo con el tipo de actividad realizada por los grupos de trabajo.

Redes

Los trabajos ejecutados en las redes eléctricas comprenden las intervenciones efectuadas por las cuadrillas operativas a nivel de poste, red de bajo voltaje, transformador y línea de medio voltaje. La energía se distribuye desde la subestación eléctrica por medio de las líneas de medio voltaje (22,000 voltios), conducidas a través del tendido eléctrico sobre postes. Se denomina Alimentador Primario a la secuencia de postes que conforma un mismo ramal eléctrico; estos alimentadores se extienden varios kilómetros. Los transformadores se conectan a la Línea de Medio Voltaje (LMV) y transforman la energía para entregarla a la Red de Bajo Voltaje (RBV), lista para consumo doméstico (220 voltios). La RBV se extiende hasta una distancia típica de 100 metros y suministra servicio eléctrico a una manzana o vecindario.

Acometidas

Las acometidas constituyen las derivaciones que se conectan a la RBV, transportando la energía eléctrica desde la red principal hacia los medidores de consumo eléctrico ubicados en las instalaciones de los clientes.

Medidores

Los medidores son dispositivos encargados del registro del consumo eléctrico de las instalaciones residenciales. Es posible que múltiples medidores sean alimentados por una única acometida, como ocurre en edificios de apartamentos donde cada unidad de vivienda posee su propio medidor eléctrico.

Esta estructura jerárquica de niveles conlleva un significado organizacional considerable, resultando esencial para la generación de reportes y para la caracterización de las actividades ejecutadas por los grupos de trabajo.

Más allá de los tres niveles de carga eléctrica referidos anteriormente, se definen los siguientes tipos de actividad:

Alumbrado

Comprende el mantenimiento y reparación de luminarias destinadas al alumbrado público, instaladas en los postes de la infraestructura de EERSSA.

Nuevos Servicios

Se refiere a la instalación de nuevos medidores para nuevos clientes que se incorporan a la red de distribución eléctrica.

Servicios Ocasionales

Designa instalaciones eléctricas temporales cuyo período de funcionamiento no excede los tres meses.

Cada una de estas categorías se conoce en el contexto organizacional como Cuenta Contable, denominación que obedece a que estas actividades deben ser reportadas con la finalidad de documentar la distribución presupuestaria de la EERSSA en función del tiempo dedicado por cada grupo de trabajo a cada categoría específica.

Durante la transformación de los datos, se analiza el texto descriptivo de la actividad y se aplica un algoritmo probabilístico (*Naive Bayes*) para determinar la etiqueta correspondiente. Este mecanismo de entrenamiento y clasificación se describe en detalle en el [Apéndice II](#).

Una vez completados los procesos de transformación, realizadas las correcciones pertinentes y generada la etiqueta de la columna "Cuenta", se obtiene la denominada matriz de actividades. En este punto, la información se almacena en Delta Lake utilizando la librería 'delta-rs', completando así el ciclo de transformación y enriquecimiento de datos.

Tabla 6. Funcionalidades y sus librerías en el microservicio concatenador. Elaboración propia

Librería	Aplicación
Quix	Cliente de Apache Kafka para el consumo de datos.
SciKitLearn	Librería que permite la aplicación del análisis probabilístico del texto.
Delta Lake	API en Python para la desarrollada en <i>Rust</i> para el manejo de <i>Delta Lake</i>

La Tabla 6 sintetiza las tres librerías fundamentales que sustentan la funcionalidad del microservicio concat.py, cada una contribuyendo a un aspecto específico del procesamiento de datos. Quix proporciona la interfaz necesaria para consumir identificadores de órdenes de trabajo desde Apache Kafka, esta librería permite la implementación del paradigma ventanas de saltos. SciKitLearn posibilita la aplicación de algoritmos probabilísticos para el análisis y clasificación del texto descriptivos de las actividades. Finalmente, la API de Delta Lake en Python, facilita el manejo eficiente de operaciones de lectura y escritura en la base de datos local. Esta combinación de librerías permite que concat.py ejecute el ciclo completo de transformación, enriquecimiento y clasificación de datos provenientes de MongoDB, integrando procesamiento de lenguaje natural y garantizando el almacenamiento eficiente en Delta Lake.

3.6.2. Limpieza manual de actividades

La limpieza y corrección automática de datos debe complementarse con una revisión y validación ejecutada por revisores humanos. Es necesario considerar que los revisores no poseen un elevado conocimiento informático que les permita interactuar con la base de datos de una manera programática. Estas consideraciones hacen imperativa la implementación de una interfaz gráfica de usuario que facilite la revisión y corrección de datos de forma intuitiva y accesible.

El desarrollo de una interfaz gráfica personalizada desde cero constituye una tarea extensiva y laboriosa que, si bien podría producir resultados óptimos, este desarrollo trasciende el alcance de este trabajo de titulación. Ante esta limitación, se han evaluado alternativas que permitan el prototipado rápido, facilitando la recopilación de retroalimentación de usuarios y la ejecución de mejoras iterativas adaptadas a las necesidades reales. Esta búsqueda condujo a la implementación de *Marimo Notebooks* como solución tecnológica.

Marimo Notebooks es una tecnología que transforma los cuadernos de código tradicionales en interfaces interactivas. En su despliegue final es posible ocultar el código fuente, exponiendo al usuario únicamente los controles interactivos necesarios para visualizar y editar tablas de datos. De este modo, *Marimo Notebooks* funciona como una interfaz de usuario controlada que permite a los técnicos interactuar con y modificar datos en Delta Lake sin riesgo de alterar involuntariamente la integridad del sistema. Esta arquitectura posiciona a *Marimo Notebooks* como una solución efectiva de prototipado rápido (Marimo Inc., 2025)

La Ilustración 9 (abajo) presenta un ejemplo de notebook configurado para la revisión individual de órdenes de trabajo. El flujo operativo es el siguiente: el usuario ingresa el identificador único de la orden de trabajo, parámetro con el cual se consulta la base de datos para recuperar la información asociada. Los datos se presentan en un formato de tabla editable, permitiendo que el revisor examine el contenido y realice las correcciones que considere necesarias. Una vez que el revisor valida la conformidad de la información modificada, activa el botón "OK", acción que dispara el procesamiento y actualización de los registros en la base de datos Delta Lake.

REVISIÓN DE ÓRDENES DE TRABAJO

Seleccionar número de la OT para editar Identificador : pulse para ejecutar ->

	Item	Cuenta	Evento	Actividad	Alimentado
1	1	transporte	Nos trasladamos de la agencia Yantzaza a San Pablo, Subestación	TRANSP	-
2	2	REDES	Subestación Yantzaza. Con autorización del Ing. De turno y	NO PROG	E.E. Sur -
3	5	transporte	DAÑO REPORTADO POR Centro de Control. (Mensaje de WhatsApp).	TRANSP	-
4	6	REDES	Los Almendro. Entre las estructura 31877 y 31878, se encuentra la primera	NO PROG	Los Encuentros
5	7	lunch	LUNCH EN LOS ALMENDROS.	ALIMEN	-
6	8	REDES	Montalvo. estructura 35057, se encuentra destruida a causa de un derrumbe. Se	NO PROG	Yantzaza III
7	9	transporte	Nos trasladamos de Montalvo a la agencia Yantzaza.	TRANSP	-
8	10	se_labora	SE LABORA: AA, LP de 08:42 a 13:30 y de 14:30 a 17:10	LABORA	-
9	12	transporte	Para obtener más información del banco de transformadores de la estructura	TRANSP	Yantzaza III
10	13	?	Barrio Central. estructura 23023, (19:25) Lecturas de Corriente:	PROG	Yantzaza III
11	14	transporte	DAÑO REPORTADO POR Centro de Control. (Mensaje de WhatsApp).	TRANSP	-

☒ Exito cambios guardados en Delta Lake!

Ilustración 9. Ejemplo de interfaz de usuario en Marimo Notebooks. Elaboración propia.

Una funcionalidad de Delta Lake es su capacidad de "viaje temporal" (*time travel*), la cual permite consultar y revisar versiones anteriores de la base de datos. Esta característica proporciona un mecanismo robusto para deshacer cambios y auditar modificaciones. Se trata de una funcionalidad avanzada cuyo uso se reserva exclusivamente para el administrador de la base de datos, su disponibilidad asegura un control riguroso y trazable de todas las operaciones de modificación realizadas sobre los datos.

4. Evaluación del sistema

Una vez desarrollado e implementado el Sistema de Gestión de Órdenes de Trabajo en un entorno local, se realizaron diversas pruebas para obtener una evaluación base del rendimiento del sistema en condiciones controladas. Este capítulo documenta los resultados de la experimentación, analizando métricas de rendimiento, calidad de datos y escalabilidad del pipeline implementado.

El desarrollo del sistema se ha realizado en la plataforma GitHub, es lo puede encontrar en la siguiente dirección web: [Repositorio TFM Ernesto Palacios](#)

4.1. Objetivos de la Evaluación

La evaluación del sistema se estructuró en torno a tres objetivos principales:

1. **Capacidad de procesamiento *batch*:** Analizar la capacidad del sistema para procesar grandes volúmenes de órdenes de trabajo históricas, incluyendo todos los registros disponibles desde septiembre de 2018.
2. **Rendimiento del pipeline *end-to-end*:** Medir latencias en cada etapa del flujo de datos (extracción, ingesta, transformación, almacenamiento).
3. **Calidad y consistencia de datos:** Evaluar la efectividad de los mecanismos de validación automática y clasificación mediante NLP.

El procesamiento de datos históricos es fundamental para establecer una línea base analítica que permita el entrenamiento de modelos predictivos y la generación de *insights* sobre patrones operativos de largo plazo.

4.2. Configuración del entorno de pruebas

El microservicio productor hace uso de la computación en paralelo mediante el *framework* DASK para el procesamiento de las órdenes de trabajo que se encuentren en el directorio provisto para este efecto. Este escenario, en el cual se procesan miles de órdenes de trabajo, no es lo normal en el desarrollo diario de actividades, sin embargo, con la finalidad de evaluar el sistema y de cargar la información histórica se ha configurado para su procesamiento. Se considera este escenario como una prueba de estrés para medir las capacidades y limitaciones del procesamiento de órdenes de trabajo en paralelo mediante el microservicio productor.

4.2.1. Dataset de Evaluación

El corpus de evaluación comprende el registro completo de órdenes de trabajo de las cuadrillas operativas en la provincia de Zamora Chinchipe desde el mes de septiembre de 2018 hasta diciembre del 2024:

Tabla 7. Metadatos de documentos para evaluación del sistema. Elaboración propia.

Métrica	Valor
Periodo temporal	Septiembre 2018 - Diciembre 2024 (76 meses)
Total de OT procesadas	23603 documentos PDF
Total de actividades individuales	397450 registros
Tamaño en formato original (PDF)	1.1 GB
Tamaño en formato intermedio (JSON)	167 MB
Tamaño en Delta Lake (con versiones)	1.6 GB
Promedio de actividades por OT	16.8 actividades/documento
Volumen mensual promedio	310 OT/mes

La Tabla 7 muestra las características de los documentos utilizados en la evaluación del sistema.

Análisis del Tamaño en Disco

Un aspecto notable del dataset es que el repositorio Delta Lake (1.6 GB) supera en tamaño tanto a los archivos PDF originales (1.1 GB) como a la representación JSON intermedia (167 MB). Esta aparente paradoja se explica por tres factores arquitectónicos de Delta Lake:

1. **Versionado completo (*Time Travel*):** Delta Lake mantiene un registro transaccional de todas las modificaciones realizadas sobre el dataset. Al momento de la obtención de los datos para la elaboración de la Tabla 7 ya se habían ejecutado múltiples procesos de limpieza y corrección de datos, cada uno generando una nueva versión. Esta funcionalidad de "*time travel*" permite revertir a cualquier estado histórico de los datos, esencial para auditoría y recuperación ante errores.
2. **Fragmentación por ventanas de saltos:** La técnica de procesamiento mediante ventanas de saltos (*tumbling windows*) genera múltiples archivos Parquet pequeños en lugar de un único archivo monolítico. Al momento de la evaluación, el repositorio contenía 2,884 archivos Parquet. Esta fragmentación, si bien incrementa la cantidad de archivos, mejora el paralelismo en lecturas y permite actualizaciones incrementales eficientes.

Optimización del Almacenamiento

Delta Lake proporciona tres operaciones para gestión de almacenamiento:

- **OPTIMIZE**: Compacta archivos pequeños en archivos más grandes, mejorando rendimiento de lectura. Ejecutado periódicamente durante la experimentación.
- **Z-ORDER**: Reorganiza datos para optimizar consultas con filtros múltiples. No aplicado en la evaluación actual.
- **VACUUM**: Elimina archivos obsoletos de versiones antiguas, liberando espacio en disco. **No ejecutado intencionalmente** para preservar trazabilidad completa durante la fase de evaluación.

La documentación de Delta Lake recomienda mantener menos de 10,000 archivos por partición para rendimiento óptimo (The Linux Foundation, 2025). Con 2,884 archivos en una única partición, el sistema opera dentro de los umbrales recomendados. Para escalabilidad futura, se recomienda implementar particionamiento por año y mes cuando el volumen supere 50,000 OT.

4.2.2. Infraestructura de Hardware

La evaluación se realizó en un entorno híbrido local-cloud:

Tabla 8. Características del entorno de evaluación del sistema. Elaboración propia.

Componente	Especificación	Justificación
Equipo de desarrollo	Intel Core i7-11800H (8 núcleos físicos, 16 hilos), 16GB RAM DDR4, SSD NVMe 512GB	Representa hardware típico de estaciones de trabajo en oficinas de EERSSA
MongoDB Atlas	Tier gratuito M0 (512MB RAM, 5GB storage, cluster compartido)	Validación de funcionalidad con recursos mínimos; simula restricciones presupuestarias
Apache Kafka	Docker container (2 cores, 2GB RAM, Kafka 7.4.4)	Despliegue reproducible; facilita pruebas sin instalación nativa
Sistema Operativo	Ubuntu 22.04 LTS	Plataforma de desarrollo estándar Linux
Python	3.11.5 con 'uv' 0.4.18 para gestión de entornos	Versión estable con mejoras de rendimiento en I/O

La Tabla 8 muestra las características del entorno utilizado para la evaluación del sistema. Se ha utilizado el mismo entorno de desarrollo.

Limitaciones del Entorno

Este entorno de pruebas presenta limitaciones que deben considerarse al interpretar resultados:

1. **MongoDB Atlas *tier* gratuito:** Limitado a 512MB de RAM y conexiones concurrentes restringidas. En producción, un *tier* pago (M10+) ofrecería latencias significativamente menores.
2. **Kafka en contenedor local:** Configuración con un único *bróker* sin replicación. No representa configuración de alta disponibilidad requerida en producción.
3. **Ausencia de red distribuida real:** Productores y consumidores ejecutan en la misma máquina, eliminando latencias de red reales que existirían con revisores en ubicaciones geográficas dispersas.

Estas limitaciones implican que las métricas de rendimiento obtenidas representan un escenario optimista (*best-case scenario*). El rendimiento en producción será aproximadamente 20-30% inferior debido a latencias de red, contención de recursos compartidos y variabilidad de conectividad en campo.

4.3. Métricas de Rendimiento del Pipeline

El procesamiento del corpus histórico completo (23,603 OT) se ejecutó en modo *batch* utilizando el microservicio 'producer.py' configurado con paralelización DASK sobre 8 núcleos físicos locales.

Analizamos el dataset generado en MongoDB para encontrar la primera OT del mes de septiembre procesada obteniendo los siguientes valores:

Tabla 9. Metadatos de la primera OT procesada. Elaboración propia

Parámetro	Valor
Fecha de OT	01 de septiembre 2018
Identificador	15941
Creación	2025-08-12 12:42:30.274320

La Tabla 9 nos muestra los metadatos del primer documento procesado, estos datos son generados por el programa productor el momento que se crea el documento JSON que luego es cargado en la base de datos MongoDB

De la misma manera identificamos la última orden de trabajo del año 2024.

Tabla 10. Metadatos de la última OT procesada. Elaboración propia

Parámetro	Valor
Fecha de OT	31 de diciembre 2024
Identificador	144428
Creación	2025-08-12 17:30:04.633390

La Tabla 10 nos muestra los metadatos del último documento procesado y cargado en MongoDB para el año 2024.

En cada archivo procesado se cuenta con un campo donde se registra la fecha en la que fue procesada por el sistema, valiéndonos de esta información podemos calcular el tiempo de procesamiento total para los datos históricos el cual es de 4 horas y 48 minutos. Con estos datos obtenemos las siguientes métricas.

Tabla 11. Métricas de rendimiento del sistema desarrollado. Elaboración Propia.

Parámetro	Valor
Tiempo de procesamiento	17280 segundos (4.8 horas)
Cantidad de OT	23603
Promedio de procesamiento de OT en paralelo	0.732 segundos (16 núcleos)
Promedio de procesamiento De OT individual	3 segundos (1 núcleo)

La Tabla 11 muestra las métricas del sistema en modo *batch* y compara estos valores con el procesamiento individual donde la ventaja de procesamiento paralelo desaparece. Se puede apreciar una marcada diferencia entre el procesamiento en paralelo y el procesamiento individual. Esto se debe a que cada archivo puede ser procesado por un único hilo de procesamiento (*thread*), esto es una ventaja cuando se procesa los archivos en paralelo, sin embargo, con un solo archivo, existe una sobrecarga debido a la serialización de los datos impuesta por el *framework* DASK. En este contexto lo óptimo es procesar uno o dos archivos directamente en el hilo de Python, y a partir de tres o más archivos obtenemos las ventajas que nos ofrece la computación paralela.

Existe un mecanismo para acelerar el procesamiento de un solo documento provisto nativamente por la librería de extracción de texto *PyMuPDF*, mediante la identificación

de datos tabulares donde es posible paralelizar el procesamiento de un solo archivo PDF, en pruebas el promedio de proceso para un solo documento con la técnica de datos tabulares es de 1.5 segundos, una mejora considerable que tiene una desventaja, no es posible combinar el reconocimiento de tablas con el procesamiento en paralelo. Es decir, no se puede aplicar la optimización para un solo documento y a su vez paralelizarlo. Por esta razón se ha optado por mantener las capacidades de paralelismo, ya que una vez se ha procesado todo el catálogo histórico, el procesamiento diario de OT no supone una latencia significativa en el modelo de negocio. La posibilidad combinar los dos paradigmas, reconocimiento de datos tabulares y procesamiento masivo en paralelo, será desarrollado en el apartado de trabajos futuros.



Ilustración 10. Captura del monitor de procesamiento en paralelo DASK. Elaboración propia.

En la Ilustración 10 se muestra una captura del procesamiento en paralelo local en el *framework* DASK, el cual provee de un mecanismo de monitoreo de los trabajos paralelos ejecutados. Podemos observar los 16 núcleos del procesador (color verde), cada uno procesando una OT individual, el tiempo de procesamiento varía de acuerdo con la complejidad y cantidad de actividades en la OT. En la parte superior (color azul) se indica la distribución interna del trabajo realizada de manera automática por la librería DASK. La imagen captura un trabajo de procesamiento de 4316 órdenes de trabajo de las cuales han sido ya procesadas 126 al momento de capturar la imagen.

Tabla 12. Evaluación de métricas de procesamiento en paralelo. Elaboración Propia.

Parámetro	Valor
Total de OT por procesar	4316
Total de OT ya procesadas	126
Cantidad de trabajadores	4
Hilos por cada trabajador	4 (16 hilos en total)
Memoria por trabajador	128 MB (aproximadamente)
Memoria total	1 GB (aproximado)
Tareas por cada trabajador	1000 OT (promedio)

La Tabla 12 muestra un resumen de los datos relevantes de la Ilustración 10. La limitación en cuanto al cómputo distribuido local es la memoria RAM del equipo, una vez que esta memoria está próxima a su llenado DASK automáticamente libera los datos que ya no son útiles a la vez que guarda en la memoria local (NVMe) los datos ya procesados. En este punto la eficiencia de la computación se ve disminuida (Anaconda, Inc., 2025).

4.3.1. Evaluación del Clasificador Naive Bayes

El modelo de clasificación se entrenó con las actividades desarrolladas por los grupos de trabajo en el año 2023. Se recolectó y etiquetó esta información con el fin de obtener un dataset que permita entrenar un modelo predictivo supervisado para la clasificación automática de actividades. De este el 80% del dataset se utilizó para el entrenamiento y se evaluó sobre el 20% restante. En el posterior análisis es importante evaluar el desempeño de la tasa de recuperación (*recall*).

Los valores que se muestran en las tablas siguientes y el desarrollo detallado del modelo predictivo se presentan en el [Apéndice II](#).

Tabla 13. Conversión de cuenta contable a tipo de actividad. Elaboración propia

Cuenta Contable	Tipo de Actividad
511.04.001	Redes
511.04.002	Alumbrado
511.05.001	Acometidas
511.05.002	Medidores

La Tabla 13 muestra la conversión entre la etiqueta de Cuenta Contable con que fue entrenado el dataset y su correspondiente tipo de actividad, de acuerdo con lo indicado en la Ilustración 8. Se realiza este cambio con la finalidad de que sea fácil y comprensible conocer lo que representa cada cuenta contable.

El dataset ha sido dividido en dos secciones ya que el modelo clasificador necesita únicamente el set de entrenamiento y el set de pruebas. Se han organizado de la siguiente manera:

- Tamaño del dataset de entrenamiento: (4758)
- Tamaño del dataset de prueba: (1228)
- Tamaño total del dataset: (6096)

Este es el total de actividades en que cuentan con una etiqueta de Cuenta Contable en el año 2023, debe notarse que no todas las actividades cuentan con esta etiqueta, aquellas actividades informativas como: alimentación y transporte no son consideradas.

Tabla 14. Desempeño por clases del modelo clasificador Naive Bayes. Elaboración propia.

Cuenta Contable	Precisión	Recall	F1-Score	Soporte
Redes	0.87	0.93	0.90	384
Alumbrado	0.97	0.98	0.98	349
Acometidas	0.73	0.88	0.80	285
Medidores	0.92	0.52	0.66	210

La Tabla 14 muestra los resultados obtenidos para cada cuenta contable, se realizará un análisis detallado de cada cuenta.

Cuenta con desempeño excelente:

- **Alumbrado:** F1-Score de 0.98 (precisión 0.97, *recall* 0.98), representando la clase mejor clasificada con 355 muestras del dataset test.
- **Redes:** F1-Score de 0.90 (precisión 0.87, *recall* 0.93), clase más numerosa con 382 muestras, demostrando robustez estadística.

Cuenta con desempeño moderado:

- **Acometidas:** F1-Score de 0.80 (precisión 0.73, *recall* 0.88), con 283 muestras. La precisión relativamente baja (0.73) indica una tasa de falsos positivos del 27%, sugiriendo confusión con otras clases.

Cuenta con desafíos significativos:

- **Medidores:** F1-Score de 0.66 (precisión 0.92, *recall* 0.52), con 208 muestras. El *recall* bajo (0.52) indica que el modelo clasifica correctamente solo el 52% de las cuentas de Medidor reales, fallando en detectar el 48% restante. Esta limitación se atribuye al menor tamaño de muestra (210 vs 384 en Redes) y vocabulario más diverso.

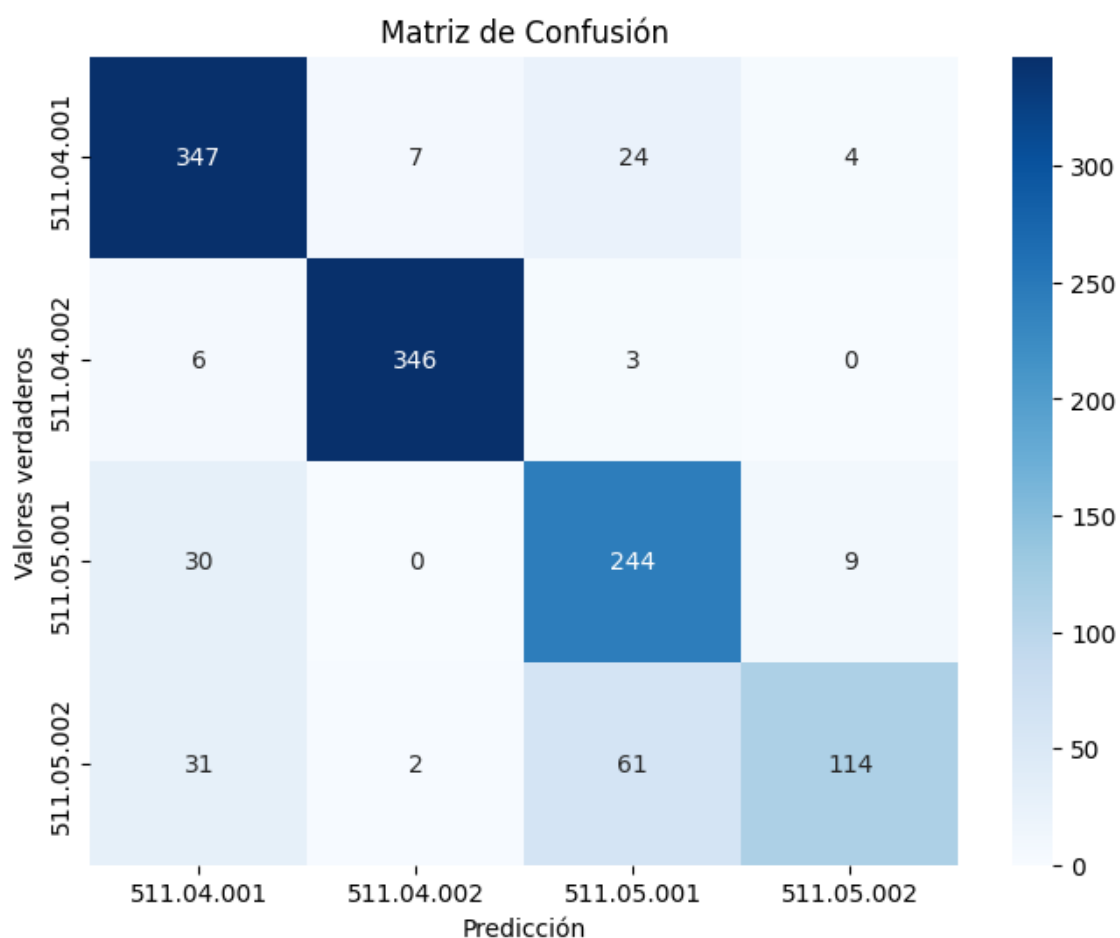


Ilustración 11. Matriz de confusión del modelo clasificador de actividades. Elaboración propia.

La Ilustración 11 permite visualizar el desempeño del modelo clasificador sobre el dataset de prueba (1228 muestras). La diagonal principal muestra las predicciones correctas para cada clase. Las celdas fuera de la diagonal revelan patrones de confusión, siendo notable la confusión entre Medidores y Acometidas (61 casos), y entre Acometidas y Redes (30 casos).

Es comprensible que exista confusión entre las actividades de Acometidas y Medidores, si se recuerda la Ilustración 8 la acometida tiene punto de conexión tanto con las redes eléctricas como con el Medidor del cliente, por lo que las descripciones de las intervenciones realizadas tanto en Redes como en Medidores es común la mención de la acometida, lo cual puede confundir al clasificador provocando errores de detección.

5. Conclusión

Este Trabajo de Fin de Máster ha desarrollado exitosamente un sistema integral de data-pipeline basado en Arquitectura Medallón para la automatización del procesamiento de órdenes de trabajo en la Empresa Eléctrica Regional del Sur S.A. (EERSSA). Los resultados obtenidos validan la hipótesis central: es posible transformar procesos manuales intensivos en conocimiento mediante tecnologías modernas de ingeniería de datos, generando valor estratégico para organizaciones del sector eléctrico.

A nivel de implementación, el sistema se ha concebido como un producto mínimo viable (MVP, por sus siglas en inglés). Esta aproximación permite una validación temprana del valor generado, utilizando tecnologías compatibles con la nube que aseguran su futura implementación e integración directa con la infraestructura de nube privada existente en la EERSSA.

5.1. Cumplimiento de objetivos específicos

Objetivo 1: Data-pipeline escalable y basado en eventos

Se implementó con éxito una arquitectura de microservicios desacoplados haciendo uso de la tecnología provista por Apache Kafka, logrando:

- Ingesta distribuida: El microservicio *producer.py* procesa 23603 órdenes de trabajo históricas (septiembre 2018 - diciembre 2024) en 4.8 horas utilizando paralelización DASK sobre 16 núcleos, equivalente a 0.732 segundos por documento en modo *batch*.
- Procesamiento asíncrono: La combinación de paradigmas streaming (ventanas de saldos de 5 segundos) y *micro-batching* garantiza latencias inferiores a 10 segundos para documentos individuales en modo producción.
- Escalabilidad horizontal: La arquitectura Pub/Sub de Kafka permite agregar productores distribuidos geográficamente sin modificar consumidores, validando el diseño para futuras expansiones departamentales.

Se ha identificado una limitación, durante el desarrollo se hizo uso del *tier* gratuito de MongoDB Atlas (512MB RAM) lo cual introduce latencias de red (50-100ms por operación) que en producción se reducirían implementando MongoDB de manera local. No obstante, estas latencias son aceptables para el volumen actual de 310 OT/mes.

Objetivo 2: Modelo predictivo de Machine Learning

El clasificador Naive Bayes Multinomial entrenado sobre 4758 actividades etiquetadas alcanza:

Exactitud global: 86.3% (1052 de 1228 actividades de prueba correctamente clasificadas)

- F1-Score ponderado: 0.86

Desempeño por clase:

- Excelente ($F1 \geq 0.90$): Alumbrado (0.98), Redes (0.90)
- Aceptable (0.70-0.89): Acometidas (0.80)
- Limitado (< 0.70): Medidores (0.66)

El modelo supera el etiquetado base humano histórico. En base al registro interno (2022-2023) se encuentran tasas de error manual del 15-20% en la asignación de cuentas contables, mientras que el modelo automatizado presenta un error del 14% (1 - 0.86). Más significativo aún, el modelo procesa actividades en <50ms versus 8-12 segundos del proceso manual, representando una aceleración de 160 a 240 veces más rápido. Críticamente la combinación del etiquetado automatizado y la posterior revisión manual mejora la eficiencia significativamente.

La clase "Medidores" exhibe tasa de recuperación (*recall*) de 0.52, implicando que 48% de actividades reales de medidores son mal clasificadas (principalmente como Redes o Acometidas). Este problema se atribuye al desbalance de clases en el dataset original (210 vs 384 muestras), también el vocabulario heterogéneo sin terminología distintiva consistente, ya que los elementos que intervienen en estas actividades son similares, y es mediante la semántica y no solo las palabras empleadas en la descripción que se logra una correcta clasificación.

En el sistema desarrollado se implementa un sistema de umbrales de confianza que marca predicciones de baja confianza para revisión manual.

Objetivo 3: Repositorio centralizado de datos estructurados

La implementación de Delta Lake como capa Plata de la Arquitectura Medallón proporciona:

- Almacenamiento ACID: 397450 actividades individuales desde 2018 en formato tabular optimizado (Parquet + Delta Log).
- Versionado completo: *Time-travel* habilitado para auditoría y *rollback* de correcciones erróneas.
- Generación de informes: Se desarrollaron notebooks interactivos (Marimo) que permiten a personal no técnico:
 - Visualizar y editar actividades individuales mediante interfaces gráficas
 - Exportar reportes mensuales por cuadrilla/alimentador en <2 segundos
 - Validar clasificaciones automáticas con retroalimentación para reentrenamiento

Estos notebooks pueden ser implementados en un servidor centralizado permitiendo a los usuarios interactuar mediante su navegador simplificando el manejo e instalación de software local, inclusive permite el acceso a la información desde fuera de la institución de ser requerido.

5.2. Contribuciones Metodológicas

Arquitectura híbrida local-cloud

A diferencia de implementaciones puramente *cloud* (costosas para PYMES) o completamente *on-premise* (poco escalables), este proyecto demuestra la viabilidad de arquitecturas híbridas:

- MongoDB Atlas (*cloud*): Almacenamiento documental con 99.9% *Service License Agreement* (SLA), sin sobrecarga de mantenimiento. (MongoDB, Inc., 2025)
- Delta Lake local: Procesamiento OLAP de bajo costo en hardware existente.
- Apache Kafka: Implementado en contenedor para un rápido desarrollo, permite fácil y rápida integración con una futura implementación *on-premise*.

Mecanismo de latido '*Heartbeat*' para ventanas de saltos.

El [Apéndice I](#) documenta una solución original al problema de cierre de ventanas en procesamiento streaming: en lugar de enviar mensajes de latido periódicos (spam al bróker), el sistema implementa un *thread* monitor inteligente que envía único pulso cuando se cumplen la condición:

$$(tiempo_actual - ultimo_mensaje) > duracion_ventana$$

Esta optimización reduce mensajes *heartbeat* comparado con cronómetros fijos, disminuyendo carga en Kafka y mejorando eficiencia del sistema.

Validación automática con retroalimentación

El sistema genera reportes PDF ([Apéndice III](#)) clasificados por nivel de criticidad (Fatal - Error - Revisión - Informativo) que permiten a revisores:

- Identificar inconsistencias (fechas erróneas, actividades vacías, carencias reportadas)
- Decidir si corregir el documento original o aceptar como válido
- Reenviar versiones corregidas que actualizan automáticamente MongoDB y Delta Lake

Esta función de retroalimentación cierra el ciclo de calidad de datos, elemento ausente en pipelines tradicionales unidireccionales.

5.3. Limitaciones y Desafíos Encontrados

Durante el desarrollo de este trabajo de titulación se encontraron desafíos, muchos de los cuales pudieron ser superados, otros necesitan una mayor inversión de tiempo y recursos para su resolución. A continuación, un recuento de las dificultades encontradas:

5.3.1. Integración con Sistemas Propietarios

Problema: La EERSSA utiliza un sistema privativo para generación de órdenes de trabajo cuya base de datos es inaccesible para consultas directas por restricciones del departamento de TI. Esto forzó la decisión de procesar documentos PDF finales en lugar de datos estructurados del origen.

Impacto:

- Pérdida de información: Los PDF no contienen metadatos como *usuario_creador*, o *timestamps* de edición.
- Procesamiento más lento: Extraer texto de PDF (*PyMuPDF*) es significativamente más lento que consultas SQL.
- Fragilidad ante cambios: Si el formato del PDF cambia (nueva plantilla), el *parser* requiere actualización.

Lección aprendida: En proyectos de ingeniería de datos para organizaciones establecidas, la política de acceso a datos puede ser el cuello de botella principal, no la tecnología. Invertir tiempo en negociaciones con TI habría ahorrado semanas de desarrollo.

5.3.2. Desbalance de Clases en Dataset de Entrenamiento

El dataset original (Tabla 16, [Apéndice II](#)) presenta desbalance severo:

- Redes: 5520 muestras (54% del total)
- Servicios ocasionales: 8 muestras (0.08%)

Técnicas de balanceo aplicadas:

- *Downsampling* de clase mayoritaria: Redes reducida de 5520 a 1946 muestras
- Exclusión de clases con menos de 200 muestras: 3 categorías eliminadas

Alternativas no implementadas (por limitaciones de tiempo):

- SMOTE (*Synthetic Minority Over-sampling Technique*): Generación sintética de ejemplos para clases minoritarias.

- *Class weighting*: Penalizar errores en clases minoritarias durante entrenamiento.
- Métodos combinatorios: Combinar múltiples clasificadores entrenados sobre submuestras balanceadas.
- Validación de nuevas actividades, aumentar el dataset de entrenamiento aumentando el etiquetado manual y verificado de las actividades desarrolladas durante en el 2024.

Consecuencia: El modelo actual privilegia clases mayoritarias (Redes, Alumbrado) a costa de una tasa de recuperación (*recall*) bajo en Medidores (0.52).

Mitigación: Búsqueda de palabras clave para la clasificación de actividades que no superan el umbral de confianza. Por ejemplo, las palabras “instalación nuevo servicio” permiten identificar las actividades de la cuenta contable “Nuevos Servicios”. Así mismo con las palabras “servicio ocasional”.

5.3.3. Ausencia de Validación en Campo

Problema: El sistema no ha sido desplegado en producción real con revisores operativos. Toda la evaluación se realizó sobre datos históricos (dataset del 2023).

Riesgos no evaluados:

- Aceptación del usuario: ¿Los revisores confiarán en clasificaciones automáticas?
- Variación de conceptos: ¿El modelo mantendrá rendimiento con actividades de 2025 si vocabulario evoluciona?
- Robustez ante adversarios: ¿Trabajadores podrían “engañar” al clasificador para asignar actividades a cuentas incorrectas?

Mitigación parcial: El diseño de umbrales de confianza y revisión manual obligatoria para clases problemáticas reduce riesgos, pero un piloto de 3 meses con 2-3 revisores es indispensable antes de despliegue masivo.

6. Trabajos Futuros

El Sistema de Gestión de Órdenes de Trabajo constituye la base fundamental sobre la cual se desarrollarán e integrarán que amplíen sus funcionalidades. Los trabajos futuros se enfocan, en fortalecer la robustez del sistema y expandir sus capacidades de la gestión y análisis de órdenes de trabajo, con el objetivo de llegar a gestionar el ciclo de vida completo de las órdenes de trabajo.

6.1.1. Mejoras al Modelo de Clasificación

El proyecto de titulación se ha centrado en las actividades una vez han sido registradas por los grupos de trabajo, para incrementar los beneficios que se obtienen de estos datos es necesario mejorar su mecanismo de minería de datos. Dado que el texto constituye el principal recurso disponible, es necesario explorar las capacidades de explotación de este recurso más allá de las etiquetas de cuentas contables para la elaboración de informes. Los modelos de Machine Learning que este sistema alimentará requerirán un mayor nivel de detalle sobre las actividades realizadas. Se proponen plazos para trabajos futuros, que permitan una mayor extracción de valor.

Prioridad Inmediata (0-6 meses)

1. Reentrenamiento con dataset 2024-2025 completo:
 - Objetivo: Incrementar de 6096 a ~31000 actividades etiquetadas
 - Técnica: Implementar *active learning* para etiquetar eficientemente muestras de baja confianza durante la revisión de OT. Retroalimentando el modelo de clasificación.
 - Implementación de SMOTE para balanceo: Eliminar *downsampling* de Redes, reteniendo información valiosa.
 - Resultado esperado: F1-Score global superior a 0.92, *recall* de Medidores superior a 0.75
2. *Random Forest Classifier*: Técnica avanzada de clasificación, es necesario comparar los resultados con Naive Bayes.
3. Detección temprana de cuenta contable mediante técnica de palabras clave: se aplican para las actividades con un bajo nivel de confianza (menor a 0.75)

Prioridad Media (6-12 meses)

Modelos alternativos: extracción de características del texto.

- *Python pattern matching*: extracción de información clave del texto: número de poste, lugar, causa de la falla eléctrica, mediante técnicas de programación tradicionales.
- Embeddings contextuales: Word2Vec entrenado en corpus eléctrico ecuatoriano.

- *BERT multilingüal (distilled)*: técnica moderna aplicando LLM para la identificación de información clave.

Prioridad Largo plazo (12 meses en adelante)

Modernización del sistema de registro de órdenes de trabajo.

1. Dispositivos móviles como prioridad: Android y iOS como clientes principales para el ingreso de órdenes de trabajo. La mayoría de los trabajadores registran las actividades desde sus dispositivos móviles, el sistema actual dificulta esta labor.
2. Funcionamiento sin conexión: el sistema actual se requiere de una conexión a internet para el ingreso de datos, esto puede cambiar en una app nativa.
3. Ingreso por voz: Existe al menos un teclado *open source* con capacidades de voz a texto en español y que no requiere conexión a internet (FUTO, 2025).
4. Validación de datos en tiempo real: algoritmo que verifique que toda la información necesaria se ha ingresado adecuadamente en el dispositivo (*Edge computing*).

Un proyecto integral para el manejo de la información generada por el personal obrero debe situarlos en el centro del proceso de recolección de datos, adaptando el sistema a sus casos de uso y facilitando el ingreso de información. Este tipo de sistema tiene un gran potencial de aplicación no solo en el sector eléctrico, sino en cualquier organización donde las aportaciones del personal en campo sean de gran utilidad, lo cual engloba a la gran mayoría de las organizaciones, si no a todas.

En el caso particular de la EERSSA, se pueden aprovechar herramientas modernas para la validación de información directamente en el dispositivo móvil. Por ejemplo, mediante tecnología gratuita y de código abierto como DuckDB, el cual tiene capacidades compatibles con GeoParquet lo que permite ser embebida en aplicaciones móviles con una base de datos ligera que contenga información de la infraestructura como, por ejemplo, números de poste y sus ubicaciones. De esta manera, el sistema puede verificar rápidamente la información ingresada y alertar al usuario sobre posibles errores en el momento en que se generan, permitiendo su corrección inmediata desde la fuente.

6.1.2. Implementación de Gobernanza de Datos

A medida que se incrementen las capacidades del sistema de gestión y se integren nuevas fuentes de datos, resultará imprescindible implementar un marco robusto de Gobernanza de Datos. Este proceso requiere la definición clara de políticas de acceso, así como la identificación y configuración de roles y usuarios autorizados para interactuar con el sistema de datos.

Los sistemas de almacenamiento tabular moderno como Delta Lake ofrecen capacidades avanzadas de control de acceso que operan en múltiples niveles de granularidad:

- Nivel de tabla: Permisos para lectura, escritura o modificación de tablas completas
- Nivel de columna: Restricción o enmascaramiento de columnas específicas que contienen información sensible
- Nivel de fila: Filtrado dinámico que permite a diferentes usuarios visualizar distintos subconjuntos de datos dentro de la misma tabla

Es importante señalar que algunas de estas capacidades avanzadas, particularmente el control granular a nivel de columna y fila, dependen de herramientas complementarias como *Unity Catalog*, disponible en la plataforma Databricks. Por tanto, es necesario realizar un análisis costo-beneficio que considere las necesidades específicas de seguridad de la organización frente a las implicaciones técnicas y económicas de adoptar soluciones comerciales versus alternativas *open source* o implementaciones mediante vistas con políticas de seguridad.

La implementación de gobernanza de datos cobra especial relevancia en la generación de reportes estratégicos y documentos analíticos. En muchos casos, es necesario mantener un conocimiento restringido de cierta información durante las etapas tempranas de análisis, permitiendo el acceso únicamente a personal autorizado hasta obtener un producto final validado y aprobado para su distribución general. Esto no solo protege información sensible, sino que también previene la difusión de análisis preliminares que podrían generar interpretaciones erróneas o decisiones basadas en datos incompletos.

Adicionalmente, la gobernanza de datos proporciona capacidades de auditoría mediante el registro de transacciones (*transaction log*), permitiendo rastrear quién accedió a qué información y cuándo, estableciendo así una trazabilidad completa que facilita el cumplimiento de regulaciones de protección de datos y estándares de seguridad de la información.

6.1.3. Sistemas Tabulares Alternativos

Delta Lake ha demostrado un desempeño adecuado durante el desarrollo e implementación de la primera versión del Sistema de Gestión de Órdenes de Trabajo. No obstante, a medida que el sistema evoluciona y se incorporan funcionalidades adicionales, resulta estratégico evaluar la posibilidad de migrar hacia sistemas tabulares alternativos que puedan ofrecer ventajas específicas alineadas con la arquitectura tecnológica de la organización.

Una alternativa prometedora es DuckLake, una tecnología emergente introducida en 2025 que presenta capacidades comparables a Delta Lake en términos de

transaccionalidad ACID, versionado de datos y optimización de consultas (DuckDB Foundation, 2025).

La razón fundamental para considerar la migración hacia DuckLake radica en su arquitectura de gestión de metadatos y registros transaccionales. Mientras que Delta Lake almacena su *transaction log* como archivos JSON en un sistema de ficheros, DuckLake registra esta información crítica en bases de datos relacionales tradicionales como PostgreSQL, SQLite o MySQL.

Esta diferencia arquitectónica presenta ventajas significativas para el contexto específico de la EERSSA:

1. Compatibilidad con infraestructura existente: La EERSSA ya cuenta con sistemas de bases de datos relacionales establecidos y robustos, lo que facilita la integración sin necesidad de infraestructura adicional.
2. Gestión empresarial consolidada: Los registros transaccionales pueden ser administrados mediante las herramientas y procedimientos ya establecidos para bases de datos relacionales, incluyendo respaldos automatizados, monitoreo y recuperación ante desastres.
3. Reducción de complejidad operativa: Al eliminar la necesidad de gestionar archivos JSON dispersos en sistemas de ficheros, se simplifica la operación y mantenimiento del sistema.
4. Aproximación a nube privada: Esta arquitectura facilita el despliegue de una solución de nube privada *on-premise*, aprovechando la infraestructura actual de la organización sin depender de servicios de nube pública.
5. Integración con sistemas corporativos: La utilización de bases de datos relacionales permite una integración más directa con otros sistemas empresariales que ya utilizan esta tecnología.

A pesar de las ventajas mencionadas, es importante considerar que DuckLake es una tecnología relativamente nueva (2025) y su adopción requiere evaluar:

- Madurez del ecosistema y disponibilidad de herramientas complementarias.
- Soporte de la comunidad y documentación disponible.
- Compatibilidad con herramientas de análisis y visualización actuales.
- Capacidades de Gobernanza de datos.
- Costo de migración versus beneficios a largo plazo.

Esta posible migración no tiene un sentido de urgencia, principalmente debido a la falta de madurez de la tecnología, dicho esto sus ventajas no pueden ser ignoradas en especial para el caso de uso de la EERSSA.

6.1.4. Implementación de metodología CI/CD

Considerando los cambios propuestos y los trabajos futuros planteados en las secciones anteriores, resulta imprescindible adoptar una metodología de desarrollo

moderna basada en los principios de Integración Continua y Despliegue Continuo (CI/CD). Esta arquitectura de desarrollo representa un cambio paradigmático en la forma de gestionar el ciclo de vida del software, permitiendo actualizaciones frecuentes, seguras y confiables del sistema.

La Integración Continua (CI) es una práctica de desarrollo de software donde los desarrolladores integran código en un repositorio compartido de forma frecuente, idealmente varias veces al día. Cada integración es verificada automáticamente mediante la construcción del proyecto y la ejecución de pruebas automatizadas, permitiendo detectar errores de integración de manera temprana. El Despliegue Continuo (CD) extiende esta práctica al automatizar la liberación de cambios validados hacia entornos de producción o preproducción, reduciendo significativamente el tiempo entre el desarrollo de una funcionalidad y su disponibilidad para los usuarios finales (Humble & Farley, 2011).

El uso de estas metodologías permitirá un alto nivel de confianza que los cambios introducidos al sistema sean validados, garantizando una mayor fiabilidad y menos defectos en el software final. Para esto es posible hacer uso de las capacidades nativas de GitHub.

Esto prepara el ambiente del desarrollo del sistema ampliándolo a la colaboración con el Departamento de Informática para una gradual integración de los diferentes sistemas con los que cuenta la EERSSA.

Para el Sistema de Gestión de Órdenes de Trabajo, la implementación de CI/CD debe incluir los siguientes componentes:

1. Sistema de control de versiones: Repositorio centralizado (actualmente GitHub) que registra todos los cambios del código fuente.
2. Pipeline automatizado de construcción: Proceso que compila el código automáticamente tras cada *commit*, verificando la integridad sintáctica.
3. Suite de pruebas automatizadas:
 - Pruebas unitarias: Verifican el funcionamiento correcto de componentes individuales del código
 - Pruebas de integración: Validan la interacción entre diferentes módulos del sistema
 - Pruebas de regresión: Aseguran que las nuevas funcionalidades no afecten negativamente a las existentes
 - Pruebas de rendimiento: Monitorean el desempeño del sistema bajo diferentes cargas de trabajo
4. Entornos diferenciados:
 - Desarrollo: Espacio experimental para nuevas funcionalidades
 - *Testing/Staging*: Réplica del ambiente productivo para validación final
 - Producción: Sistema operativo accesible a usuarios finales
5. Automatización del despliegue: Scripts y herramientas que gestionan la promoción de código entre entornos de manera controlada y reversible.

La adopción de metodologías CI/CD proporciona beneficios sustanciales:

- Detección temprana de errores: Las pruebas automatizadas identifican problemas inmediatamente después de su introducción, cuando el contexto del cambio aún está fresco en la mente del desarrollador, reduciendo significativamente el tiempo y costo de corrección.
- Mayor confiabilidad del software: Cada cambio pasa por un proceso riguroso de validación antes de alcanzar producción, garantizando que solo código probado y funcional sea desplegado.
- Reducción de defectos en producción: La validación multinivel disminuye drásticamente la probabilidad de que errores críticos lleguen a los usuarios finales.
- Ciclos de desarrollo más rápidos: La automatización elimina tareas manuales repetitivas, acelerando el tiempo de entrega de nuevas funcionalidades.
- Documentación implícita: El pipeline CI/CD documenta automáticamente qué cambios fueron implementados, cuándo, por quién y con qué resultados.
- Facilita el *rollback*: En caso de problemas, la arquitectura permite revertir cambios rápidamente a versiones anteriores estables.

GitHub ofrece capacidades nativas para implementar pipelines CI/CD mediante *GitHub Actions*, una plataforma de automatización que permite definir flujos de trabajo personalizados. Las ventajas de utilizar *GitHub Actions* incluyen:

- Integración nativa: Al estar incorporado en el repositorio de código, elimina la necesidad de servicios externos adicionales.
- Configuración como código: Los pipelines se definen mediante archivos YAML versionados junto con el código, permitiendo rastrear cambios en la infraestructura de desarrollo.
- Ecosistema de acciones preconstruidas: Amplia biblioteca de componentes reutilizables para tareas comunes (testing, despliegue, notificaciones).
- Sin costo adicional: Para repositorios públicos es gratuito, y para privados ofrece minutos generosos en planes estándar.
- Ejecución en múltiples entornos: Permite probar el código en diferentes sistemas operativos y versiones de dependencias simultáneamente.

La implementación de una arquitectura CI/CD establece las bases para una colaboración estructurada con el Departamento de Informática de la EERSSA. Esta integración es altamente deseable y permitirá modernizar todos los procesos de desarrollo de software dentro de la institución.

6.2. Reflexión final

Este proyecto demuestra que la transformación digital de procesos operativos en el sector eléctrico no requiere presupuestos millonarios ni equipos masivos. Con arquitecturas modernas, tecnologías *open source* y metodologías ágiles, una PYME

puede implementar capacidades de ingeniería de datos que rivalizan con soluciones *Enterprise*.

La verdadera innovación no reside en la adopción de tecnología de moda (LLM, IA generativa), sino en la aplicación pragmática de herramientas maduras (Kafka, Delta Lake, Naive Bayes) para resolver problemas reales. El sistema desarrollado no es perfecto, pero es funcional, económico y evolutivo.

El futuro del proyecto depende de tres factores críticos:

1. Compromiso institucional: La EERSSA debe asignar recursos para la implementación global, mantenimiento y evolución del sistema.
2. Cultura basada en datos: Los revisores deben confiar en clasificaciones automáticas y retroalimentar correcciones.
3. Inversión en datos: La calidad del modelo depende linealmente de la calidad de la información ingresada por los trabajadores y del etiquetado manual.

Si estos factores se alinean, el sistema puede evolucionar de automatización de tareas administrativas a optimización estratégica de operaciones, posicionando a EERSSA como referente de innovación en el sector eléctrico ecuatoriano.

La ingeniería de datos no trata sobre tecnología. Trata sobre transformar datos en decisiones.

7. Referencias

Amazon Web Services. (17 de febrero de 2025). *What's the Difference Between ETL and ELT?* Obtenido de aws: <https://aws.amazon.com/compare/the-difference-between-etl-and-elt/>

Anaconda, Inc. (02 de noviembre de 2025). *Dask Documentation*. Obtenido de distributed.dask.org: <https://distributed.dask.org/en/stable/memory.html>

Anthropic. (25 de octubre de 2025). *Claude sonnet 4.5*. Obtenido de claude.ai: <https://claude.ai/chat>

Apache Software Foundation. (20 de octubre de 2025). *Tumbling Window*. Obtenido de Flink Documentation: <https://nightlies.apache.org/flink/flink-docs-release-2.1/docs/dev/datastream-v2/builtin-funcs/windows/>

Asian Institute of Managment. (02 de noviembre de 2025). *Time Series Analysis Handbook*. Obtenido de Vector Autoregressive (VAR) Models: https://phdinds-aim.github.io/time_series_handbook/03_VectorAutoregressiveModels/03_VectorAutoregressiveMethods.html

ByteByteGo. (11 de junio de 2024). *What is a Data Pipeline?* Obtenido de YouTube: <https://www.youtube.com/watch?v=kGT4PcTEPP8&t=51s>

Databricks Inc. (24 de Julio de 2025). *Medallion Architecture*. Obtenido de databricks: <https://www.databricks.com/glossary/medallion-architecture>

DuckDB Foundation. (31 de octubre de 2025). *DuckLake*. Obtenido de ducklake.select: <https://ducklake.select/>

Eager, G. (2021). *Data Engineering with AWS*. Birmingham: Packt Publishing Limited.

EERSSA. (31 de 10 de 2025). *Geoportal Técnico*. Obtenido de SIG: <https://portalserver.eerssa.gob.ec/>

Feick, M., Kleer, N., & Kohn, M. (2018). Fundamentals of Real-Time Data Processing Architectures Lambda and Kappa. *Gesellschaft für Informatik* (pág. 4). Bonn: Lecture Notes in Informatics.

FUTO. (31 de octubre de 2025). *Futo Keyboard*. Obtenido de Futo Keyboard: <https://keyboard.futo.org/>

Humble, J., & Farley, D. (2011). *Continuous Delivery*. Boston, MA: Pearson Education, Inc.

Hyndman, R. J. (02 de noviembre de 2025). *The ARIMAX model muddel*. Obtenido de robjhyndman.com: <https://robjhyndman.com/hyndsight/arimax/>

- Kreps, J. (2 de Julio de 2014). Questioning the Lambda Architecture. *O'Reilly Radar Topics*, págs. 4-5.
- Marimo Inc. (12 de octubre de 2025). *The future of Python notebooks is here*. Obtenido de marimo: <https://marimo.io/>
- MongoDB, Inc. (26 de octubre de 2025). *MongoDB Atlas*. Obtenido de MongoDB: <https://www.mongodb.com/>
- Narkhede, N., Shapira, G., & Palino, T. (2017). *Kafka - The Definitive Guide*. Sebastopol: O'Reilly Media.
- NATS Authors. (22 de octubre de 2025). *Welcome to NATS .NET*. Obtenido de JetStream: <https://nats-io.github.io/nats.net/documentation/jetstream/intro.html>
- Quix Analytics. (20 de octubre de 2025). *Windowed Aggregations*. Obtenido de Quix Docs: <https://quix.io/docs/quix-streams/windowing.html?h=#updating-window-definitions>
- Reis, J., & Housley, M. (2022). *Fundamentals of Data Engineering - Plan and Build Robust Data Systems*. Sebastopol, CA 95472.: O'Reilly Media.
- Singh, J. (2023). *Natural Language Processing in the Real World*. Boca Raton: CRC Press.
- Strengtholt, P. (2025). *Building Medallion Architectures*. Sebastopol: O'Reilly Media.
- The Linux Foundation. (26 de octubre de 2025). *Concurrency Control*. Obtenido de Delta Lake Docs: <https://docs.delta.io/latest/concurrency-control.html#concurrency-control>
- Wei, W. (2006). *Time series analysis: univariate and multivariate methods*. Boston MA: Pearson Education, Inc.

Apéndice I: Implementación del mecanismo *Heartbeat*

En el sistema de órdenes de trabajo, luego de cargar los datos a la base de datos MongoDB enviamos los identificadores únicos de la orden de trabajo (OT) hacia Apache Kafka para su posterior procesamiento. Para procesar estas nuevas OT se ha implementado un paradigma de procesamiento de **ventanas de saltos** (*tumbling window*). En este mecanismo existe el riesgo de que los últimos eventos de una ventana no sean procesados si no llegan eventos posteriores que “empujen” el cierre de una ventana.

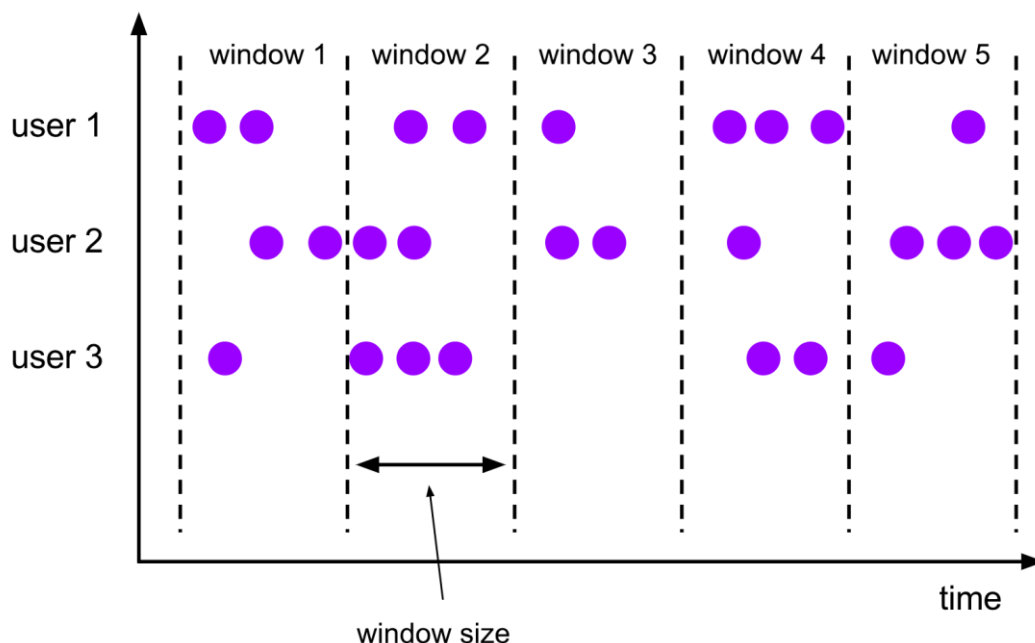


Ilustración 12. Concepto de ventanas de saltos (*tumbling windows*). Fuente (Apache Software Foundation, 2025)

En la Ilustración 12 se muestra un gráfico práctico de ventanas de salto, se aprecia como los eventos (color púrpura) se dividen en secciones de tiempo (líneas verticales) las cuales que no se montan (*no overlap*), cuando una finaliza empieza la siguiente. La clave está en la variable **tiempo**. La pregunta cable es ¿Quién rastrea el tiempo transcurrido? Es muy tentador suponer que el tiempo es rastreado por el proceso que recibe los mensajes, pero esto no es así.

En la librería **Quix Streams** implementada, las ventanas siempre usan el tiempo del evento. El tiempo del evento es obtenido desde la estampa de tiempo de los mensajes que llegan desde Kafka. Cada ventana de agregación rastrea y avanza el tiempo usando como referencia estos mensajes. (Quix Analytics, 2025)

Esto tiene mucho sentido si consideramos que puede llegar existir una latencia considerable entre el bróker de Kafka y nuestro consumidor. Lo que se desea es garantizar que cada ventana procese los mensajes que llegaron a Kafka en el periodo

de tiempo configurado. Por otro lado, esta limitación supone un desafío, para que la ventana se cierre debe llegar el primer mensaje de la siguiente ventana, esto avanza el reloj interno y detona el procesamiento, pero ¿qué pasa con ese primer mensaje de una nueva ventana? Debe esperar la llegada de nuevos mensajes que avancen el reloj, y si los mensajes no llegan, quedará en un estado perpetuo de espera, hasta que un nuevo mensaje rompa la marca del tamaño de la ventana.

Este no es el comportamiento deseado, si ha transcurrido el tiempo de la ventana, deseamos que los eventos sean procesados, incluso si existe un solo mensaje en la ventana. La solución implementada es la de enviar un mensaje de pulso (*heartbeat*) este mensaje tiene el propósito de forzar el cierre de la ventana. El contenido del mensaje no es importante, solo es necesario reconocerlo para filtrarlo luego. La solución fácil es crear un proceso que envíe este mensaje como cronómetro, en cada cierre de la ventana, en el caso del sistema de OT, cada cinco segundos. Esta no es una solución óptima, y tendrá el efecto de “*spam*” en nuestro bróker Kafka. En su lugar se ha desarrollado una solución más elegante.

En el caso de que se mantenga una corriente continua de mensajes no es necesario enviar un mensaje de pulso, si esta corriente se detiene por un tiempo mayor al de nuestra ventana, conviene enviar un solo mensaje a nuestro bróker.

Esta solución se ha implementado en la versión de **Python 3.11** utilizada el desarrollo del sistema, esto es antes de la aparición de Python 3.14 en la que se elimina el bloqueo global de Interpretre (GIL).

Se implementa un segundo hilo de procesamiento (*thread*) el cual mantiene comunicación con el hilo principal, este hilo mantiene el rastreo de la última vez que se envió un mensaje al bróker, si este tiempo supera el tiempo de la ventana, se envía un solo mensaje de *heartbeat* al bróker, y vuelve a esperar.

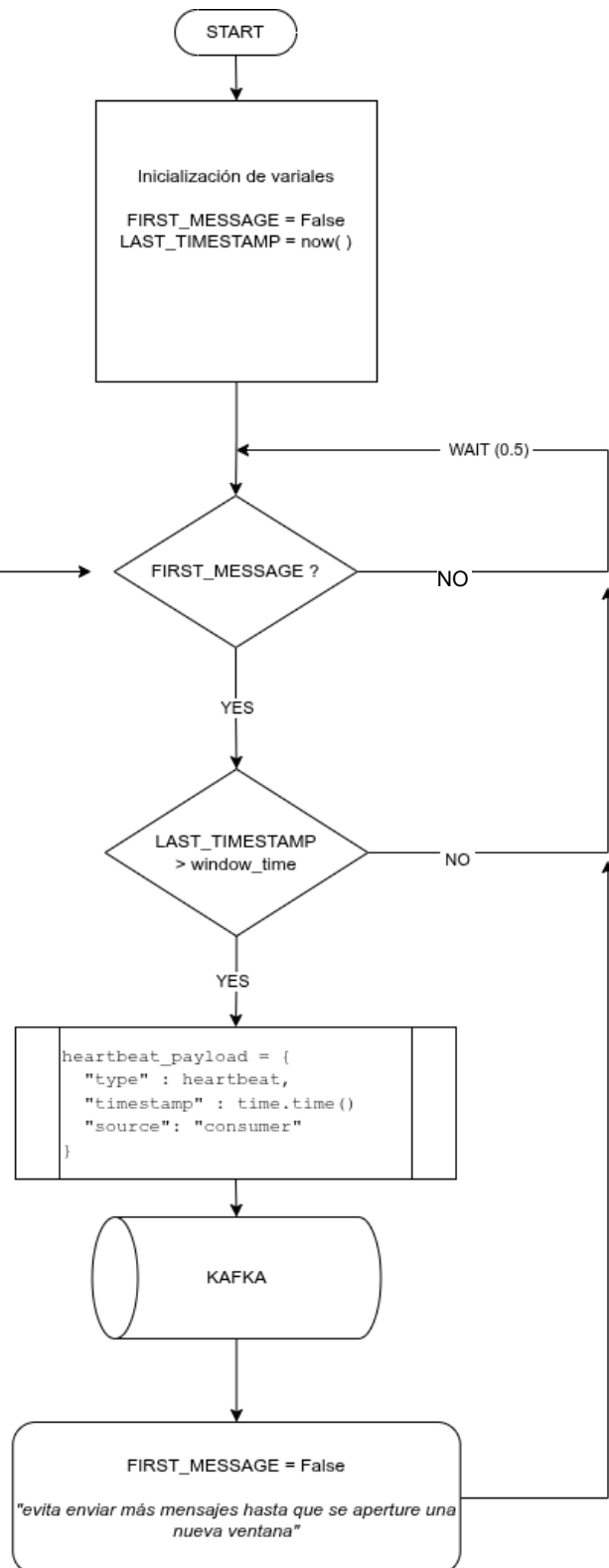
El algoritmo es sencillo de implementar, se utilizan dos variables compartidas entre ambos hilos para conocer el estado de la ventana en el bróker Kafka y enviar el mensaje *heartbeat* solamente cuando sea necesario.

En la Ilustración 13 (abajo) se muestra el diagrama de flujo con la implementación del algoritmo propuesto. Este hilo de procesamiento forma parte del programa “consumidor.py” y efectivamente es ejecutado en paralelo. Con este hilo de procesamiento se envía un mensaje de pulso únicamente cuando es necesario para cerrar la ventana de procesamiento.

Consumer.py

En el hilo principal, cada vez que se envía un mensaje al broker actualiza las variables globales:

```
FIRST_MESSAGE = True
LAST_TIMESTAMP = now()
```



Elaborado: Ernesto Palacios © 2025

Ilustración 13. Diagrama de flujo de la implementación del algoritmo heartbeat.
Elaboración propia.

Apéndice II: Desarrollo del Clasificador Naive Bayes

Problema

Se cuenta con textos cortos que necesitan ser clasificados, el tamaño del texto es de alrededor de 300 caracteres, los cuales describen las actividades realizadas diariamente por los grupos de trabajo de la Empresa Eléctrica Regional del Sur (EERSSA).

Fundamento teórico

La clasificación es una técnica de aprendizaje supervisado en la cual los modelos requieren datos de entrenamiento previamente etiquetados, compuestos por muestras de datos junto con sus correspondientes etiquetas o clases. El modelo aprende a partir de las relaciones conocidas entre entradas y salidas, permitiendo posteriormente clasificar nuevas muestras de datos no vistas previamente durante el entrenamiento (Singh, 2023).

Naive Bayes es un modelo probabilístico fundamentado en el teorema de Bayes que destaca por su escalabilidad, ya que el número de parámetros requeridos es proporcional al número de características (*features*) del dataset. En tareas de clasificación, este modelo calcula la probabilidad de que cada documento de texto pertenezca a cada una de las clases posibles, asignando finalmente la clase con mayor probabilidad. Este tipo de modelos se conocen como modelos generativos (Singh, 2023).

Existen tres variantes principales del clasificador Naive Bayes:

- **Gaussiano:** Asume distribución normal de las características
- **Multinomial:** Diseñado para datos de conteo (frecuencias)
- **De Bernoulli:** Para características binarias

Para clasificación de texto, el *Naive Bayes Multinomial* es la variante más utilizada (Singh, 2023) y constituye una elección popular para establecer modelos de referencia (*baseline*) debido a su simplicidad, eficiencia computacional y rendimiento robusto.

Fuente de datos

Con el objetivo de etiquetar las actividades se ha digitalizado el proceso manual de clasificación llevada a cabo en el año 2023 hasta el mes de noviembre, se cuenta entonces con un archivo en formato Excel denominado “2023 CALIFICACION ACTIVIDADES.xlsx” con todas las actividades etiquetadas correctamente.

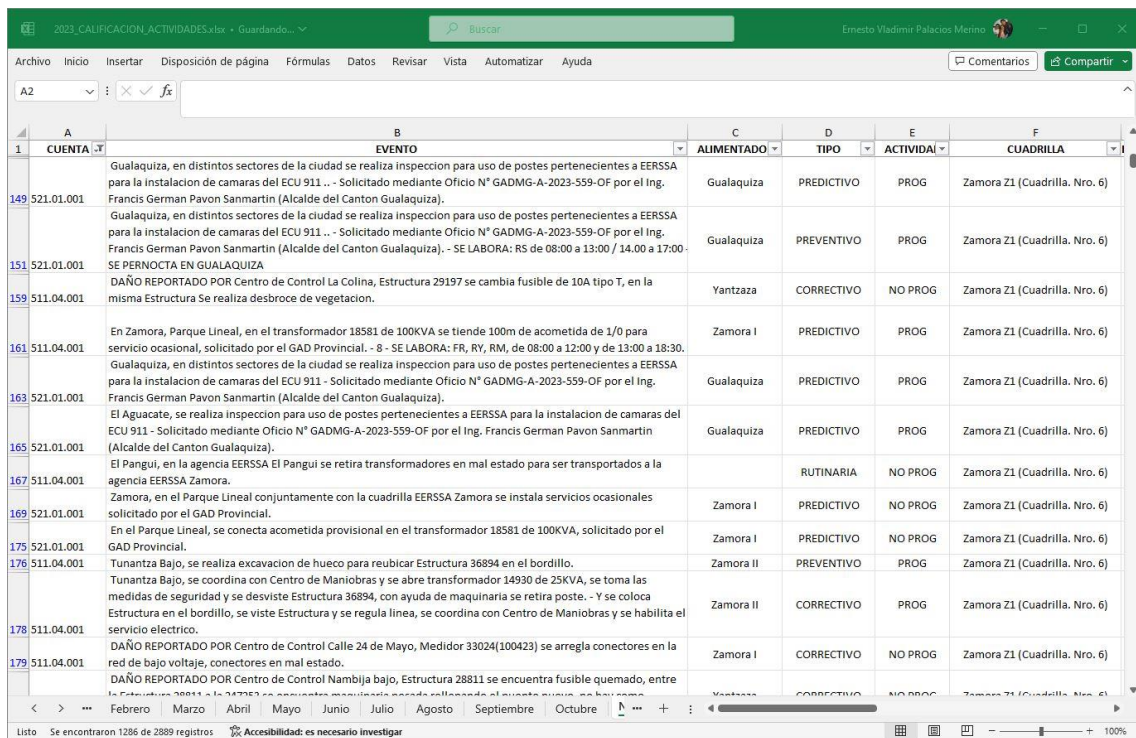
La aplicación de esta etiqueta se la encuentra en la columna: “**CUENTA**” haciendo referencia a la *Cuenta Contable* utilizada por la empresa eléctrica para el seguimiento de las actividades, tenemos las siguientes cuentas y su descripción.

Tabla 15. Cuentas contables para clasificación de actividades. Elaboración propia.

Cuenta Contable	Descripción
511.04.001	Actividades en las redes de distribución (postes).
511.04.002	Actividades del alumbrado público (lámparas).
511.05.001	Actividades de acometidas hacia el Medidor.
511.05.002	Actividades de medidores eléctricos (cliente final).
511.03.003	Actividades de líneas de transmisión (Alta Tensión)
521.01.001	Actividades de comercialización del servicio eléctrico.
OT-01-2022-GECOM	Instalación de un nuevo medidor para el cliente final
OT-07-2022-GECOM	Restitución, cambio de un medidor para el cliente final.
Informativo	Es el texto que no describe ninguna actividad, sino que sirve para transmitir información hacia los superiores.

La Tabla 15 muestra todas las posibles categorías de Cuentas Contables en las que se puede clasificar las actividades de los grupos de trabajo, sin embargo, como se demuestra más adelante las cuatro primeras categorías componen la mayoría de los trabajos realizados diariamente y son las más fáciles de identificar. El objetivo del clasificador será de identificar y etiquetar estas cuatro cuentas, aquellas que se tenga un bajo nivel de confianza, inferior al 60% se aplicará la etiqueta desconocido “?” con la finalidad de que el revisor pueda filtrar estas actividades y clasificarlas según corresponda.

La ilustración 14 (abajo) muestra una captura del archivo que se utiliza en el entrenamiento del clasificador. Este archivo fue elaborado a lo largo del año 2023 ha sido revisado para validar que las etiquetas de las cuentas contables sean correctas. En la imagen se muestra el filtrado de las actividades con una Cuenta contable, filtrando las de tipo informativas (alimentación y transporte).



A	B	C	D	E	F
CUENTA	EVENTO	ALIMENTADO	TIPO	ACTIVIDAD	CUADRILLA
149 521.01.001	Gualaquiza, en distintos sectores de la ciudad se realiza inspeccion para uso de postes pertenecientes a EERSSA para la instalacion de camaras del ECU 911... - Solicitado mediante Oficio N° GADMG-A-2023-559-OF por el Ing. Francis German Pavon Sanmartin (Alcalde del Canton Gualaquiza).	Gualaquiza	PREDICTIVO	PROG	Zamora Z1 (Cuadrilla. Nro. 6)
151 521.01.001	Gualaquiza, en distintos sectores de la ciudad se realiza inspeccion para uso de postes pertenecientes a EERSSA para la instalacion de camaras del ECU 911... - Solicitado mediante Oficio N° GADMG-A-2023-559-OF por el Ing. Francis German Pavon Sanmartin (Alcalde del Canton Gualaquiza).	Gualaquiza	PREVENTIVO	PROG	Zamora Z1 (Cuadrilla. Nro. 6)
159 511.04.001	SE PERNOCITA EN GUALAQUIZA DAÑO REPORTADO POR Centro de Control La Colina, Estructura 29197 se cambia fusible de 10A tipo T, en la misma Estructura Se realiza desbroce de vegetacion.	Yantzaza	CORRECTIVO	NO PROG	Zamora Z1 (Cuadrilla. Nro. 6)
161 511.04.001	En Zamora, Parque Lineal, en el transformador 18581 de 100KVA se tiende 100m de acometida de 1/0 para servicio ocasional, solicitado por el GAD Provincial. - 8 - SE LABORA: FR, RY, RM, de 08:00 a 12:00 y de 13:00 a 18:30.	Zamora I	PREDICTIVO	PROG	Zamora Z1 (Cuadrilla. Nro. 6)
163 521.01.001	Gualaquiza, en distintos sectores de la ciudad se realiza inspeccion para uso de postes pertenecientes a EERSSA para la instalacion de camaras del ECU 911 - Solicitado mediante Oficio N° GADMG-A-2023-559-OF por el Ing. Francis German Pavon Sanmartin (Alcalde del Canton Gualaquiza).	Gualaquiza	PREDICTIVO	PROG	Zamora Z1 (Cuadrilla. Nro. 6)
165 521.01.001	El Aguacate, se realiza inspeccion para uso de postes pertenecientes a EERSSA para la instalacion de camaras del ECU 911 - Solicitado mediante Oficio N° GADMG-A-2023-559-OF por el Ing. Francis German Pavon Sanmartin (Alcalde del Canton Gualaquiza).	Gualaquiza	PREDICTIVO	PROG	Zamora Z1 (Cuadrilla. Nro. 6)
167 511.04.001	El Pangui, en la agencia EERSSA El Pangui se retira transformadores en mal estado para ser transportados a la agencia EERSSA Zamora.		RUTINARIA	NO PROG	Zamora Z1 (Cuadrilla. Nro. 6)
169 521.01.001	Zamora, en el Parque Lineal conjuntamente con la cuadrilla EERSSA Zamora se instala servicios ocasionales solicitado por el GAD Provincial.	Zamora I	PREDICTIVO	NO PROG	Zamora Z1 (Cuadrilla. Nro. 6)
175 521.01.001	En el Parque Lineal, se conecta acometida provisional en el transformador 18581 de 100KVA, solicitado por el GAD Provincial.	Zamora I	PREDICTIVO	NO PROG	Zamora Z1 (Cuadrilla. Nro. 6)
176 511.04.001	Tunantza Bajo, se realiza excavacion de hueco para reubicar Estructura 36894 en el bordillo.	Zamora II	PREVENTIVO	PROG	Zamora Z1 (Cuadrilla. Nro. 6)
178 511.04.001	Tunantza Bajo, se coordina con Centro de Maniobras y se abre transformador 14930 de 25KVA, se toma las medidas de seguridad y se desviste Estructura 36894, con ayuda de maquinaria se retira poste. - Y se coloca Estructura en el bordillo, se viste Estructura y se regula linea, se coordina con Centro de Maniobras y se habilita el servicio electrico.	Zamora II	CORRECTIVO	PROG	Zamora Z1 (Cuadrilla. Nro. 6)
179 511.04.001	DAÑO REPORTADO POR Centro de Control Calle 24 de Mayo, Medidor 33024(100423) se arregla conectores en la red de bajo voltaje, conectores en mal estado.	Zamora I	CORRECTIVO	NO PROG	Zamora Z1 (Cuadrilla. Nro. 6)
	DAÑO REPORTADO POR Centro de Control Nambija bajo, Estructura 28811 se encuentra fusible quemado, entre la Estructura 28811 y la 347363 se encuentra maquinaria usada colgando al poste, se repara.	Yantzaza	CORRECTIVO	NO PROG	Zamora Z1 (Cuadrilla. Nro. 6)

Ilustración 14. Captura del archivo con los datos de entrenamiento. Elaboración propia

Procesamiento y transformación de los datos

Se aplican las siguientes transformaciones al texto con la finalidad normalizar y generalizar la solución desarrollada.

1. Cargar el archivo de Excel al entorno de Python en formato de Pandas *Dataframe*.
2. Se elimina aquellas filas con la etiqueta de alimentación y transporte.
3. Se utiliza solo con las columnas Cuenta y Actividad.
4. Retiro de acentos (diacríticos) de las palabras.
5. Se convierte todos los caracteres a minúsculas.
6. Se convierte abreviaciones comúnmente usadas por los grupos de Trabajo y aquellas palabras comúnmente mal escritas, por ejemplo:
 - a. est ► estructura
 - b. trafo ► transformador
 - c. breiker ► breaker
 - d. med ► medidor
7. Se remueve las palabras comunes en el idioma español “*stop words*” (*el, y, los, sr, etc.*)
8. Se crea un array cada elemento es una palabra (*Tokenización*)
9. Se balancea el dataset para que cada categoría tenga un valor similar de elementos clasificados.
10. Se divide la base de datos para entrenamiento y validación: 80/20

11. Vectorizar el dataset para el entrenamiento. ($n_gram_range = 2$)
12. Entrenamiento de clasificador: “*Multinomial Naive Bayes: MultinomialNB()*”
13. Evaluación de resultados.

Resultados

El dataset inicial cuenta con 30433 entradas totales, desde enero a noviembre del 2023. Luego de filtrar aquellos eventos que se han etiquetado, paso 3, obtenemos la siguiente cantidad de eventos.

Tabla 16. Cantidad de eventos para cada Cuenta Contable. Elaboración propia.

Cuenta	Cantidad de eventos
511.04.001	5520
511.04.002	1718
511.05.001	1411
521.05.001	1021
511.03.003	24
OT-01-2022-GECOM	159
OT-07-2022-GECOM	8

La Tabla 16 muestra que hay cuatro cuentas que tienen un muy bajo conteo, esto se debe a que se tratan de tareas específicas que normalmente no son realizadas por el grupo de trabajo desde el que se obtuvieron los datos. Para continuar hasta el entrenamiento se consideran las tareas principales que si se tiene un número considerable de muestras. Además, se ha balanceado la cuenta “511.04.001” para alinear sus muestras con las otras cuentas contables. El resultado es el siguiente:

Tabla 17. Cantidad de eventos luego de balancear el Dataset. Elaboración propia

Cuenta	Cantidad de eventos
511.04.001	1946
511.04.002	1718
511.05.001	1411
521.05.001	1021

La Tabla 17 indica la cantidad de eventos (actividades) para cada cuenta contable una vez se ha balanceado los datos (*downsampling*) y eliminado las cuentas contables de menos de 200 eventos.

Una vez se ha balanceado el *Dataframe*, aplicamos las transformaciones de texto:

Python

```
df_balanced['transformacion_1'] = (df_balanced['EVENTO']
    .str.normalize('NFKD')
    .str.encode('ascii', errors='ignore')
    .str.decode('utf-8')
    .str.lower()
)
```

Ilustración 15. código python para la transformación del texto de la actividad. Elaboración propia.

La Ilustración 15 muestra la cadena de transformaciones aplicadas a la columna 'EVENTO' la cual contiene el texto que describe la actividad realizada. A partir de aquí se continúa con las transformaciones del texto. A medida que se aplican las transformaciones podemos revisar la manera en la que se modifica el texto:

operación de reemplazo con regex

```
df_balanced['correcciones'] = df_balanced['transformacion_1'].replace( reemplazar, regex=True)
df_balanced['correcciones'] = df_balanced['correcciones'].str.replace('[^\w\s]','', regex=True) # Remover puntuacion . ,
df_balanced.sample(10)
```

	index	CUENTA	EVENTO	transformacion_1	correcciones
	1610	10202 511.04.002	En San Manuel Estructura 219601 se arregla l...	en san manuel estructura 219601 se arregla l...	en san manuel estructura 219601 se arregla lu...
	2277	13983 511.04.002	En el mismo sector en la est 064735 se reali...	en el mismo sector en la est 064735 se reali...	en el mismo sector en la estructura 064735 se...
	3913	22473 511.04.001	En Yantzaza en Reconector Estadio de Yantza...	en yantzaza en reconector estadio de yantza...	en yantzaza en reconector estadio de yantza...
	4408	24066 511.05.002	Nos trasladamos de San Carlos a el Guayacan i...	nos trasladamos de san carlos a el guayacan i...	nos trasladamos de san carlos a el guayacan i...
	973	6325 511.04.001	La Fragancia, en los Reguladores Estructura 2...	la fragancia, en los reguladores estructura 2...	la fragancia en los reguladores estructura 27...
	44	296 511.04.001	DAÑO REPORTADO POR Centro de Control Derivaci...	dano reportado por centro de control derivaci...	dano reportado por centro de control derivaci...
	5576	28042 511.04.002	Chuchumbleza, Se Atiende Reclamo 1100500118: ...	chuchumbleza, se atiende reclamo 1100500118: ...	chuchumbleza se atiende reclamo 1100500118 en ...
	5174	26945 511.05.001	Se traslada de Santa Rita a la Recta de Pach...	se traslada de santa rita a la recta de pachi...	se traslada de santa rita a la recta de pachi...
	1861	12501 511.04.002	Gualaquiza centro de la Ciudad en la Estructu...	gualaquiza centro de la ciudad en la estructu...	gualaquiza centro de la ciudad en la estructu...
	5339	27239 511.05.001	En la Sixto Duran se realiza suspension de se...	en la sixto duran se realiza suspension de se...	en la sixto duran se realiza suspension de se...

Ilustración 16. Transformación del texto con corrección de palabras. Elaboración propia.

La ilustración 16 muestra el resultado de aplicar las transformaciones de texto a la columna de texto. Se ha destacado en un recuadro rojo una muestra en la que se puede notar la transformación del texto.

Continuamos aplicando las transformaciones hasta que se obtiene los *tokens* con las palabras comunes del idioma español.

	CUENTA	reduccion
779	511.04.001	[panguitza, alto, 29177, derivacion, san, greg...
321	511.04.001	[chimbutza, 62399, encuentra, fusible, quemado...
4843	511.04.001	[san, ramon, 32738, lmt3, poda, arbol, frutal,...
4403	511.05.002	[traslada, jorge, mosquera, pachicutza, 31502,...
4684	511.05.001	[zamora, coordinacion, companeros, ep, pa, bar...

Ilustración 17. Muestra de dataset con todas las transformaciones del texto aplicadas. Elaboración propia.

La ilustración 17 permite visualizar una muestra al azar de cinco actividades a las cuales se les ha aplicado las transformaciones al texto, se observan la columna de etiquetas 'CUENTA' y la columna con el texto procesado 'reduccion'. En esta columna se puede notar que las únicas palabras y ninguna es un artículo o palabra común (el, los, y, en, etc.) Con este dataset es posible entrenar el clasificador para lo cual dividimos nuestro dataset entre entrenamiento / testeo.

- ✓ Entrenamiento mediante NAIVE BAYES

```
[ ] from sklearn.feature_extraction.text import TfidfVectorizer

# Join the tokens back into strings for vectorization
train['text'] = train['reduccion'].apply(lambda x: ' '.join(x))
test['text'] = test['reduccion'].apply(lambda x: ' '.join(x))

vectorizer = TfidfVectorizer()

[ ] dtm = vectorizer.fit(train['text'])

[ ]  from sklearn.naive_bayes import MultinomialNB

# Transform the training and testing data using the fitted vectorizer
train_vectors = vectorizer.transform(train['text'])
test_vectors = vectorizer.transform(test['text'])

# Train a Multinomial Naive Bayes classifier
classifier = MultinomialNB()
classifier.fit(train_vectors, train['CUENTA'])
```

▼ MultinomialNB ⓘ ?

MultinomialNB()

Ilustración 18. Captura del proceso de entrenamiento del clasificador Naive Bayes. Elaboración propia.

La Ilustración 18 muestra una captura del proceso de entrenamiento del modelo clasificador. Una vez que hemos entrenado nuestro clasificador procedemos a evaluar los resultados:

```
from sklearn.metrics import classification_report, accuracy_score

# Make predictions on the test data
predictions = classifier.predict(test_vectors)

# Evaluate the classifier
print("Accuracy:", accuracy_score(test['CUENTA'], predictions))
print("\nClassification Report:\n", classification_report(test['CUENTA'], predictions))
```

Accuracy: 0.8631921824104235

	precision	recall	f1-score	support
511.04.001	0.87	0.93	0.90	384
511.04.002	0.97	0.98	0.98	349
511.05.001	0.73	0.88	0.80	285
511.05.002	0.92	0.52	0.66	210
accuracy			0.86	1228
macro avg	0.87	0.83	0.83	1228
weighted avg	0.87	0.86	0.86	1228

Ilustración 19. Captura de la evaluación del modelo clasificador. Elaboración propia.

La Ilustración 19 nos permite analizar la evaluación del modelo entrenado. Globalmente se obtiene una exactitud del 86% destacándose las actividades relacionadas al alumbrado público (511.04.002) con el 97% y aquellas con las que se tiene mayor dificultad son las relacionadas a las Acometidas (511.05.001).

Una vez que se ha recolectado y aplicado una mayor cantidad de muestras, se planea recolectar todas las actividades desarrolladas en el año 2025, revisar y limpiar las etiquetas de las 31,354 actividades registradas hasta el momento para volver a entrenar al clasificador, esperando obtener una mayor precisión.

Una vez entrenado nuestro clasificador lo utilizamos en el proceso de generación de la *Matriz de actividades* al transformar los datos previo al ingreso en Delta Lake.

Apéndice III: Reporte de Orden de Trabajo

Reporte de Orden de Trabajo

estado TERMINADO

Gualaquiza (Agencia)
Sin Fecha
LITUMA CORDOVA CESAR RUBEN
id_ot : 24758

§1. Novedades encontradas

Informativo 1.1 (Informativo - 2025-08-12 17:21:46).
CREACION DE LA OT, se encuentra un archivo PDF valido
Ubicacion:
/home/vlad/Documents/temp_borrar/aTest/Orden de trabajo Gualaquiza 01-03-2019 (CL).pdf

Error 1.1 (ERROR - 2025-08-12 17:21:47).
No coinciden las fechas en la HOJA UNO
Fecha de la OT: 2019-03-01T00:00:00-05:00
Fecha Mitad: unconverted data remains: /
análisis/se
Fecha String: viernes, 01 de marzo del 2019
ANÁLISIS DE SE

Revisar 1.1 (Revisar - 2025-08-12 17:22:07).
Se reportan CARENCIAS
Revisar si estan reportadas CARENCIAS

Error 1.2 (ERROR - 2025-08-12 17:22:11).
No pudo corregir las fechas erroneas
Ocurrio un error al corregir fechas mal digitadas. Desde >> corregir_fechaInicio/Fin

Ilustración 20. Ejemplo de "Reporte de Orden de Trabajo". Elaboración propia

La Ilustración 20 muestra un ejemplo del Reporte generado para el revisor, se indican las novedades encontradas en la Orden de Trabajo los colores indican su gravedad.

Anexo I: El presente y futuro de PySpark

Se desea confirmar la validez del uso de una arquitectura de nube privada para el presente trabajo de titulación, se desea realizar una comparación con las herramientas tradicionales de tipo corporativas (*Enterprise*). Se ha delegado esta tarea a una herramienta de IA generativa. Se comparo dos herramientas: Google Gemini 2.5 Pro y Claude Sonnet 4.5, se eligió Claude ya que se apegó a la limitación de la extensión del texto y se mantuvo enfocado en la comparación económica.

Este anexo ha sido generado mediante el uso de la herramienta de IA Generativa: Claude Sonnet (Anthropic, 2025) al cual se le ha provisto de la siguiente instrucción:

Escribe un ensayo corto, de mil palabras, comparando las capacidades, costos y beneficios del uso de PySpark en organizaciones pequeñas y medianas comparándolas con herramientas modernas como DuckDB, Polars, Daft. Analiza su costo en el despliegue en la nube y cuando es recomendable el uso de PySpark.

La Encrucijada del Procesamiento de Datos: PySpark vs. Herramientas Modernas para PYMES

Introducción

En la última década, Apache Spark se consolidó como el estándar de facto para el procesamiento distribuido de datos a gran escala. Sin embargo, el panorama tecnológico ha evolucionado drásticamente. Herramientas como DuckDB, Polars y Daft han emergido desafiando la hegemonía de Spark, especialmente en el contexto de pequeñas y medianas empresas (PYMES). Esta revolución no es meramente tecnológica; representa un cambio fundamental en cómo las organizaciones deben evaluar sus necesidades reales de infraestructura de datos versus las soluciones "enterprise-grade" que dominaron la década pasada. Este ensayo analiza críticamente las capacidades, costos y casos de uso óptimos de PySpark frente a sus competidores modernos, proporcionando un marco de decisión para organizaciones con recursos limitados.

El Peso del Legado: PySpark y su Ecosistema

PySpark, la interfaz Python de Apache Spark, fue diseñado en 2014 para resolver problemas de empresas como *Yahoo* y *Netflix* que procesaban petabytes de datos diariamente. Su arquitectura distribuida permite ejecutar computación en clústeres de cientos de nodos, particionando datos automáticamente y tolerando fallos de hardware. Esta capacidad impresionante tiene un costo oculto significativo: complejidad operativa.

Una instalación típica de PySpark en la nube (AWS EMR, Databricks o Google Dataproc) requiere configurar múltiples componentes: un clúster Spark con nodos maestros y trabajadores, sistemas de almacenamiento distribuido (HDFS o S3), gestores de recursos (YARN o Kubernetes) y herramientas de monitoreo. Para una PYME procesando 100GB-1TB de datos mensuales, esta infraestructura es similar a alquilar un camión de carga para transportar comestibles. Funciona, pero el costo-beneficio es cuestionable.

El costo monetario es tangible. Un clúster modesto en AWS EMR (3 nodos *m5.xlarge* con 4 vCPUs cada uno) cuesta aproximadamente \$1.20/hora, o \$864/mes operando 24/7. Databricks, que simplifica significativamente la operación, comienza en \$0.40/DBU adicional al costo del cómputo subyacente, elevando costos mensuales fácilmente sobre \$2,000 para cargas de trabajo continuas. Estos números asumen utilización constante; en la práctica, las PYMEs raramente necesitan procesamiento 24/7, pero la arquitectura de Spark incentiva mantener clústeres activos para evitar tiempos de inicio (*warm-up*) de 5-10 minutos.

La Revolución de las Herramientas In-Process

DuckDB, Polars y Daft representan un paradigma radicalmente diferente: procesamiento *in-process* de alto rendimiento. DuckDB, un motor SQL embebido inspirado en SQLite, pero optimizado para análisis, ejecuta consultas directamente en el proceso de la aplicación, sin necesidad de servidores externos. Polars, escrito en Rust con bindings para Python, ofrece un *DataFrame* API similar a Pandas, pero con optimizaciones de consulta y ejecución paralela automática. **Daft**, el más reciente, combina el procesamiento local de Polars con capacidades distribuidas opcionales para datasets multi-modales.

La ventaja económica es dramática. DuckDB y Polars operan en una única máquina, eliminando costos de infraestructura distribuida. Una instancia EC2 *c6i.4xlarge* (16 vCPUs, 32GB RAM) cuesta \$0.68/hora (\$489/mes), procesando eficientemente hasta 500GB de datos Parquet. Para cargas puntuales, el modelo **serverless** es aún más económico: AWS Lambda con contenedores personalizados ejecutando DuckDB cuesta \$0.0000166667 por GB-segundo, permitiendo procesar 100GB en 5 minutos por aproximadamente \$0.50. Esta diferencia de tres órdenes de magnitud (\$0.50 vs \$1,440 mensual de Spark) no es trivial para PYMEs.

Benchmarks de Rendimiento: Derribando Mitos

Un mito persistente es que Spark es inherentemente más rápido. Benchmarks independientes contradicen esto categóricamente. En el TPC-H benchmark (consultas analíticas estándar), DuckDB procesa *queries* sobre 100GB en una máquina de 64GB RAM entre 2-10x más rápido que Spark en un clúster de 8 nodos. Polars demuestra resultados similares. ¿Cómo es posible?

La respuesta radica en la sobrecarga de la distribución. Spark debe serializar datos entre nodos (*overhead* de red), mantener sincronización entre ejecutores y gestionar

tolerancia a fallos mediante checkpointing. Para datasets que caben en la RAM de una máquina moderna (hasta ~500GB con instancias de alto memoria), estas operaciones son puro desperdicio. DuckDB y Polars explotan vectorización SIMD (*Single Instruction, Multiple Data*), procesando miles de valores por ciclo de CPU, y optimizaciones de compilación que Spark, diseñado para compatibilidad con JVM, no puede igualar.

El punto de inflexión ocurre cuando los datos superan la capacidad de una sola máquina. Para datasets de 5TB+, Spark distribuye eficientemente la carga. Sin embargo, instancias modernas con 1-2TB de RAM (AWS X2idn) son ahora económicamente viables, retrasando este umbral significativamente.

Casos de Uso: El Contexto es Rey

Para una PYME con 500GB de órdenes de trabajo históricas (similar al caso EERSSA), las herramientas modernas son superiores en todos los aspectos. Un pipeline típico—ingestar PDFs, extraer texto, clasificar con ML, agregar métricas—se ejecuta en Polars en 20-30 minutos en una máquina de \$500/mes. El mismo proceso en PySpark requiere configurar un clúster, particionar datos manualmente y depurar errores de serialización, completándose en 45-60 minutos con costos 4x superiores.

PySpark recupera ventaja en tres escenarios específicos:

Streaming en tiempo real a escala masiva: Spark Structured Streaming procesa millones de eventos/segundo con garantías exactamente-una-vez (*exactly-once*). Alternativas como Flink compiten aquí, pero para organizaciones ya invertidas en el ecosistema Spark, la integración es ventajosa.

Machine Learning distribuido: MLlib de Spark entrena modelos sobre datasets que no caben en memoria de una máquina. Aunque frameworks como Ray y Dask ofrecen alternativas, la madurez de MLlib en producción es superior.

Integración con ecosistemas **legacy**: Organizaciones con inversiones masivas en Hadoop, Hive o Delta Lake sobre Spark enfrentan costos de migración prohibitivos. Aquí, PySpark es la ruta de menor resistencia.

El Factor Humano: Talento y Curva de Aprendizaje

Un costo oculto crucial es el capital humano. PySpark requiere ingenieros con conocimientos de programación distribuida, tuning de Spark (configurar particiones, memoria de ejecutores, shuffles) y depuración de errores crípticos de JVM. El salario promedio de un ingeniero Spark senior supera \$120,000 anuales en EE.UU. DuckDB y Polars, con APIs intuitivas cercanas a Pandas, pueden ser utilizados eficazmente por analistas de datos con salarios de \$70,000-\$80,000. Para una PYME, esta diferencia de \$40,000 anuales financia toda su infraestructura de datos.

Conclusión: Pragmatismo sobre Prestigio

La pregunta "¿PySpark o herramientas modernas?" es fundamentalmente una pregunta de escala y complejidad. Para el 80% de las PYMEs procesando gigabytes a terabytes de datos, DuckDB, Polars o Daft ofrecen rendimiento superior, costos dramáticamente menores y simplicidad operativa. La recomendación es clara: iniciar con herramientas *in-process* y migrar a Spark solo cuando los datos superen 2-5TB o se requiera streaming distribuido complejo.

La industria tecnológica sufre de "*cargo cult engineering*"—adoptar tecnologías porque empresas grandes las usan, ignorando contextos radicalmente diferentes. Netflix necesita Spark para 100PB de datos; una distribuidora eléctrica con 500GB no. El verdadero profesionalismo en ingeniería de datos reside en seleccionar la herramienta mínimamente suficiente, no la maximalmente impresionante.

Para EERSSA y organizaciones similares, la arquitectura óptima es un híbrido pragmático: Polars/DuckDB para ETL batch, Delta Lake para almacenamiento (sin Spark), y Marimo/Streamlit para interfaces. Reservar PySpark para el día—si llega—en que los datos justifiquen su complejidad. Hasta entonces, la simplicidad es la sofisticación suprema.

Anexo II: Reflexión acerca del pronóstico de fallas eléctricas

El siguiente texto es de autoría propia, se ha utilizado la herramienta IA Generativa *Claude Sonnet 4.5* (2025) para mejorar la estructura y redacción del texto, sin embargo, las ideas fundamentales, incluyendo la investigación de los modelos planteados, son propias del autor.

Visión y Alcance del Modelo Predictivo

El desarrollo del presente trabajo de titulación trasciende la automatización de reportes operativos, aspirando a establecer las bases para un sistema predictivo integral que permita pronosticar las desconexiones fortuitas de las redes eléctricas con horizontes temporales de uno, tres e incluso seis meses. Si bien la generación automatizada de informes posee aplicaciones prácticas inmediatas para la gestión operativa, el verdadero potencial transformador reside en la capacidad de desarrollar modelos de pronóstico que orienten estratégicamente los esfuerzos de mantenimiento preventivo.

Un modelo predictivo robusto permitiría no solamente focalizar recursos limitados en las secciones más críticas de la red, sino también cuantificar objetivamente el impacto de las acciones preventivas implementadas tomando como referencia los modelos predictivos, así como los registros históricos. Esta capacidad de medición representa un cambio paradigmático: de una gestión reactiva basada en incidentes, hacia una administración proactiva fundamentada en evidencia empírica y análisis predictivo.

El desarrollo de un modelo predictivo efectivo requiere un análisis exhaustivo y multidimensional de los factores que influyen en el comportamiento de las redes eléctricas. El Sistema de Gestión de Órdenes de Trabajo, objeto central de este trabajo, constituye el primer pilar fundamental de esta visión integradora.

Perspectiva del Trabajador: Sistema de Gestión de Órdenes de Trabajo

Este sistema proporciona una metodología estructurada para capturar, almacenar y consultar las actividades de mantenimiento desde la perspectiva del personal operativo en campo. Representa la memoria institucional de las intervenciones realizadas, documentando qué se hizo, cuándo, dónde y con qué recursos. Sin embargo, esta perspectiva, aunque fundamental, es incompleta sin el contexto de la infraestructura física sobre la cual se realizan estas actividades.

Perspectiva de la Infraestructura: Limitaciones Actuales del Sistema SIG

El Sistema de Información Georreferenciado (SIG) existente proporciona una vista instantánea del estado presente de la red eléctrica. Cada modificación infraestructural (nuevos transformadores, reemplazo de postes, reconfiguración de circuitos) se registra en el sistema, pero este se actualiza sobrescribiendo la información anterior

sin mantener un historial de cambios. Esta arquitectura de "estado actual sin memoria" presenta una limitación crítica para el modelado predictivo.

El desafío metodológico es evidente: para maximizar el valor del conjunto de datos de entrenamiento, es imprescindible correlacionar las intervenciones realizadas (documentadas en órdenes de trabajo) con el estado preciso de la infraestructura en el momento de dichas intervenciones (que debería provenir del SIG). Sin esta sincronización temporal, el modelo carecería del contexto infraestructural necesario para identificar patrones causales significativos.

Necesidad de un Sistema SIG con Memoria Histórica

Por consiguiente, se identifica como necesidad prioritaria el desarrollo de un nuevo Sistema de Información Georreferenciado que, análogamente al sistema de órdenes de trabajo, capture, almacene y preserve la memoria histórica evolutiva de la infraestructura eléctrica. Esta es una tarea de considerable complejidad, dado que la red eléctrica experimenta modificaciones continuas y la información histórica que permitiría reconstruir esta memoria se encuentra dispersa en documentos sin formato estandarizado, análogos a la situación inicial de las órdenes de trabajo.

La magnitud de este desafío excede incluso la complejidad del sistema de órdenes de trabajo, pues requiere no solo capturar información prospectiva, sino reconstruir retrospectivamente el estado de la red en múltiples puntos temporales del pasado, integrando fuentes documentales heterogéneas y frecuentemente incompletas.

Delimitación del Caso de Estudio: Alimentador Primario Zamora I

Ante las limitaciones descritas, resulta metodológicamente prudente delimitar el alcance inicial del modelo predictivo a una sección de la red eléctrica que presente características favorables para el análisis. Específicamente, se busca un segmento que haya experimentado la menor cantidad de modificaciones infraestructurales durante los últimos cinco años, minimizando así la complejidad introducida por la naturaleza evolutiva de la red.

Características del Alimentador Zamora I

Basándose en el conocimiento técnico acumulado en la organización, el Alimentador Primario Zamora I emerge como el candidato óptimo para este estudio piloto. Este circuito eléctrico presenta características particulares que lo hacen especialmente apropiado:

- **Extensión y trazado definido:** Se origina en la Central Eléctrica "Carlos Mora" y se extiende por aproximadamente 24 kilómetros hasta la ciudad de Zamora (EERSSA, 2025), siguiendo principalmente el corredor de la vía de acceso a la ciudad.

- Segmentación natural del territorio: El alimentador atraviesa tres zonas claramente diferenciadas con perfiles de riesgo distintos:
 - Sección vial: Tramos adyacentes a la carretera principal
 - Sección montañosa: Atravesando terreno accidentado con vegetación densa
 - Sección urbana: Dentro del perímetro de la ciudad de Zamora
- Estabilidad demográfica y económica: La zona no ha experimentado crecimiento poblacional significativo ni desarrollo económico acelerado durante el período de estudio, lo que implica una demanda eléctrica relativamente estable y mínimas modificaciones infraestructurales para expansión de cobertura.
- Configuración infraestructural estable: Las características físicas y topológicas del alimentador han permanecido relativamente invariantes, facilitando el análisis temporal sin las complicaciones que introducirían reconfiguraciones mayores.

Estas condiciones favorables convierten al Alimentador Zamora I en un caso de estudio ideal que, una vez modelado exitosamente, puede servir como referencia metodológica para extender el análisis hacia otros alimentadores con características similares en la red de la EERSSA.

Perspectiva Climatológica: El Factor Ambiental

Las condiciones meteorológicas ejercen influencia significativa sobre el desempeño de las redes eléctricas, constituyendo una tercera perspectiva fundamental para el modelo predictivo.

Mecanismos de Afectación

La precipitación pluvial impacta la red eléctrica mediante múltiples mecanismos:

1. Crecimiento acelerado de vegetación: Las lluvias abundantes estimulan el desarrollo de especies vegetales como enredaderas y bambú, que pueden alcanzar e interferir con el tendido eléctrico, generando riesgos de cortocircuito o deterioro del aislamiento.
2. Descargas atmosféricas: Los eventos de tormenta eléctrica incrementan la probabilidad de impactos directos de rayos sobre la infraestructura eléctrica. Aunque existen sistemas de protección (pararrayos, aisladores), la intensidad de ciertas descargas puede superar las capacidades de estos dispositivos, causando fallas.
3. Deterioro de materiales: La humedad sostenida acelera procesos de corrosión en componentes metálicos y degradación de aisladores.

Fuente de Datos Climatológicos: Estación Científica San Francisco

Afortunadamente, en las proximidades del Alimentador Zamora I se encuentra la Estación Científica "San Francisco", una instalación de investigación que mantiene registros meteorológicos detallados de la zona. Actualmente se encuentran en curso gestiones de colaboración interinstitucional con la esperanza de que esta valiosa información climatológica histórica pueda ser compartida para el desarrollo del modelo predictivo. La disponibilidad de estos datos permitiría correlacionar eventos de precipitación con incidencias en la red, identificando patrones estacionales y umbrales críticos de precipitación que incrementan la probabilidad de fallas.

Variables del Modelo Predictivo

A partir del análisis multidimensional expuesto, se identifican las siguientes variables para el modelo de pronóstico del Alimentador Zamora I:

Variables Independientes (Predictoras)

1. Actividades Preventivas (tiempo): Horas-hombre invertidas en mantenimiento preventivo (poda de vegetación, inspección de aisladores, limpieza de equipos, etc.). Esta variable captura el esfuerzo proactivo destinado a mantener en condiciones óptimas el tendido eléctrico.
2. Ubicación Geográfica (categórica): Segmentación del alimentador en tres zonas (vial, montañosa, urbana). Cada zona presenta perfiles de riesgo diferenciados: la zona montañosa enfrenta mayor interferencia vegetal, la zona vial está expuesta a accidentes vehiculares, y la zona urbana presenta mayor densidad de conexiones con potencial de manipulación inadecuada.
3. Índice de Vegetación (cuantitativa): Mediante un recorrido sistemático de los 24 kilómetros del alimentador, se puede elaborar un censo de la vegetación existente en proximidad a las líneas eléctricas, clasificando especies, altura, distancia al conductor y tasa de crecimiento estimada. Este censo puede actualizarse periódicamente para construir una serie temporal del riesgo vegetal.
4. Precipitación Pluvial (cuantitativa): Registro histórico de precipitaciones (mm/día) en el sector del Alimentador Zamora I, obtenido idealmente de la Estación Científica San Francisco o de fuentes meteorológicas oficiales.

Variable Dependiente (Respuesta)

Tiempo de Desconexión (correctivo): Horas totales de interrupción del servicio eléctrico causadas por fallas no programadas, que requieren intervención correctiva. Esta métrica cuantifica el impacto operacional de las fallas y constituye el objetivo principal del pronóstico.

Enfoque Metodológico: Análisis de Series Temporales Multivariadas

Justificación del Enfoque Temporal

La naturaleza temporal de las variables identificadas sugiere que un primer acercamiento mediante análisis de series temporales resulta metodológicamente apropiado. Dos de los factores clave, actividades preventivas y precipitación pluvial, exhiben efectos que se manifiestan con desfases temporales:

El mantenimiento preventivo realizado hoy reduce la probabilidad de fallas en los meses subsiguientes, mostrando un efecto retardado (*lag*).

La precipitación pluvial presenta patrones de cierta regularidad en regiones tropicales como Ecuador, aunque sin la estacionalidad marcada de las cuatro estaciones en latitudes templadas.

Estas características temporales hacen que los valores pasados de las variables predictoras contengan información relevante para pronosticar valores futuros de la variable de respuesta.

Marco Conceptual: Estacionariedad

El análisis de series temporales mediante modelos ARIMA (*Autoregressive Integrated Moving Average*) descansa fundamentalmente sobre el supuesto de estacionariedad. Una serie temporal es débilmente estacionaria cuando su media, varianza y auto covarianza permanecen constantes a lo largo del tiempo. Series no estacionarias que exhiben tendencias, varianza cambiante o patrones dependientes del tiempo violan los supuestos de ARIMA y pueden conducir a regresiones espurias (relaciones aparentes, pero estadísticamente inválidas) (Wei, 2006).

Antes de aplicar cualquier modelo, será necesario evaluar la estacionariedad de las series mediante pruebas estadísticas (Dickey-Fuller Aumentado, KPSS) y aplicar transformaciones apropiadas (diferenciación, logaritmos) cuando sea necesario.

Modelos Candidatos para Series Temporales Multivariadas

Dado que el modelo contempla múltiples variables predictoras, se deben considerar enfoques específicos para series temporales multivariadas:

1. ARIMAX (ARIMA con Variables Exógenas)

El modelo ARIMAX constituye una extensión natural de ARIMA que incorpora variables exógenas (predictoras externas) además de la estructura autorregresiva de la variable de respuesta (Hyndman & Athanasopoulos, 2021). Este enfoque asume que:

- Las variables exógenas (tiempo preventivo, precipitación, índice vegetal) influyen sobre el tiempo de desconexión
- La relación es unidireccional: las predictoras afectan a la respuesta, pero no viceversa
- No existe estacionalidad pronunciada en los datos

2. SARIMAX (ARIMAX Estacional)

Si el análisis exploratorio revela patrones estacionales significativos (por ejemplo, variaciones sistemáticas entre estaciones lluviosas y secas), el modelo SARIMAX (Seasonal ARIMAX) proporciona el marco apropiado, incorporando componentes estacionales adicionales que capturan fluctuaciones periódicas regulares.

3. VAR (Vector Autoregression - Autorregresión Vectorial)

Cuando las relaciones entre variables son potencialmente bidireccionales, el modelo VAR resulta más apropiado (Asian Institute of Management, 2025). VAR trata todas las variables como endógenas, permitiendo que:

- El tiempo preventivo afecte al tiempo de desconexión (relación esperada)
- El tiempo de desconexión también pueda influir en la planificación del tiempo preventivo futuro (feedback loop)

Esta flexibilidad captura dinámicas más complejas del sistema, aunque requiere mayor cantidad de datos para su estimación confiable.

4. Enfoques Bayesianos

Alternativamente, se pueden explorar técnicas bayesianas para el pronóstico, que ofrecen ventajas particulares:

- Incorporación natural de incertidumbre mediante distribuciones de probabilidad
- Capacidad de integrar conocimiento previo (conocimientos informativos basados en experiencia operativa)
- Cuantificación probabilística de los pronósticos (intervalos de credibilidad)

Modelos como BSTS (*Bayesian Structural Time Series*) o Prophet (desarrollado por Facebook) podrían proporcionar alternativas robustas, especialmente cuando los datos presentan irregularidades o valores atípicos.

Más Allá del Pronóstico Temporal: Localización Espacial de Fallas

El pronóstico del tiempo de desconexión, aunque valioso, constituye solo una dimensión del problema completo. La localización geográfica de las fallas pronosticadas posee implicaciones operativas significativas que deben ser modeladas:

Impacto Diferencial por Ubicación

Una falla al inicio del alimentador (cerca de la Central Carlos Mora) puede afectar a todo el circuito aguas abajo, interrumpiendo el servicio a miles de usuarios, mientras que una falla en un ramal terminal dentro de la ciudad de Zamora impacta a un número considerablemente menor de clientes. Por lo tanto, el modelo predictivo debe incorporar no solo cuándo es probable que ocurra una falla, sino también dónde es más probable que se materialice.

Modelado Espaciotemporal

Este componente espacial puede abordarse mediante:

1. Modelos de riesgo por segmento: Dividir el alimentador en segmentos de longitud uniforme (por ejemplo, 1 km) y estimar probabilidades de falla diferenciadas para cada segmento, considerando sus características particulares (vegetación circundante, historial de incidencias, accesibilidad para mantenimiento).
2. Análisis de puntos calientes: Identificar mediante análisis histórico las ubicaciones que concentran mayor frecuencia de fallas y caracterizar los factores que las hacen especialmente vulnerables.
3. Simulaciones de Monte Carlo: Una vez establecido el modelo predictivo de tiempos de desconexión y probabilidades espaciales, se pueden realizar simulaciones estocásticas que generen múltiples escenarios posibles de fallas, permitiendo evaluar:
 - Distribuciones de probabilidad de interrupciones por zona
 - Impacto agregado esperado (usuarios-hora de interrupción)
 - Priorización de intervenciones preventivas según retorno de inversión esperado

Perspectiva de Desarrollo: Un Programa de Investigación de Largo Plazo

El desarrollo completo del modelo predictivo integral descrito (incorporando análisis temporal multivariado, componentes espaciales y simulación estocástica) constituye un esfuerzo de investigación de considerable magnitud. Fácilmente podría abarcar uno o varios trabajos de titulación paralelos, cada uno focalizándose en componentes específicos:

- Proyecto 1: Reconstrucción histórica del SIG y sincronización con órdenes de trabajo
- Proyecto 2: Censo vegetal y modelado de crecimiento de vegetación
- Proyecto 3: Desarrollo y validación de modelos de series temporales multivariadas
- Proyecto 4: Modelo espacial de riesgo y simulación de escenarios

Valor Acumulativo del Conocimiento

Cada avance hacia este ambicioso macroproyecto, por incremental que sea, contribuye significativamente a la comprensión del comportamiento de los factores que influyen en el desempeño del tendido eléctrico. Este conocimiento acumulativo será de gran utilidad al momento de:

- Planificar intervenciones: Optimizando el calendario y ubicación de actividades preventivas
- Asignar recursos: Fundamentando solicitudes presupuestarias con proyecciones cuantitativas de beneficio
- Evaluar desempeño: Estableciendo métricas objetivas para medir la efectividad de las estrategias implementadas
- Mejorar el servicio: Reduciendo la frecuencia y duración de interrupciones, cumpliendo así con el fin último de la EERSSA de proporcionar servicio eléctrico confiable y de calidad a la ciudadanía.

El presente trabajo de titulación, centrado en el Sistema de Gestión de Órdenes de Trabajo, constituye la piedra angular de este programa de investigación de largo plazo, estableciendo la infraestructura de datos y la metodología analítica sobre la cual se construirán los componentes subsiguientes del modelo predictivo integral.